AskData项目代码

前言

有需要的可以下载**压缩包和PDF文件**，但是请不要倒卖，AskData是题主在学校做了一年多的横向项目结合我的实习经历包装而来，**项目有落地的业务场景和闭环的逻辑链**，本身具有一定的难度，整理和包装不易，请尊重题主的劳动成果，感谢🙏（最后有用的话记得多推荐一下😁）

版本更新&代码下载

> 代码目前已更新至V2.0.0，更新时间：2026年8月22日
>
> 无法下载的话📕私信联系）

版本号	功能实现	代码下载
V1.0.0	完成AskData初步链路的实现，实现创建Schema索引->Schema两阶段检索->CoT思维链生成->SQL语句生成->MCP路由执行代码的Pipeline。	
V1.1.0	增加长短期记忆和意图路由的单元py代码实现，在原本的Pipeline增加这两个模块。	askdata_agent.zip 404.20KB
V1.2.0	增加反思记忆模块	askdata_agent_demo.zip 484.44KB
V2.0.0	完成前后端的初步实现，完成上下文聚合和动态路由、索引建立与Schema检索、Human-in-the-loop意图澄清，基础Skill问答与问数、MCP工具管理与权限校验、长短记忆机制。	

PDF文件下载

求反馈（重要‼️‼️‼️）

大家觉得这个项目怎么样？用过这个项目的或者看完项目的可以在小红书找我给我反馈一下用着怎么样（想收集一波建议，也找一下优化方向）

V2版本代码解析

> V2版本代码在V1代码基础上优化了冗余的流程，并且引入了最新的Agent技术，具有完整的前后端系统。

🔥 **项目技术栈：**

- **前端：**Vue 3 + TypeScript + Vite
- **后端：**FastAPI + LangGraph + REST API
- **数据库：**DuckDB + Milvus（目前为本地Json存储）
- **AI大模型：**通义千问系列模型 + text-embedding-v4 + Qwen3-Rerank

环境准备

本地隔离环境

安装后端：

代码块

```
py -3.11 -m venv backend\.venv
backend\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
```

安装前端：

代码块

```
cd frontend
.\npm-local.cmd install
cd ..
```

启动后端：

代码块

```
cd backend
.\.venv\Scripts\python.exe run.py
```

新开一个终端，在项目根目录启动前端：

代码块

```
cd frontend
.\npm-local.cmd run dev
```

全局环境

安装依赖：

代码块

```
py -3.11 -m pip install -r backend\requirements.txt

cd frontend
npm install
cd ..
```

启动后端：

代码块

```
cd backend
py -3.11 run.py
```

新开一个终端，在项目根目录启动前端：

代码块

```
cd frontend
npm run dev
```

访问地址：

- 前端：`http://127.0.0.1:5173`
- 后端：`http://127.0.0.1:8000`
- API 文档：`http://127.0.0.1:8000/docs`

项目整体架构

项目整体架构

```
askdata_studio/
├─ frontend/                                  Vue 3 前端
│  ├─ src/
│  │  ├─ App.vue                              主页面、对话与上下文配置
```

```
5  │   │   ├── api.ts                          后端接口调用
6  │   │   ├── types.ts                        前后端数据类型
7  │   │   ├── components/
8  │   │   │   └── ResultTableCard.vue         表格卡片、分页与Excel导出
9  │   │   ├── main.ts                         Vue启动入口
10 │   │   └── style.css                       全局页面样式
11 │   ├── package.json
12 │   └── vite.config.ts
13 │
14 ├── backend/                                FastAPI 后端
15 │   ├── app/
16 │   │   ├── main.py                         FastAPI应用入口
17 │   │   ├── api/
18 │   │   │   └── routes.py                   查询、记忆、澄清与工具接口
19 │   │   │
20 │   │   ├── services/                       应用服务与会话管理
21 │   │   │   ├── askdata_service.py          查询服务入口
22 │   │   │   ├── session_context.py          会话上下文管理
23 │   │   │   ├── short_term_memory.py        短期记忆与异步摘要
24 │   │   │   ├── session_archive.py          会话归档
25 │   │   │   └── memory_store.py             用户保存字段与表格
26 │   │   │
27 │   │   ├── workflows/                      LangGraph工作流编排
28 │   │   │   ├── query_graph.py              问答、问数、澄清与执行流程
29 │   │   │   ├── state.py                    工作流状态定义
30 │   │   │   └── result_builder.py           查询结果构建
31 │   │   │
32 │   │   ├── skills/                         可扩展任务Skill
33 │   │   │   ├── data_qa/                    数据问答Skill
34 │   │   │   ├── database_query/             数据库查询Skill
35 │   │   │   └── registry.py                 Skill注册与加载
36 │   │   │
37 │   │   ├── retrieval/                      Schema检索与Schema图构建
38 │   │   │   ├── service.py                  混合检索、RRF与Rerank
39 │   │   │   ├── store.py                    本地Schema索引存储
40 │   │   │   └── graph.py                    表字段关系图构建
41 │   │   │
42 │   │   ├── querying/                       SQL生成、执行与结果分析
43 │   │   │   ├── single_database_agent.py    单库查询智能体
44 │   │   │   ├── duckdb_engine.py            DuckDB SQL执行器
45 │   │   │   └── response_generator.py       数据分析与文字说明生成
46 │   │   │
47 │   │   ├── mcp_runtime/                    MCP工具运行时
48 │   │   │   ├── client.py                   MCP工具发现与调用
49 │   │   │   ├── server.py                   MCP工具注册
50 │   │   │   └── tools/
51 │   │   │       ├── time_tools.py           当前时间与日期范围工具
```

```
52  │   │   │       └── database_tools.py          只读数据库查询工具
53  │   │   │
54  │   │   ├── security/                          用户认证与权限控制
55  │   │   │   ├── auth.py                        用户身份认证
56  │   │   │   └── access_control.py              数据库、表级访问控制
57  │   │   │
58  │   │   ├── preprocessing.py                   上下文聚合、动态路由与检索意图提取
59  │   │   ├── model_client.py                    大模型、Embedding与Rerank调用
60  │   │   ├── database.py                        静态Schema与表关系定义
61  │   │   ├── models.py                          API入参与出参模型
62  │   │   ├── config.py                          模型、记忆与检索参数配置
63  │   │   └── errors.py                          工作流异常定义
64  │   │
65  │   ├── data/
66  │   │   ├── databases/
67  │   │   │   └── askdata_mock/                  一个文件夹代表一个数据库
68  │   │   │       ├── orders_current.csv
69  │   │   │       ├── orders_history.csv
70  │   │   │       ├── customers.csv
71  │   │   │       ├── products.csv
72  │   │   │       └── sales_targets.csv
73  │   │   ├── schema_store.json                  本地Schema索引
74  │   │   ├── saved_memories.json                用户保存的长期记忆
75  │   │   └── session_archive.db                 会话与短期记忆归档
76  │   │
77  │   ├── tests/                                 后端单元测试与链路测试
78  │   ├── run.py                                 后端启动入口
79  │   └── requirements.txt
80  │
81  └── README.md                                  项目说明与启动文档
```

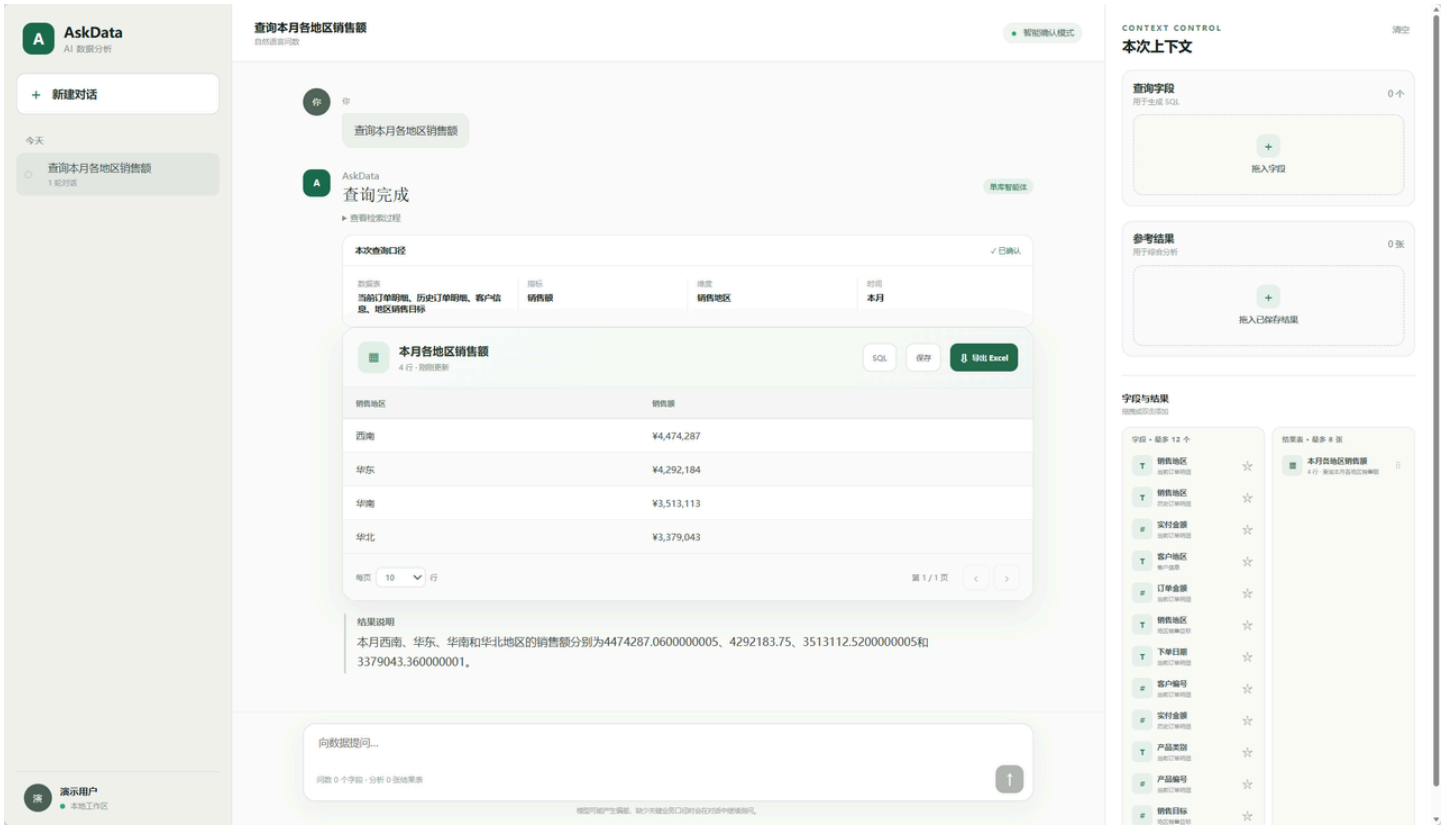
前端界面

> 目前就分为两个页面，一个登录页，一个操作页

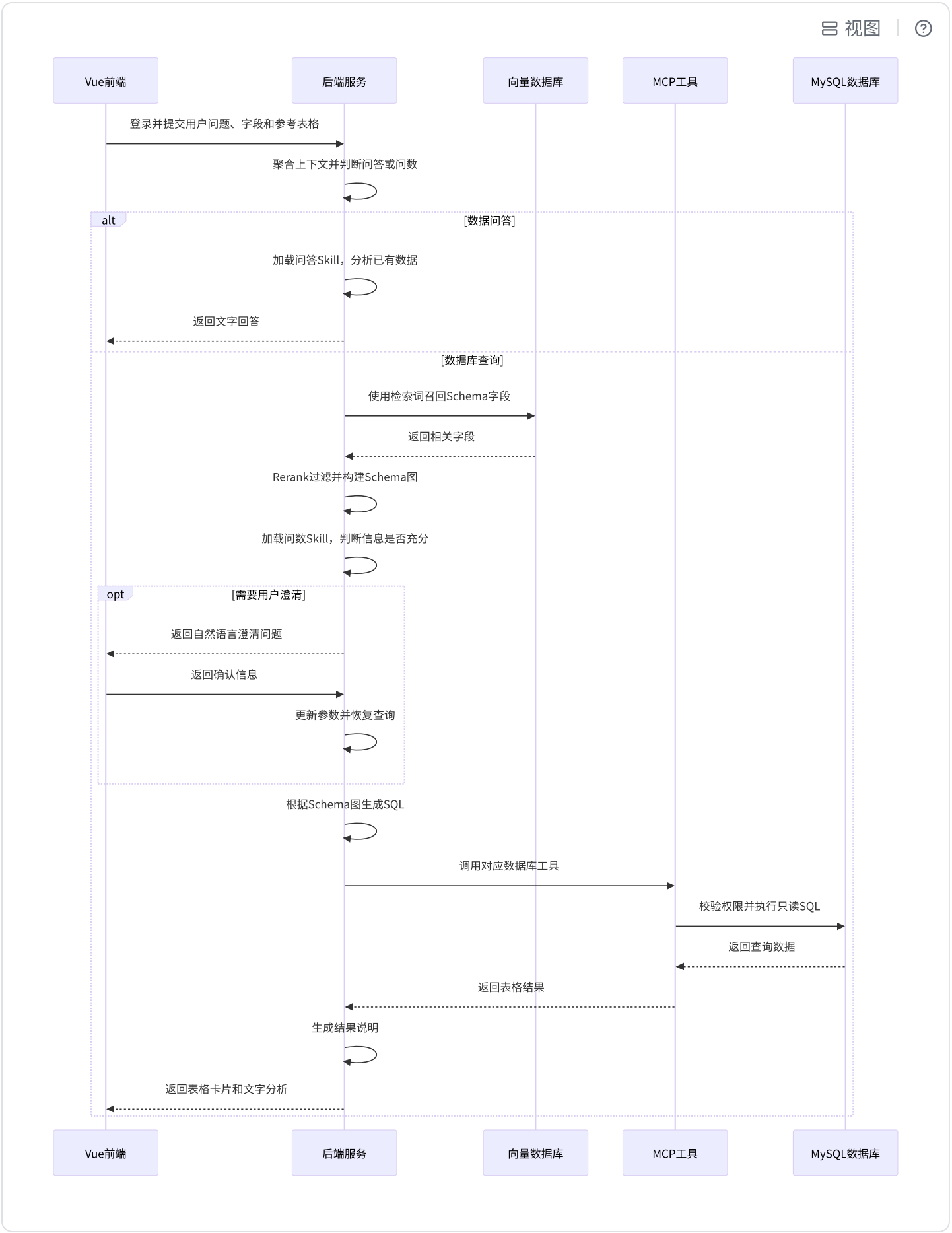
登录界面：


AskData
AI DATA ASSISTANT
用自然语言，读懂你的数据。
字段级 Schema 检索、权限隔离和可保存的数据记忆，都从一个问题开始。
FastAPI · LangGraph · MCP
欢迎回来
登录 AskData Studio
账号
admin
密码
登录
演示账号
admin
管理员 · 全部数据
admin123
sales
普通用户 · 隔离历史订单
sales123
Mock 登录仅用于本地学习，不适合生产环境。

问答页面：


AskData
新建对话
查询本月各地区销售额
查询本月各地区销售额
查询完成
本月各地区销售额
销售地区
销售额
西南
¥4,474,287
华东
¥4,292,184
华南
¥3,513,113
华北
¥3,379,043
结果说明
本月西南、华东、华南和华北地区的销售额分别为4474287.0600000005、4292183.75、3513112.5200000005和3379043.360000001。
向数据提问...
CONTEXT CONTROL
本次上下文
查询字段
参考结果
字段与结果

后端主链路：



核心代码解析

上下文聚合&意图路由&关键词提取

🔥 **整体业务逻辑如下：**

1. 获取当前会话最近6轮轻量上下文。
2. 将当前问题和上下文一次性交给预处理模型。
3. 模型结合上下文补全当前问题，并判断请求类型：
   - `database_query`：查询新的数据库数据；
   - `data_qa`：分析或解释已有数据；
   - `direct_response`：闲聊、普通问答或意图不明确的灰色请求。
4. `database_query` 同时生成独立问题并提取 Schema 检索词。
5. `data_qa` 进入数据分析流程；`direct_response` 由当前模型直接回答或询问用户。
6. 将路由结果写入 LangGraph 状态，进入对应流程。

核心调用链

```
SessionContext.route_context          获取轻量上下文
→ RequestPreprocessor.prepare         聚合上下文、判断意图、提取检索词
→ QueryWorkflow._preprocess           写入工作流状态
→ LangGraph 条件路由
   ├─ direct_response                 直接回复
   ├─ data_qa                         分析已有数据
   └─ database_query                  检索Schema并查询数据库
```

获取轻量上下文

文件：`backend/app/services/session_context.py`

代码块

```python
def route_context(
    self,
    session_id: str,
    workspace: dict[str, Any] | None = None,
) -> str:
    # 获取最近6轮轻量对话
    recent_items = self._memory_items(session_id)[
        -self.route_context_turns:
    ]

    context = {
        "recent_turns": [
```

```
            self._route_item(item)
            for item in recent_items
        ],
    }

    # 用户选中的表格只加入表头，不加入数据行
    selected_headers = self._selected_table_headers(
        session_id,
        workspace or {},
    )

    if selected_headers:
        context["selected_table_headers"] = selected_headers

    return (
        json.dumps(context, ensure_ascii=False)
        if recent_items or selected_headers
        else ""
    )
```

路由上下文只包含近期问题、回答摘要、处理状态、表头和行数，不读取短期记忆摘要，也不传递完整表格数据。

调用预处理模型

文件：`backend/app/preprocessing.py`

代码块

```
def prepare(
    self,
    query: str,
    recent_context: str,
) -> PreparedRequest:
    system = """
一次完成以下任务：

1. 结合近期上下文理解当前问题；
2. 判断请求类型；
3. 问数时补全独立问题并提取Schema检索词；
4. 其他请求直接回答或询问用户。
"""

    user = (
        f"当前问题：{query}\n"
        f"近期轻量上下文：{recent_context or '无'}"
    )
```

```python

    try:
        # 一次模型调用完成路由和关键词提取
        payload = self.model_client.chat_json(
            system,
            user,
        )

        return self._parse(payload, query)

    except RuntimeError as exc:
        # 模型不可用时不进入数据库查询流程
        logger.warning(
            "route_fallback_used error=%s",
            exc,
        )

        return PreparedRequest(
            action="direct_response",
            confidence=0.0,
            reason="预处理模型不可用",
            response="大模型服务暂时不可用，请稍后重试。",
            source="model_unavailable_fallback",
        )
```

这一步只调用一次大模型，同时完成上下文理解、意图判断和检索词提取。

核心入参：

代码块

```
{
  "query": "那华东呢？",
  "recent_context": {
    "recent_turns": [
      {
        "user_message": "查询本月各地区销售额",
        "route": "database_query",
        "status": "completed",
        "result_title": "本月各地区销售额",
        "columns": ["销售地区", "销售额"],
        "row_count": 4
      }
    ]
  }
}
```

核心出参：路由结果：

```
代码块
1  {
2    "action": "database_query",
3    "confidence": 0.95,
4    "reason": "需要查询华东地区的新数据",
5    "standalone_query": "查询本月华东地区销售额",
6    "rewritten": true,
7    "retrieval": {
8      "explicit_terms": ["本月", "华东", "销售额"],
9      "semantic_query": "查询本月华东地区销售额",
10     "metrics": ["销售额"],
11     "dimensions": ["地区"],
12     "filters": ["华东"],
13     "time_expressions": ["本月"],
14     "operations": ["汇总"]
15   }
16 }
```

其中：

- `action` 决定后续流程；
- `standalone_query` 是结合上下文补全后的独立问题；
- `explicit_terms` 用于BM25检索；
- `semantic_query` 用于Embedding向量检索；
- `retrieval` 中的其他字段用于描述查询指标、维度和筛选条件。

如果请求意图不明确，模型可以直接返回：

```
代码块
1  {
2    "action": "direct_response",
3    "confidence": 0.72,
4    "reason": "无法确定是查询新数据还是分析已有结果",
5    "response": "你希望查询新的销售数据，还是分析已有的销售结果？",
6    "response_type": "clarification"
7  }
```

LangGraph条件路由

文件：`backend/app/workflows/query_graph.py`

代码块

```
builder.add_conditional_edges(
    "preprocess",

    # 根据模型返回的action选择后续流程
    lambda state: {
        "direct_response": "respond_directly",
        "data_qa": "answer_qa",
        "database_query": "retrieve_schema",
    }[state["intent"]["action"]],

    {
        "respond_directly": "respond_directly",
        "answer_qa": "answer_qa",
        "retrieve_schema": "retrieve_schema",
    },
)
direct_response → 直接返回模型回答
data_qa         → 加载短期记忆并分析已有数据
database_query  → 检索Schema并执行数据库查询
```

索引建立&Schema检索&Schema图构建

🔥 **索引建立为离线过程，Schema检索和图构建都是在线过程**

1. 以字段为单位建立Schema索引，每个字段包含关键词文本、向量文本和Rerank文本。
2. 预处理模型统一提取 `retrieval_terms` 。
3. BM25和Embedding使用同一组检索词召回候选字段。
4. 使用RRF融合两路召回结果，再用完整Query进行Rerank。
5. 只保留超过置信度阈值的字段，并加入用户确认字段。
6. 根据字段所属表和表间关系构建Schema图。
7. 将Schema图提供给后续SQL生成流程。

代码块

```
SchemaIndex.ensure_built            建立字段级三级索引
→ SchemaIndex.retrieve               BM25与Embedding混合检索
→ RRF                                融合两路召回结果
→ Rerank                             使用完整Query精排
→ SchemaIndex.include_workspace      加入用户确认字段
→ SchemaGraphBuilder.build           构建Schema图
```

建立字段级索引

文件：`backend/app/retrieval/service.py`

代码块

```
def _append_table_documents(
    self,
    output: list[dict[str, Any]],
    connection: Any,
    table: dict[str, Any],
) -> None:
    for field in table["fields"]:
        # 获取字段样例和数据分布
        samples = [
            row[0]
            for row in connection.execute(
                f'SELECT DISTINCT "{field["name"]}" '
                f'FROM "{table["id"]}" '
                f'WHERE "{field["name"]}" IS NOT NULL '
                f'LIMIT 5'
            ).fetchall()
        ]

        profile = self._field_profile(
            connection,
            table["id"],
            field,
        )

        # BM25检索文本
        keyword_text = " ".join([
            table["id"],
            table["label"],
            table["description"],
            field["name"],
            field["label"],
            field.get("description", ""),
            *field.get("aliases", []),
        ])

        # Embedding向量文本
        semantic_text = (
            f"{field['label']}属于{table['label']}。"
            f"表用途：{table['description']}。"
            f"字段含义：{field.get('description', '')}。"
            f"样例：{samples}。"
        )
```

```
43
44          # Rerank精排文本
45          rerank_text = (
46              f"字段: {table['id']}.{field['name']}; "
47              f"中文名: {field['label']}; "
48              f"所属表: {table['label']}; "
49              f"字段含义: {field.get('description', '')}; "
50              f"样例: {samples}; "
51              f"数据分布: {profile}。"
52          )
53
54          output.append({
55              "doc_id": f"{table['id']}.{field['name']}",
56              "database_id": table.get(
57                  "database",
58                  "askdata_mock",
59              ),
60              "table_id": table["id"],
61              "field_name": field["name"],
62              "field_label": field["label"],
63              "keyword_text": keyword_text,
64              "semantic_text": semantic_text,
65              "rerank_text": rerank_text,
66          })
```

每个字段包含三种检索表示：

代码块

```
keyword_text    → BM25关键词检索
semantic_text   → Embedding向量检索
rerank_text     → Rerank相关性精排
```

生成字段文档后，对 `semantic_text` 计算向量并保存：

代码块

```
semantic_texts = [
    item["semantic_text"]
    for item in raw_documents
]

vectors = self.model_client.embed(
    semantic_texts
)

```

```
    for raw, vector in zip(
        raw_documents,
        vectors,
    ):
        documents.append(
            FieldDocument(
                **raw,
                dense_vector=vector,
            )
        )

    self.store.replace_all(
        signature,
        documents,
        self.embedding_source,
    )
```

目前索引保存在本地 `LocalSchemaStore` 中。

混合检索与Rerank

文件： `backend/app/retrieval/service.py`

代码块

```
def retrieve(
    self,
    query: str,
    *,
    retrieval_terms: list[str],
    access_scope: AccessScope,
) -> dict[str, Any]:
    self.ensure_built()

    # 对检索词去重并组合为短文本
    terms = list(
        dict.fromkeys(retrieval_terms or [])
    )
    retrieval_text = " ".join(terms)

    # 只允许召回用户有权限查看的字段
    allowed_doc_ids = {
        document.doc_id
        for document in self.documents
        if access_scope.allows_table(
            document.database_id,
            document.table_id,
```

```
            )
        }

        # BM25和Embedding使用同一组检索词
        bm25 = self.store.bm25_search(
            retrieval_text,
            self.config.bm25_top_k,
            allowed_doc_ids,
        )

        dense = self.store.dense_search(
            self._query_vector(retrieval_text),
            self.config.dense_top_k,
            allowed_doc_ids,
        )

        # RRF融合两路召回排名
        fused = {}

        for rank_map in (
            self._rank_map(bm25),
            self._rank_map(dense),
        ):
            for index, rank in rank_map.items():
                fused[index] = (
                    fused.get(index, 0.0)
                    + 1 / (60 + rank)
                )

        candidates = sorted(
            fused.items(),
            key=lambda item: item[1],
            reverse=True,
        )[:self.config.rrf_top_k]

        # Rerank使用完整Query判断最终相关性
        rerank_documents = [
            self.documents[index].rerank_text
            for index, _ in candidates
        ]

        rerank_result = self.model_client.rerank(
            query,
            rerank_documents,
            len(rerank_documents),
        )

```

```
        # 低于阈值的字段不进入Schema图
        selected = [
            item
            for item in rerank_result
            if item[1]
            >= self.config.schema_recall_threshold
        ]

        return self._build_retrieval_result(
            selected,
            bm25,
            dense,
            fused,
        )
```

检索过程为：

代码块

```
retrieval_terms
├─→ BM25召回
└─→ Embedding召回
        ↓
      RRF融合
        ↓
完整Query → Rerank精排
        ↓
      阈值过滤
```

完整Query不会进入Embedding，只在Rerank阶段使用。

核心入参

代码块

```
{
  "query": "请结合前面的讨论，查询本月各地区销售额",
  "retrieval_terms": [
    "本月",
    "地区",
    "销售额"
  ],
  "access_scope": {
    "allowed_databases": ["askdata_mock"],
    "allowed_tables": [
      "orders_current",
```

```
        "customers"
      ]
    }
  }
```

加入用户确认字段(可选)

文件：`backend/app/workflows/query_graph.py`

代码块

```
retrieval = self.schema_index.include_workspace(
    retrieval,
    query_workspace,
    state.get("access_scope"),
)
```

用户在前端拖入的字段会直接加入召回结果：

代码块

```
{
  "source": "user_confirmed",
  "selection_reason": "user_confirmed",
  "score": 1.0
}
```

用户确认字段仍然需要通过Schema权限校验。

构建Schema图

文件：`backend/app/retrieval/graph.py`

代码块

```
def build(
    self,
    hits: list[dict[str, Any]],
    access_scope: AccessScope,
) -> dict[str, Any]:
    # 获取召回字段所属的数据表
    selected_tables = {
        hit["table_id"]
        for hit in hits
    }

```

```
        # 查找连接这些表的最短关系路径
        relations = self._shortest_path_relations(
            selected_tables,
            set(access_scope.allowed_tables),
        )

        graph_tables = set(selected_tables)

        for relation in relations:
            graph_tables.update([
                relation["left_table"],
                relation["right_table"],
            ])

        return {
            "database": self._database(
                graph_tables
            ),
            "tables": self._build_tables(
                graph_tables
            ),
            "fields": self._build_fields(
                hits,
                relations,
            ),
            "joins": relations,
        }
```

核心出参

代码块

```
{
  "database": "askdata_mock",
  "tables": [
    {
      "id": "orders_current",
      "label": "当前订单"
    },
    {
      "id": "customers",
      "label": "客户信息"
    }
  ],
  "fields": [
    {
```

```
        "id": "orders_current.paid_amount",
        "label": "销售额",
        "source": "retrieval",
        "score": 0.96
      },
      {
        "id": "customers.customer_level",
        "label": "客户等级",
        "source": "user_confirmed",
        "score": 1.0
      }
    ],
    "joins": [
      {
        "left_table": "orders_current",
        "left_field": "customer_id",
        "right_table": "customers",
        "right_field": "customer_id"
      }
    ]
  }
```

Schema图最终提供相关表、字段和关联条件，作为后续SQL生成的结构上下文和关联约束。

Human-in-the-loop意图澄清

🔥 **意图澄清基于Langgraph实现：**

1. 完成Schema检索并构建Schema图。
2. 将用户问题、Schema图、召回字段和已确认参数交给问数智能体。
3. 智能体判断信息是否足以生成准确SQL：
   - 信息充分：调用数据库工具执行SQL；
   - 存在会实质改变查询结果的歧义：返回澄清问题。
4. LangGraph通过 `interrupt` 暂停当前任务，并向前端返回澄清内容。
5. 用户使用自然语言回复后，系统将答案写入已确认参数。
6. 使用同一个 `task_id` 恢复LangGraph，并重新执行Schema检索和查询流程。

代码块

```
Schema检索并构建Schema图
→ SingleDatabaseAgent判断信息是否充分
```

```
    ├─ call_tool → 执行SQL
    └─ clarify   → LangGraph interrupt暂停
                        ↓
                    用户回复
                        ↓
                Command(resume)恢复
                        ↓
              更新confirmed_parameters
                        ↓
              重新检索Schema并执行SQL
```

判断歧义并暂停任务

文件：

代码块

```
backend/app/querying/single_database_agent.py
→ backend/app/workflows/query_graph.py
```

代码块

```python
# single_database_agent.py

def prepare(
    self,
    query: str,
    database: str,
    schema_graph: dict[str, Any],
    schema_context: str,
    retrieval: dict[str, Any],
    workspace: dict[str, Any],
    access_scope: dict[str, Any],
) -> dict[str, Any]:
    payload = {
        "query": query,
        "database": database,
        "schema_graph": schema_graph,

        # 提供召回字段和低置信度候选
        "retrieval": {
            "threshold": retrieval.get("threshold"),
            "selected_fields": retrieval.get(
                "hits",
                [],
            ),
```

```
            "low_confidence_candidates": retrieval.get(
                "low_confidence_candidates",
                [],
            ),
        },

        # 提供用户已经确认的字段和参数
        "confirmed_fields": workspace.get(
            "schema_fields",
            [],
        ),
        "confirmed_parameters": workspace.get(
            "confirmed_parameters",
            {},
        ),

        "schema_text": schema_context,
        "mcp_tools": tools,
    }

    decision = self.model_client.chat_json(
        system,
        json.dumps(payload, ensure_ascii=False),
    )

    if decision["action"] == "clarify":
        clarification = decision.get("clarification")

        # 代码只校验澄清结果的结构
        if (
            not isinstance(clarification, dict)
            or len(clarification.get("options") or []) < 2
        ):
            raise ValueError(
                "智能体返回的澄清信息不完整"
            )

        return {
            "action": "clarify",
            "clarification": clarification,
        }

    # 信息充分时调用MCP数据库工具
    return {
        "action": "call_tool",
        "tool_name": decision["tool_name"],
        "arguments": decision["arguments"],
```

```
    }


# query_graph.py

def _prepare_single_database(
    self,
    state: QueryState,
) -> dict[str, Any]:
    decision = self.single_database_agent.prepare(
        state["standalone_query"],
        state["database_names"][0],
        state["schema_graph"],
        state["schema_context"],
        state["retrieval"],
        state.get("workspace") or {},
        state.get("access_scope") or {},
    )

    if decision["action"] == "clarify":
        return {
            # 将澄清内容写入LangGraph状态
            "clarification": decision["clarification"],
            "direct_sql": "",
        }

    return {
        "clarification": None,
        "mcp_execution": decision["execution"],
    }


builder.add_conditional_edges(
    "prepare_single_database",

    # 有澄清内容时暂停，否则继续执行SQL
    lambda state: (
        "human_clarification"
        if state.get("clarification")
        else "execute_single_database"
    ),

    {
        "human_clarification": "human_clarification",
        "execute_single_database": "execute_single_database",
    },
```

```
118   )
```

是否存在关键业务歧义由问数智能体进行语义判断。代码不通过表名、字段名或关键词规则强制触发澄清，只负责校验模型返回格式和控制工作流走向。

模型返回的澄清结果示例：

代码块

```
1   {
2     "action": "clarify",
3     "reason": "当前订单和历史订单会产生不同结果",
4     "clarification": {
5       "parameter": "order_scope",
6       "question": "你希望查询当前订单还是历史订单？",
7       "reason": "不同订单范围会影响最终销售额",
8       "options": [
9         {
10          "id": "orders_current",
11          "label": "当前订单",
12          "description": "查询当前订单数据",
13          "recommended": true
14        },
15        {
16          "id": "orders_history",
17          "label": "历史订单",
18          "description": "查询历史订单数据",
19          "recommended": false
20        }
21      ]
22    }
23  }
```

用户回复并恢复任务

文件：

代码块

```
1   backend/app/workflows/query_graph.py
2   → backend/app/services/askdata_service.py
```

代码块

```
1   # query_graph.py
2
```

```
def _human_clarification(
    self,
    state: QueryState,
) -> dict[str, Any]:
    # 暂停工作流，恢复后得到用户选择
    response = interrupt(
        state.get("clarification") or {}
    )

    option_id = str(
        response.get("option_id")
    )

    workspace = dict(
        state.get("workspace") or {}
    )

    parameter = str(
        state["clarification"].get(
            "parameter",
            "other",
        )
    )

    # 将用户选择加入已确认参数
    workspace["confirmed_parameters"] = {
        **dict(
            workspace.get(
                "confirmed_parameters",
            )
            or {}
        ),
        parameter: option_id,
    }

    return {
        "workspace": workspace,
        "clarification": None,
        "direct_sql": "",
        "result": {},
    }


# 澄清完成后重新进行Schema检索
builder.add_edge(
    "human_clarification",
    "retrieve_schema",
```

```
)


# askdata_service.py

def clarify(
    self,
    task_id: str,
    option_id: str,
    user_id: str | None = None,
) -> QueryResult:
    task = self.tasks[task_id]

    # 校验用户是否拥有当前任务
    access_scope = self.access_controller.resolve(
        user_id
    )

    if task["user_id"] != access_scope.user_id:
        raise PermissionError(
            "无权访问该查询任务"
        )

    # 使用原task_id恢复LangGraph
    return self.workflow.invoke(
        Command(
            resume={
                "option_id": option_id,
            }
        ),
        task_id,
    )
```

用户可以直接使用自然语言回复：

代码块

```
用户：查询各地区销售额
系统：你希望查询当前订单还是历史订单？
用户：使用当前订单
```

系统将“当前订单”匹配为 `orders_current`，写入：

代码块

```
{
```

```
    "confirmed_parameters": {
      "order_scope": "orders_current"
    }
  }
```

然后恢复原任务，重新构建Schema图并继续执行SQL，不会重新创建一个独立查询任务。

问答&数据库查询&Skill

🔥
1. 系统启动时扫描 `backend/app/skills` 目录，加载问答Skill和问数Skill。
2. LangGraph根据路由结果选择对应链路：
   - `data_qa` 加载问答Skill，结合已有数据上下文生成文字回答；
   - `database_query` 加载问数Skill，结合Schema图生成SQL并调用MCP工具。
3. 问答Skill不提供数据库工具，只允许分析已有数据。
4. 问数Skill根据配置过滤可用MCP工具，并限制工具调用次数和模型输出动作。
5. SQL执行完成后，再调用结果整理模型生成标题和文字说明。

代码块

```
SkillRegistry                          启动时加载全部Skill
→ LangGraph根据action选择链路
   ├─ data_qa
   │    → data_qa Skill
   │    → ModelClient.chat
   │    → 生成文字回答
   │
   └─ database_query
        → database_query Skill
        → 获取MCP工具列表
        → ModelClient.chat_json
        → 生成SQL和工具参数
        → MCP执行SQL
        → 生成结果说明
```

Skill加载与任务分配

文件：

代码块

```
backend/app/skills/registry.py
```

```
→ backend/app/workflows/query_graph.py
```

代码块

```python
# registry.py

class SkillRegistry:
    def __init__(
        self,
        root: Path | None = None,
    ) -> None:
        self.root = (
            root
            or Path(__file__).resolve().parent
        )
        self._skills = self._load()

    def _load(
        self,
    ) -> dict[str, SkillDefinition]:
        output = {}

        # 扫描每个Skill目录中的config.json
        for config_path in self.root.glob(
            "*/config.json"
        ):
            directory = config_path.parent
            instruction_path = (
                directory / "SKILL.md"
            )

            config = json.loads(
                config_path.read_text(
                    encoding="utf-8"
                )
            )

            name = config["name"]

            output[name] = SkillDefinition(
                name=name,
                description=config["description"],

                # SKILL.md作为大模型系统提示词
                instructions=instruction_path.read_text(
                    encoding="utf-8"
```

```
                ).strip(),

                allowed_tools=tuple(
                    config.get(
                        "allowed_tools",
                        [],
                    )
                ),
                max_tool_calls=int(
                    config.get(
                        "max_tool_calls",
                        0,
                    )
                ),
                output_actions=tuple(
                    config.get(
                        "output_actions",
                        [],
                    )
                ),
            )

        return output

    def get(
        self,
        name: str,
    ) -> SkillDefinition:
        return self._skills[name]
```

工作流初始化时，将不同Skill注入对应执行模块：

代码块

```
# query_graph.py

self.skills = SkillRegistry()

# 问数智能体使用database_query Skill
self.single_database_agent = (
    SingleDatabaseAgent(
        model_client,
        self.mcp_client,
        self.skills.get(
            "database_query"
        ),
```

```
            self.config.mcp_max_tool_calls,
        )
    )

    # 问答模型使用data_qa Skill
    self.response_generator = ResponseGenerator(
        model_client,
        self.config,
        self.skills.get("data_qa"),
    )
```

当前实现是“系统启动时加载全部Skill，运行时根据路由选择Skill”，并不是每次请求都重新读取Skill文件。

问答链路

文件：

代码块

```
backend/app/workflows/query_graph.py
→ backend/app/querying/response_generator.py
→ backend/app/model_client.py
```

代码块

```
# query_graph.py

def _answer_qa(
    self,
    state: QueryState,
) -> dict[str, Any]:
    # 组合已有数据上下文
    contexts = [
        item
        for item in (
            state.get(
                "short_term_context",
                "",
            ),
            state.get(
                "recent_result_context",
                "",
            ),
            state.get(
                "analysis_context",
```

```
                """,
            ),
        )
        if item
    ]

    answer = (
        self.response_generator.answer_qa(
            state["query"],
            "\n".join(contexts),
        )
    )

    result = ResultBuilder.qa(
        state["task_id"],
        answer,
        state["intent"],
        state.get("analysis_sources") or [],
    )

    return {
        "workflow_mode": "qa",
        "result": result.model_dump(
            mode="json"
        ),
    }
```

`ResponseGenerator` 将问答Skill作为系统提示词调用大模型：

代码块

```
# response_generator.py

def answer_qa(
    self,
    query: str,
    context: str,
) -> str:
    # data_qa/SKILL.md中的内容
    system = self.qa_skill.instructions

    user = (
        f"问题：{query}\n"
        f"可用上下文：{context or '无'}"
    )

```

```
        # 问答直接返回自然语言
        return self.model_client.chat(
            system,
            user,
        )
```

问答模型核心入参：

代码块

```
{
  "system": "data_qa/SKILL.md中的问答规则",
  "user": {
    "query": "华东为什么最高？",
    "context": {
      "short_term_context": "近期对话和摘要",
      "recent_result_context": "最近查询结果",
      "analysis_context": "用户选中的结果表"
    }
  }
}
```

问答模型直接返回文字：

代码块

```
华东地区销售额最高，主要贡献来自企业级客户和办公设备品类。
```

该链路不会生成SQL，也不会调用数据库工具。

问数链路

文件：

代码块

```
backend/app/workflows/query_graph.py
→ backend/app/querying/single_database_agent.py
→ backend/app/mcp_runtime/client.py
```

Schema检索完成后，工作流调用问数智能体：

代码块

```
# query_graph.py

decision = (
    self.single_database_agent.prepare(
        state["standalone_query"],
        database,
        state["schema_graph"],
        state["schema_context"],
        state["retrieval"],
        state.get("workspace") or {},
        state.get("access_scope") or {},
    )
)
```

问数智能体先根据Skill配置获取允许使用的MCP工具：

代码块

```
# single_database_agent.py

mcp_client = self.mcp_client_factory(
    access_scope
)

# 每次请求通过MCP获取当前工具列表
catalog = mcp_client.list_tools()

tools = [
    tool
    for tool in catalog

    # 只保留Skill允许使用的工具
    if any(
        fnmatch(
            tool["name"],
            pattern,
        )
        for pattern
        in self.skill.allowed_tools
    )

    # 只提供当前数据库的查询工具
    and (
        not tool["name"].startswith(
            "query_"
        )
```

```
            or tool["name"] == database_tool
        )
    ]
```

`database_query` Skill当前允许：

代码块

```
{
  "allowed_tools": [
    "current_datetime",
    "resolve_date_range",
    "query_*"
  ],
  "max_tool_calls": 3,
  "output_actions": [
    "call_tool",
    "clarify"
  ]
}
```

然后将问数Skill、Schema图和工具列表交给大模型：

代码块

```
system = (
    "你是单数据库问数智能体。\n\n"
    + self.skill.instructions
)

payload = {
    "query": query,
    "database": database,
    "schema_graph": schema_graph,
    "retrieval": {
        "selected_fields": retrieval.get(
            "hits",
            [],
        ),
        "low_confidence_candidates":
            retrieval.get(
                "low_confidence_candidates",
                [],
            ),
    },
    "confirmed_fields": workspace.get(
```

```
            "schema_fields",
            [],
        ),
        "confirmed_parameters": workspace.get(
            "confirmed_parameters",
            {},
        ),
        "mcp_tools": tools,
        "tool_results": observations,
    }

    decision = self.model_client.chat_json(
        system,
        json.dumps(
            payload,
            ensure_ascii=False,
        ),
    )
```

模型需要直接生成SQL，并将SQL放入数据库工具参数：

代码块

```
{
  "action": "call_tool",
  "tool_name": "query_askdata_mock",
  "reason": "按照地区汇总已支付订单销售额",
  "arguments": {
    "sql": "SELECT region AS '地区', SUM(paid_amount) AS '销售额' FROM orders_current WHERE status = '已支付' GROUP BY region ORDER BY 2 DESC"
  }
}
```

代码校验工具名称和参数后，通过MCP执行：

代码块

```
tool_name = decision["tool_name"]
arguments = decision["arguments"]

if tool_name not in tool_names:
    raise ValueError(
        "智能体选择了未提供的MCP工具"
    )

tool_result = mcp_client.call_tool(
```

```
    tool_name,
    arguments,
)

# 数据库工具执行完成后返回查询结果
if tool_name == database_tool:
    return {
        "action": "executed",
        "execution": tool_result,
        "source": "model_mcp",
    }

# 时间工具结果重新提供给模型
observations.append({
    "tool": tool_name,
    "result": tool_result,
})
```

如果用户问题包含“今天”之类的相对时间，模型可以先调用时间工具：

代码块

```
第一次调用模型
→ current_datetime

第二次调用模型
→ 根据明确日期生成SQL
→ query_askdata_mock

最多调用3次工具
```

MCP工具管理&权限校验

代码块

```
登录用户
→ AccessController.resolve          生成用户权限范围
→ SchemaIndex.retrieve              过滤无权访问的Schema字段
→ create_local_mcp_server           注册用户可用的MCP工具
→ SingleDatabaseAgent               获取工具列表并选择工具
→ LocalMcpClient.call_tool          调用数据库工具
→ DuckDbEngine._validate_sql        校验SQL和表权限
→ DuckDB执行SQL
```

生成用户权限范围

文件：`backend/app/security/access_control.py`

代码块

```
def resolve(
    self,
    user_id: str | None,
) -> AccessScope:
    user = (
        user_id
        or self.DEFAULT_USER
    )

    roles = self.USER_ROLES.get(
        user,
        (),
    )

    databases = set()
    tables = set()

    for role in roles:
        policy = self.ROLE_POLICIES[role]

        databases.update(
            policy["databases"]
        )
        tables.update(
            policy["tables"]
        )

    return AccessScope(
        user_id=user,
        roles=roles,
        allowed_databases=frozenset(
            databases
        ),
        allowed_tables=frozenset(
            tables
        ),
    )
```

权限结果会贯穿Schema检索、MCP工具注册和SQL执行流程：

代码块

```
{
  "user_id": "demo_current_sales",
  "roles": ["current_sales"],
  "allowed_databases": [
    "askdata_mock"
  ],
  "allowed_tables": [
    "orders_current",
    "customers",
    "products",
    "sales_targets"
  ]
}
```

Schema召回权限过滤

文件：`backend/app/retrieval/service.py`

代码块

```
allowed_doc_ids = {
    document.doc_id
    for document in self.documents

    # 只允许召回用户有权限访问的表字段
    if access_scope.allows_table(
        document.database_id,
        document.table_id,
    )
}

bm25 = self.store.bm25_search(
    retrieval_text,
    self.config.bm25_top_k,
    allowed_doc_ids,
)

dense = self.store.dense_search(
    query_vector,
    self.config.dense_top_k,
    allowed_doc_ids,
)
```

权限过滤发生在BM25和向量召回之前，因此无权访问的字段不会进入候选结果、Rerank和Schema图。

MCP工具动态注册

文件：`backend/app/mcp_runtime/server.py`

代码块

```
def create_local_mcp_server(
    engine: DuckDbEngine,
    access_scope: AccessScope,
) -> MCPServer:
    server = MCPServer(
        name="askdata-local-tools",
    )

    # 注册公共时间工具
    server.tool(
        name="current_datetime",
        annotations=READ_ONLY,
    )(current_datetime)

    server.tool(
        name="resolve_date_range",
        annotations=READ_ONLY,
    )(resolve_date_range)

    for database in databases:
        # 无数据库权限时不注册查询工具
        if not access_scope.allows_database(
            database
        ):
            continue

        handler = build_database_query_tool(
            database,
            engine,
            access_scope,
        )

        # 一个数据库对应一个MCP工具
        server.tool(
            name=f"query_{database}",
            annotations=READ_ONLY,
        )(handler)

    return server
```

MCP工具被标记为只读和幂等：

代码块

```
READ_ONLY = ToolAnnotations(
    readOnlyHint=True,
    destructiveHint=False,
    idempotentHint=True,
    openWorldHint=False,
)
```

获取并调用MCP工具

文件：`backend/app/querying/single_database_agent.py`

代码块

```
mcp_client = self.mcp_client_factory(
    access_scope
)

# 通过MCP获取当前用户可用的工具
catalog = mcp_client.list_tools()

tools = [
    tool
    for tool in catalog

    # 根据Skill配置过滤工具
    if any(
        fnmatch(
            tool["name"],
            pattern,
        )
        for pattern
        in self.skill.allowed_tools
    )

    # 只保留当前数据库的查询工具
    and (
        not tool["name"].startswith(
            "query_"
        )
        or tool["name"] == database_tool
    )
]
```

模型只能选择提供给它的工具：

代码块

```
tool_name = decision["tool_name"]
arguments = decision["arguments"]

if tool_name not in tool_names:
    raise ValueError(
        "智能体选择了未提供的MCP工具"
    )

tool_result = mcp_client.call_tool(
    tool_name,
    arguments,
)
```

SQL安全与权限校验

文件：`backend/app/querying/duckdb_engine.py`

代码块

```
def _validate_sql(
    self,
    database: str,
    sql: str,
    access_scope: AccessScope,
) -> str:
    statements = sqlglot.parse(
        sql,
        read="duckdb",
    )

    # 一次只允许执行一条SQL
    if len(statements) != 1:
        raise ValueError(
            "一次只允许执行一条SQL"
        )

    statement = statements[0]

    # 只允许SELECT或WITH查询
    if not isinstance(
        statement,
        exp.Query,
    ):
        raise ValueError(
            "只允许执行SELECT/WITH只读查询"
```

```
            )

        forbidden_nodes = {
            "insert",
            "update",
            "delete",
            "merge",
            "create",
            "drop",
            "alter",
            "copy",
            "attach",
            "grant",
            "revoke",
        }

        # 禁止写入和数据库管理操作
        if any(
            node.key in forbidden_nodes
            for node in statement.walk()
        ):
            raise ValueError(
                "SQL包含禁止的写入或管理操作"
            )

        # 禁止SELECT *
        for select in statement.find_all(
            exp.Select
        ):
            if any(
                isinstance(item, exp.Star)
                or (
                    isinstance(item, exp.Column)
                    and item.is_star
                )
                for item in select.expressions
            ):
                raise ValueError(
                    "不允许使用SELECT *"
                )

        # 校验数据库权限
        if not access_scope.allows_database(
            database
        ):
            raise ValueError(
                "当前用户无权访问该数据库"
```

```
        )

    # 提取SQL实际引用的数据表
    referenced_tables = {
        table.name
        for table in statement.find_all(
            exp.Table
        )
        if table.name not in cte_names
    }

    # 再次校验表级权限
    denied_tables = {
        table
        for table in referenced_tables
        if not access_scope.allows_table(
            database,
            table,
        )
    }

    if denied_tables:
        raise ValueError(
            "SQL引用了当前用户无权访问的数据表"
        )

    return sql
```

权限校验共执行三次：

代码块

```
Schema召回前      → 不召回无权访问的字段
MCP工具注册时     → 不暴露无权访问的数据库工具
SQL执行前         → 校验SQL实际引用的数据库和数据表
```

滑动窗口&异步摘要短期记忆

🔥 **短期记忆分为“滑动窗口”和“异步摘要”两部分。**

1. 获取当前会话已经生成的历史摘要。
2. 获取所有尚未进入摘要的完整轮次，每轮包含用户问题、模型回答、结果标题和表格结构。
3. 将历史摘要和尚未摘要的完整轮次组合成本次模型上下文：

4. 当前对话完成后，将最新一轮加入滑动窗口。
5. 计算历史摘要+所有尚未摘要完整轮次的Token：
6. 如果没有达到12,000 Token，继续保留所有完整轮次，不调用摘要模型。
7. 如果达到12,000 Token：
    - 至少保留最近5轮完整内容；
    - 从最早轮次开始选择；
    - 累计选择约6,000 Token；
    - 不截断单个轮次；
    - 将“上一版摘要 + 本批旧轮次”提交到后台摘要线程。
8. 摘要成功后，更新历史摘要，并将本批轮次移出滑动窗口。
9. 摘要失败后，继续使用上一版摘要，本批轮次仍保留在滑动窗口中。

核心调用链

```
SessionContext._memory_items        整理当前会话轮次
→ ShortTermMemory.context           生成当前短期记忆
→ SessionContext.remember           加入最新轮次
→ ShortTermMemory.maybe_schedule    判断是否触发摘要
→ ShortTermMemory._summarize        异步更新历史摘要
```

滑动窗口记忆

滑动窗口保存所有尚未进入摘要的完整轮次。它不是固定轮数，而是随着Token数量动态变化。

代码块

```
每轮内容较短
→ 可以保留更多轮

每轮内容较长
→ 保留轮数自然减少

无论轮次多长
→ 不会截断一轮中的部分内容
```

核心代码可以整合为：

代码块

```
def _memory_items(
```

```
    self,
    session_id: str,
) -> list[dict[str, Any]]:
    """整理当前会话中的所有轮次。"""

    output = []

    for task_id in self.session_tasks.get(
        session_id,
        [],
    ):
        task = self.tasks[task_id]
        result: QueryResult = task["result"]

        output.append({
            "task_id": task_id,
            "query": task["query"],
            "result_title": result.result_title,
            "analysis": result.analysis or "",
            "columns": result.columns,
            "row_count": len(result.rows),
        })

    return output


def context(
    self,
    session_id: str,
    items: list[dict[str, Any]],
) -> str:
    """生成本次模型使用的短期记忆。"""

    with self._lock:
        state = self._states.get(
            session_id,
            SessionSummary(),
        )

        summary = state.text
        summarized_ids = set(
            state.summarized_ids
        )

    # 没有进入摘要的轮次继续保留完整内容
    pending_turns = [
        self._public_item(item)
```

```
            for item in items
            if str(item["task_id"])
            not in summarized_ids
        ]

        if not summary and not pending_turns:
            return ""

        return json.dumps(
            {
                "history_summary": summary,
                "pending_turns": pending_turns,
            },
            ensure_ascii=False,
        )
```

`_memory_items()` 先把当前会话整理成统一的轮次结构，`context()` 再排除已经进入摘要的轮次。

最终输出：

代码块

```
{
  "history_summary": "用户正在分析地区销售情况。",
  "pending_turns": [
    {
      "turn_id": "task-008",
      "user_message": "各产品的情况呢",
      "assistant_message": "产品A销售额最高",
      "result_title": "产品销售额",
      "columns": ["产品", "销售额"],
      "row_count": 10
    }
  ]
}
```

其中：

代码块

```
history_summary
→ 已经压缩完成的旧历史

pending_turns
```

```
→ 当前滑动窗口中的完整轮次
```

当前对话完成后，通过 `remember()` 加入最新轮次：

代码块

```
def remember(
    self,
    session_id: str,
    result: QueryResult,
    query: str,
    workspace: dict[str, Any],
) -> None:
    self.tasks[result.task_id] = {
        "query": query,
        "session_id": session_id,
        "workspace": workspace,
        "result": result,
    }

    if result.task_id not in self.session_tasks[
        session_id
    ]:
        self.session_tasks[session_id].append(
            result.task_id
        )

    # 加入最新轮次后，检查是否需要摘要
    self.short_term_memory.maybe_schedule(
        session_id,
        self._memory_items(session_id),
    )
```

因此滑动窗口的更新顺序是：

代码块

```
当前对话完成
→ 保存最新轮次
→ 重新计算短期记忆
→ 判断是否需要摘要
```

异步摘要记忆

异步摘要负责将滑动窗口前部的旧轮次压缩成历史摘要。

触发、选择、执行和结果处理可以整合为：

代码块

```
def maybe_schedule(
    self,
    session_id: str,
    items: list[dict[str, Any]],
) -> bool:
    """达到Token阈值后，异步摘要最早的一批完整轮次。"""

    # 摘要功能关闭，或者轮次过少
    if (
        not self.summary_enabled
        or len(items) <= self.min_recent_turns
    ):
        return False

    with self._lock:
        state = self._states.setdefault(
            session_id,
            SessionSummary(),
        )

        # 已有摘要任务时不重复创建
        if state.running:
            return False

        unsummarized = [
            item
            for item in items
            if str(item["task_id"])
            not in state.summarized_ids
        ]

        # 计算：上一版摘要 + 所有未摘要轮次
        total_tokens = self._context_tokens(
            state.text,
            [
                self._public_item(item)
                for item in unsummarized
            ],
        )

        # 未达到12,000 Token，不触发摘要
        if total_tokens < self.trigger_tokens:
            return False
```

```
44
45          # 最近5轮受到保护，不进入本批摘要
46          candidates = unsummarized[
47              :-self.min_recent_turns
48          ]
49
50          # 从最早轮次开始，选择约6,000 Token
51          batch = self._take_complete_turns(
52              candidates
53          )
54
55          if not batch:
56              return False
57
58          previous_summary = state.text
59          batch_ids = [
60              str(item["task_id"])
61              for item in batch
62          ]
63
64          state.running = True
65
66          # 后台执行，不阻塞当前请求
67          state.future = self._executor.submit(
68              self._summarize,
69              session_id,
70              previous_summary,
71              batch,
72              batch_ids,
73          )
74
75      return True
```

摘要批次按照完整轮次选择：

代码块

```
1   def _take_complete_turns(
2       self,
3       candidates: list[dict[str, Any]],
4   ) -> list[dict[str, Any]]:
5       batch = []
6       used_tokens = 0
7
8       for item in candidates:
9           batch.append(item)
```

```
            used_tokens += self._turn_tokens(item)

            # 达到批次目标后停止
            if used_tokens >= self.batch_tokens:
                break

        return batch
```

例如：

代码块

```
当前已经选择5,800 Token
下一轮包含500 Token

加入完整下一轮
→ 本批最终为6,300 Token
→ 停止选择
```

不会为了准确达到6,000 Token而截断最后一轮。

摘要模型每次只处理上一版摘要和本批旧轮次：

代码块

```
def _summarize(
    self,
    session_id: str,
    previous_summary: str,
    batch: list[dict[str, Any]],
    batch_ids: list[str],
) -> None:
    payload = {
        "previous_summary":
            previous_summary or "无",

        "turns_to_merge": [
            self._public_item(item)
            for item in batch
        ],
    }

    try:
        new_summary = self.model_client.chat(
            system,
            json.dumps(
```

```
                payload,
                ensure_ascii=False,
            ),
        ).strip()

        if not new_summary:
            raise RuntimeError(
                "摘要模型返回空内容"
            )

        with self._lock:
            state = self._states.setdefault(
                session_id,
                SessionSummary(),
            )

            # 更新历史摘要
            state.text = new_summary

            # 本批轮次退出滑动窗口
            state.summarized_ids.update(
                batch_ids
            )

    except Exception as exc:
        # 摘要失败时不更新摘要，也不移除轮次
        logger.warning(
            "short_term_summary_failed "
            "session_id=%s error=%s",
            session_id,
            exc,
        )

    finally:
        # 无论成功失败，都结束运行状态
        with self._lock:
            state = self._states.setdefault(
                session_id,
                SessionSummary(),
            )

            state.running = False
```

成功和失败的区别集中在这里：

```
代码块
```

```
摘要成功
→ 更新history_summary
→ 更新summarized_ids
→ 本批轮次退出滑动窗口

摘要失败
→ 保留上一版history_summary
→ 不更新summarized_ids
→ 本批轮次继续保留在滑动窗口
```

第二次摘要仍然采用增量方式：

代码块

```
第一次摘要
→ 无摘要 + 第1批旧轮次

第二次摘要
→ 第一次摘要 + 第2批旧轮次

第三次摘要
→ 第二次摘要 + 第3批旧轮次
```

不会重新发送已经摘要过的全部历史原文。

超参数配置

代码块

```
# 是否开启异步摘要
SHORT_TERM_SUMMARY_ENABLED=true

# 短期记忆达到约12,000 Token后触发摘要
SHORT_TERM_SUMMARY_TRIGGER_TOKENS=12000

# 每次摘要约处理6,000 Token
SHORT_TERM_SUMMARY_BATCH_TOKENS=6000

# 至少保留最近5个完整轮次
SHORT_TERM_MIN_RECENT_TURNS=5
```

最终可以概括为：

代码块

```
滑动窗口记忆
→ 保存尚未摘要的完整轮次

达到Token阈值
→ 选择窗口前部约6,000 Token的完整轮次

异步摘要
→ 上一版摘要 + 本批旧轮次

摘要成功
→ 更新摘要并缩小滑动窗口

摘要失败
→ 保留原摘要和完整轮次
```

长期记忆

🔥 目前的长期记忆采用简化的用户主动保存模式，主要用于沉淀用户常用字段和查询结果表格，供后续检索和分析时主动引用。

具体流程如下：

1. 用户在前端主动保存查询结果，或收藏常用字段。
2. 前端调用长期记忆接口，并携带当前用户登录Token。
3. 后端校验用户身份、任务归属和字段访问权限。
4. 校验通过后，将字段或表格按用户维度持久化保存至本地JSON文件。
5. 用户重新登录后，前端读取其已保存的长期记忆内容。
6. 用户在后续对话中手动选择字段或结果表，将其加入本次问数或问答上下文。

后续优化方向

后续计划将当前的用户主动保存模式升级为自适应长期记忆机制。系统将在保留用户最终选择权的前提下，统计用户历史查询记录、字段引用次数和表格使用频次，识别高频使用的数据对象，并向用户推荐常用字段和表格。

这样可以减少用户重复查找和手动选择的操作成本，使长期记忆从“用户主动保存和引用”逐步发展为“系统推荐、用户确认后使用”的辅助能力。

代码块

```
用户点击保存
→ 前端调用API
→ FastAPI获取当前用户
→ AskDataService校验权限
```

```
→ MemoryStore写入JSON
→ 用户重新登录后加载
→ 手动选择后加入本次请求
```

保存字段和结果表

文件：

代码块

```
frontend/src/App.vue
→ frontend/src/api.ts
→ backend/app/api/routes.py
→ backend/app/services/askdata_service.py
→ backend/app/services/memory_store.py
```

前端只有在用户点击时才发起保存：

代码块

```
// 保存查询结果
async function saveResult(
  result: QueryResult,
) {
  await api.save(result.task_id)

  result.saved = true

  // 保存后重新获取当前用户的长期记忆
  savedMemories.value =
    await api.memories()
}

// 收藏或取消收藏字段
async function toggleSavedField(
  field: SchemaField,
  tableId: string,
) {
  const memoryId =
    `field:${tableId}.${field.name}`

  if (
    savedFieldKeys.value.has(
      `${tableId}:${field.name}`,
    )
  ) {
```

```
      await api.deleteMemory(memoryId)
    } else {
      await api.saveField(
        tableId,
        field,
      )
    }

    savedMemories.value =
      await api.memories()
  }
```

对应的前端API：

代码块

```
  export const api = {
    // 保存查询结果
    save: (taskId: string) =>
      request("/api/memories", {
        method: "POST",
        body: JSON.stringify({
          task_id: taskId,
        }),
      }),

    // 保存字段
    saveField: (
      tableId: string,
      field: SchemaField,
    ) =>
      request("/api/memories/fields", {
        method: "POST",
        body: JSON.stringify({
          table_id: tableId,
          name: field.name,
          label: field.label,
          field_type: field.type,
        }),
      }),

    // 获取当前用户的长期记忆
    memories: () =>
      request("/api/memories"),
  }
```

后端保存查询结果前，会校验任务状态和任务所属用户：

代码块

```
def save_memory(
    self,
    task_id: str,
    user_id: str | None = None,
) -> None:
    task = self.tasks.get(task_id)

    if (
        not task
        or task["result"].status
        != "completed"
    ):
        raise KeyError(task_id)

    scope = self.access_controller.resolve(
        user_id
    )

    # 只能保存自己的查询结果
    if task["user_id"] != scope.user_id:
        raise PermissionError(
            "无权保存该查询结果"
        )

    result = task["result"]

    self.memories.save_result(
        task_id,
        task["query"],
        result.analysis or result.message,
        result.result_title or "查询结果",
        result.columns,
        result.rows,
        scope.user_id,
    )
```

字段保存会额外校验Schema权限：

代码块

```
def save_field_memory(
    self,
    table_id: str,
```

```
    name: str,
    label: str,
    field_type: str,
    user_id: str | None = None,
) -> None:
    scope = self.access_controller.resolve(
        user_id
    )

    table = next(
        (
            item
            for item in SCHEMA
            if item["id"] == table_id
        ),
        None,
    )

    database = (
        table.get(
            "database",
            "askdata_mock",
        )
        if table
        else ""
    )

    # 无权访问的表字段不能保存
    if (
        not table
        or not scope.allows_table(
            database,
            table_id,
        )
    ):
        raise PermissionError(
            "无权保存该字段"
        )

    self.memories.save_field(
        table_id,
        name,
        label,
        field_type,
        scope.user_id,
    )
```

JSON持久化与用户隔离

文件：`backend/app/services/memory_store.py`

代码块

```
def save_result(
    self,
    task_id: str,
    query: str,
    summary: str,
    title: str,
    columns: list[str],
    rows: list[dict[str, Any]],
    user_id: str,
) -> None:
    self._upsert(
        user_id,
        {
            "id": f"result:{task_id}",
            "user_id": user_id,
            "kind": "result_table",
            "task_id": task_id,
            "query": query,
            "summary": summary,
            "title": title,
            "columns": columns,
            "rows": rows,
            "created_at":
                datetime.now().isoformat(
                    timespec="seconds"
                ),
        },
    )
```

字段使用相同结构保存：

代码块

```
{
    "id": "field:customers.customer_level",
    "user_id": "demo_current_sales",
    "kind": "schema_field",
    "table_id": "customers",
    "name": "customer_level",
    "label": "客户等级",
    "field_type": "文本"
```

```
}
```

`_upsert()` 负责去重和数量限制：

代码块

```
def _upsert(
    self,
    user_id: str,
    item: dict[str, Any],
) -> None:
    items = self._read_all()

    # 当前用户已有的其他记忆
    owned = [
        current
        for current in items
        if (
            self._owner(current) == user_id
            and current.get("id")
            != item["id"]
        )
    ]

    # 其他用户的记忆保持不变
    others = [
        current
        for current in items
        if self._owner(current)
        != user_id
    ]

    # 每个用户分别限制保存数量
    self._write([
        item,
        *owned[:self.limit - 1],
        *others,
    ])
```

数据最终写入：

代码块

```
backend/data/saved_memories.json
```

因此服务重启不会清除长期记忆。

长期记忆如何重新使用

用户登录后，前端主动读取长期记忆：

代码块

```
const [schemaResult, memories] =
  await Promise.all([
    api.schema(),
    api.memories(),
  ])

schema.value = schemaResult
savedMemories.value = memories
```

两类长期记忆的用途不同：

代码块

```
保存字段
→ 显示在字段库
→ 用户拖入查询字段
→ 加入workspace.schema_fields
→ 强制加入本次Schema图

保存结果表
→ 显示在结果表库
→ 用户拖入参考结果
→ 加入workspace.analysis_tables
→ 提供给数据问答模型分析
```

发送请求时，选中的结果表会随请求传给后端：

代码块

```
const requestWorkspace = {
  ...workspace.value,

  analysis_tables:
    selectedAnalysisTables.value.map(
      (table) => ({
        task_id: table.taskId,
        title: table.title,
        query: table.query,
```

```
          columns: table.columns,

          // 当前最多传递前50行
          rows: table.rows.slice(0, 50),
        }),
      ),
  }
```

当前长期记忆不会根据用户问题自动召回。只有用户在前端手动选择字段或结果表后，长期记忆才会进入当前请求。

V1版本代码解析

> ⭐ V1版本代码为Demo代码，参照最早的技术路线，仅有单元的后端py代码和Pipeline示例，仅供学习。

环境准备

> 本项目租了个**火山引擎云服务器**作为运行环境，有条件的也可以在自己电脑上部署，系统为 **Linux**。
> 环境配置流程如下：
>
> 1. 安装 Anaconda
> 2. 创建 Conda 虚拟环境
> 3. 安装项目依赖
> 4. 配置阿里云 API Key（**一定要换成自己的API Key！！！**）

安装Anaconda

代码块

```
# 如果没有wget，先安装
sudo apt update && sudo apt install -y wget
# CentOS/RHEL 系统用：
# sudo yum install -y wget

# 下载 Anaconda Linux x86_64 安装包
wget https://repo.anaconda.com/archive/Anaconda3-2025.12-2-Linux-x86_64.sh

# 安装
bash Anaconda3-2025.12-2-Linux-x86_64.sh
```

创建虚拟环境

代码块

```
创建环境：
conda create -n AI(环境名) python=3.10 -y

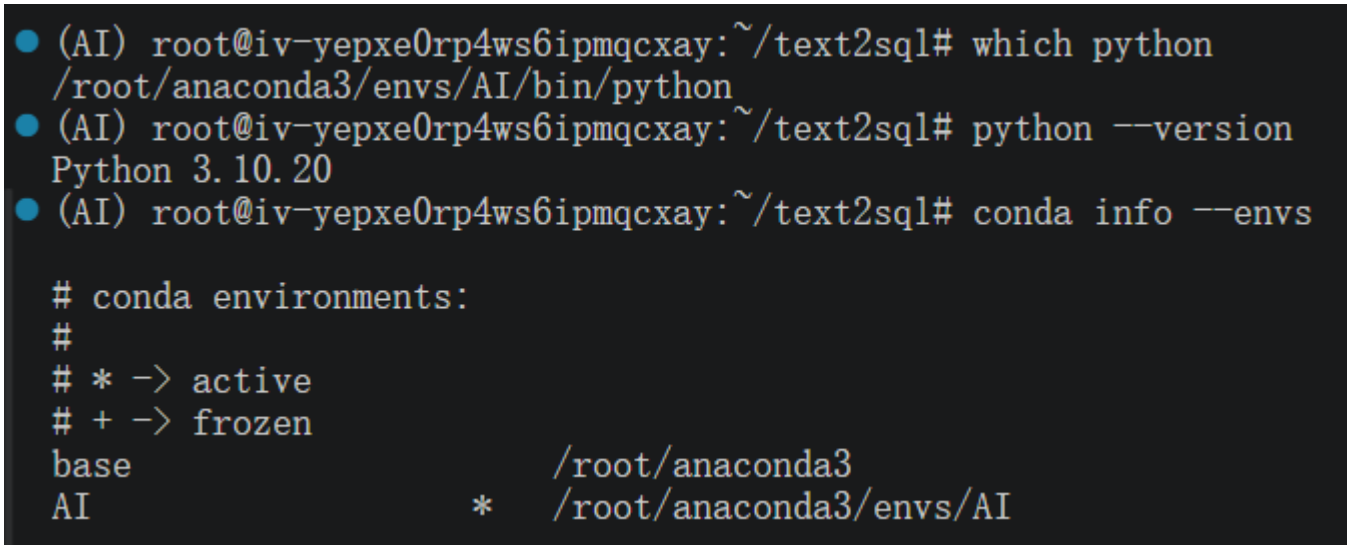
激活环境：
conda activate AI

校验当前 Python 是否来自该环境：
which python
python --version
conda info --envs

----------------------------------------------------------------
校验结果：
/root/anaconda3/envs/AI/bin/python

Python 3.10.20

# conda environments:
#
# * -> active
# + -> frozen
base                     /root/anaconda3
AI                   *   /root/anaconda3/envs/AI
```



配置环境

代码块

```
配置环境命令：
```

```
pip install -r requirements.txt

编的一个脚本用于测试环境：
python env_test.py
------------------------------------------------------------

测试结果：
================================================================================
Text2SQL 环境测试开始
================================================================================

================================================================================
1. Python 版本检查
================================================================================
当前 Python 版本: 3.10.20
✅ Python 版本满足建议要求: >= 3.9

================================================================================
2. 依赖检查
================================================================================
✅ numpy 已安装
✅ sentence-transformers 已安装
✅ torch 已安装
✅ scikit-learn 已安装

建议安装命令:
pip install numpy sentence-transformers torch scikit-learn

================================================================================
3. SQLite 检查
================================================================================
SQLite 版本: 3.53.2
✅ SQLite 创建表、写入、查询均正常

================================================================================
4. 测试数据库 Schema 解析与字段检索
================================================================================
```

```
已创建测试数据库: /tmp/text2sql_env_test.db

解析到的字段:
- orders.order_id (INTEGER), PK, samples=['101', '102', '103', '104']
- orders.user_id (INTEGER), FK -> users.user_id, samples=['1', '2', '3']
- orders.order_amount (REAL), samples=['99.0', '199.0', '88.0', '299.0']
- orders.order_time (TEXT), samples=['2024-04-01', '2024-04-02', '2024-04-03']
- orders.status (TEXT), samples=['已支付', '已取消']
- users.user_id (INTEGER), PK, samples=['1', '2', '3']
- users.city (TEXT), samples=['北京', '上海', '深圳']
- users.gender (TEXT), samples=['男', '女']
- users.register_time (TEXT), samples=['2024-01-01', '2024-02-01', '2024-03-01']

测试 Query: 每个城市的销售额是多少?
召回字段:
- orders.order_amount score=9.6415
- users.city score=8.1254

✅ 最小字段检索流程正常

==========
5. Embedding 依赖可用性检查
==========
✅ sentence-transformers 可以 import
说明: 这里不强制下载模型，避免无网络环境卡住。
后续你可以手动测试:
python -c "from sentence_transformers import SentenceTransformer; SentenceTransformer('BAAI/bge-small-zh-v1.5')"

==========
测试结果汇总
==========
python              : ✅ PASS
numpy               : ✅ PASS
sentence-transformers: ✅ PASS
torch               : ✅ PASS
sklearn             : ✅ PASS
sqlite              : ✅ PASS
schema_retrieval    : ✅ PASS
embedding_import    : ✅ PASS

✅ 基础环境可用于开发 Text2SQL Schema 检索模块。
```

```
80
81  完成。
```

> **环境配置文件：**requirement.txt

```
annotated-doc==0.0.4
anyio==4.14.1
cachetools==7.1.4
certifi==2026.6.17
charset-normalizer==3.4.7
click==8.4.2
cuda-bindings==13.2.0
cuda-pathfinder==1.5.4
cuda-toolkit==13.0.2
exceptiongroup==1.3.1
filelock==3.29.4
fsspec==2026.6.0
grpcio==1.81.1
h11==0.16.0
hf-xet==1.5.1
httpcore==1.0.9
httpx==0.28.1
huggingface_hub==1.21.0
idna==3.18
Jinja2==3.1.6
joblib==1.5.3
markdown-it-py==4.2.0
MarkupSafe==3.0.3
mdurl==0.1.2
milvus-lite==2.5.1
mpmath==1.3.0
networkx==3.4.2
numpy==2.2.6
nvidia-cublas==13.1.0.3
nvidia-cuda-cupti==13.0.85
nvidia-cuda-nvrtc==13.0.88
nvidia-cuda-runtime==13.0.96
nvidia-cudnn-cu13==9.19.0.56
nvidia-cufft==12.0.0.61
nvidia-cufile==1.15.1.6
nvidia-curand==10.4.0.35
nvidia-cusolver==12.0.4.66
nvidia-cusparse==12.6.3.3
nvidia-cusparselt-cu13==0.8.0
nvidia-nccl-cu13==2.28.9
nvidia-nvjitlink==13.0.88
nvidia-nvshmem-cu13==3.4.5
nvidia-nvtx==13.0.85
orjson==3.11.9
packaging==26.0
pandas==2.3.3
protobuf==7.35.1
```

requirements.txt

配置API Key

⭐ 一定要配置自己的API key，题主用的是阿里云百炼的大模型，种类多一些，当然也可以用Deepseek或者其他的平台。

代码块

```
配置阿里云百炼API Key，直接在环境变量里面配置，不建议直接写死在代码里
export DASHSCOPE_API_KEY="你的阿里云百炼APIKey"

如果需要指定WorkspaceId，再配置，大部分情况下不用：
export DASHSCOPE_WORKSPACE_ID="你的WorkspaceId"
```

核心代码解析

三级索引构建与存储(schema_indexing/)

⭐ Schema Indexing 主要负责数据库 Schema 信息的结构化建模、三级索引内容构建以及索引数据持久化，为后续 Schema 检索阶段提供基础能力。

该模块首先将数据库中的表结构、字段信息以及表间关系抽象为统一的数据对象，包括表格级 Schema、字段级 Schema 和表关系信息。随后以**字段级 Schema 作为核心索引单元**，结合字段所属表信息、业务描述以及数据样例等上下文信息，分别构建**关键词索引、语义向量索引以及 Rerank 精排索引**。

- 关键词索引用于字段名称、业务术语及显式关键词匹配；
- 语义索引用于基于 Embedding 的语义相似度召回；
- Rerank 索引用于对召回结果进行更精细的相关性排序。

索引构建完成后，系统通过 Embedding 服务生成字段语义向量，并将字段级索引信息以及表关系信息写入向量数据库进行存储。最终形成支持关键词召回、语义召回和精排优化的 Schema 检索基础，为自然语言到 SQL 转换流程提供准确的 Schema 上下文。

目录结构

```
schema_indexing/
├── __init__.py
│
├── objects.py                    # 索引构建核心数据结构
│                                 # ColumnSchema / TableRelation / SchemaGraph
│
├── text_builder.py               # 三级索引文本构建
│                                 # keyword_text：关键词召回文本
│                                 # vector_text：向量召回文本
```

```
10  │                                  # rerank_text: Rerank 精排文本
11  │
12  ├── embedding_client.py            # 阿里云 Embedding 服务端
13  │                                  # 对 vector_text 生成向量
14  │                                  # 无 API Key 时支持 Mock Embedding
15  │
16  ├── milvus_client.py               # Milvus 索引存储封装
17  │                                  # 字段级索引 Collection
18  │                                  # 表关系索引 Collection
19  │
20  ├── index_builder.py               # Schema 索引构建主流程
21  │                                  # 构建字段三级索引
22  │                                  # 写入字段索引和表关系索引
23  │
24  ├── build_index_demo.py            # 索引构建 Demo
25  │                                  # 构造 Demo Schema
26  │                                  # 构造 Demo 表关系
27  │                                  # 写入 Milvus / Milvus Lite
```

数据结构/object.py

⭐ `ColumnSchema`、`TableRelation` 和 `SchemaGraph` 分别对应 Schema 建模中的三个层次：

- `ColumnSchema` 用于描述单个字段，是 Schema 检索和索引构建的基本单元。
- `TableRelation` 用于描述表与表之间的关联关系，为多表查询和 Join 路径构建提供依据。
- `SchemaGraph` 用于将当前用户 Query 相关的表、字段和表关系组织成一个局部 Schema 子图，并转换为 CoT 生成和 SQL 生成所需的 Prompt 上下文。

ColumnSchema

🔥 `ColumnSchema` 用于描述数据库中的单个字段，包含字段的结构信息、业务语义、数据特征以及关联关系等内容。

主要参数如下：

- `database`：字段所属的数据库名称。
- `table_name`：字段所属的数据表名称。
- `column_name`：字段在数据库中的实际名称。
- `data_type`：字段的数据类型。

- `nullable`：字段是否允许为空。
- `description`：字段的业务含义。
- `aliases`：字段的业务别名或其他常用表达。
- `table_description`：字段所属表的业务描述。
- `table_aliases`：字段所属表的业务别名。将表级信息注入字段对象，有助于补充字段所在的业务场景。
- `samples`：字段的典型样例值。
- `value_range`：字段的取值范围。
- `data_distribution`：字段的数据分布特征。
- `business_usage`：字段在业务分析中的主要用途。
- `semantic_role`：字段在查询中的语义角色，例如指标、维度、筛选条件、时间字段或关联键。
- `is_primary_key`：标识字段是否为主键。
- `foreign_key_ref`：字段引用的外键目标，格式为 `table.column`。
- `index_texts`：字段对应的三级索引文本，包括关键词索引文本、向量索引文本和 Rerank 精排文本。

代码块

```
@dataclass
class ColumnSchema:
    """字段级 Schema，是 Schema 检索的基本单元。"""

    database: str
    table_name: str
    column_name: str
    data_type: str
    nullable: bool = True

    description: str = ""  # 字段业务含义
    aliases: List[str] = field(default_factory=list)  # 字段业务别名

    table_description: str = ""  # 所属表业务描述
    table_aliases: List[str] = field(default_factory=list)  # 所属表别名

    samples: List[str] = field(default_factory=list)  # 字段样例值
    value_range: str = ""  # 字段取值范围
    data_distribution: str = ""  # 字段数据分布

    business_usage: str = ""  # 字段业务用途
```

```python
    semantic_role: str = ""  # 字段语义角色，如 metric、dimension、filter、time、join_key

    is_primary_key: bool = False
    foreign_key_ref: Optional[str] = None  # 外键目标，格式如 table.column

    index_texts: IndexTextBundle = field(default_factory=IndexTextBundle)  # 三级索引文本

    @property
    def full_name(self) -> str:
        """字段全限定名。"""
        return f"{self.database}.{self.table_name}.{self.column_name}"
```

`full_name` 属性用于生成字段的全限定名称，格式为：

代码块

```
database.table_name.column_name
```

该名称可以唯一标识字段，避免不同数据库或数据表中的同名字段发生冲突。

TableRelation

🔥 `TableRelation` 用于描述两张数据表之间的关联关系，主要为 SchemaGraph 构建和多表 SQL 生成提供关联依据。

主要参数如下：

- `database`：表关系所属的数据库。
- `source_table`：关联关系中的起始表。
- `source_column`：起始表中的关联字段。
- `target_table`：关联关系中的目标表。
- `target_column`：目标表中的关联字段。
- `relation_type`：关系类型，默认值为 `foreign_key`，也可以设置为业务关联等其他类型。
- `description`：关联关系的业务含义。

代码块

```
@dataclass
```

```python
class TableRelation:
    """表间关联关系，用于构建 SchemaGraph。"""

    database: str
    source_table: str
    source_column: str
    target_table: str
    target_column: str
    relation_type: str = "foreign_key"  # 关系类型，如 foreign_key、business_relation
    description: str = ""  # 关系业务含义

    @property
    def relation_key(self) -> str:
        """
        表关系索引键。

        两阶段字段检索后，只能拿到命中的字段和字段所属表。
        构建 SchemaGraph 时，需要通过两张表的表名反查关联键。

        为了让（表A，表B）和（表B，表A）都能查到同一条关系，
        这里使用排序后的无向 pair key。
        """
        left, right = sorted([self.source_table, self.target_table])
        return f"{self.database}:{left}__{right}"

    @property
    def join_condition(self) -> str:
        """SQL Join 条件。"""
        return (
            f"{self.source_table}.{self.source_column} = "
            f"{self.target_table}.{self.target_column}"
        )
```

`relation_key` 属性用于生成表关系的索引键。该属性会对两张表的名称进行排序，生成无方向的表对标识：

代码块

```
database:table_a__table_b
```

因此，无论按照“表 A 与表 B”还是“表 B 与表 A”的顺序查询，都可以定位到同一条表关系。

`join_condition` 属性用于生成 SQL Join 条件，格式为：

```
source_table.source_column = target_table.target_column
```

该结果可以作为后续 SQL 生成时的表关联依据。

SchemaGraph

> 🔥 `SchemaGraph` 表示根据当前用户 Query 构建的局部 Schema 子图。它只包含与当前查询相关的表、字段和表间关系，而不是完整的数据库 Schema。

主要参数如下：

- `database`：当前 Schema 子图所属的数据库。
- `tables`：相关表集合，以表名为键、`TableSchema` 对象为值。
- `columns`：相关字段集合，以表名为键，并将对应的 `ColumnSchema` 对象按表分组。
- `relations`：相关表之间的关联关系列表。

代码块

```python
@dataclass
class SchemaGraph:
    """面向当前 Query 构建的 Schema 子图，作为 CoT 和 SQL 生成上下文。"""

    database: str
    tables: Dict[str, TableSchema]  # 相关表集合
    columns: Dict[str, List[ColumnSchema]]  # 按表分组的相关字段
    relations: List[TableRelation]  # 相关表间关系

    def to_prompt_context(self) -> str:
        """转换为可放入 Prompt 的 Schema 上下文。"""
        lines: List[str] = []

        lines.append(f"数据库: {self.database}")
        lines.append("")

        for table_name, table in self.tables.items():
            lines.append(f"表名: {table_name}")

            if table.description:
                lines.append(f"表格摘要: {table.description}")

            if table.aliases:
                lines.append(f"表别名: {', '.join(table.aliases)}")

            if table.primary_keys:
```

```
                lines.append(f"主键: {', '.join(table.primary_keys)}")

            lines.append("字段: ")

            for col in self.columns.get(table_name, []):
                lines.append(f"- 字段名: {col.column_name}")
                lines.append(f"  数据类型: {col.data_type}")

                if col.description:
                    lines.append(f"  字段含义: {col.description}")

                if col.aliases:
                    lines.append(f"  字段别名: {', '.join(col.aliases)}")

                if col.semantic_role:
                    lines.append(f"  字段角色: {col.semantic_role}")

                if col.samples:
                    lines.append(f"  样例值: {', '.join(col.samples[:8])}")

                if col.value_range:
                    lines.append(f"  取值范围: {col.value_range}")

                if col.data_distribution:
                    lines.append(f"  数据分布: {col.data_distribution}")

                if col.business_usage:
                    lines.append(f"  业务用途: {col.business_usage}")

            lines.append("")

        if self.relations:
            lines.append("表关联关系: ")

            for rel in self.relations:
                description = f"; 关联含义: {rel.description}" if
rel.description else ""
                lines.append(
                    f"- {rel.source_table}.{rel.source_column} "
                    f"↔ {rel.target_table}.{rel.target_column}"
                    f"{description}"
                )

        return "\n".join(lines)
```

`to_prompt_context()` 方法用于将结构化的 SchemaGraph 转换为大模型可以直接使用的文本上下文，主要包括：

- 数据库名称；
- 表名、表描述、表别名和主键；
- 字段名称、数据类型、业务含义和字段别名；
- 字段语义角色、样例值、取值范围和数据分布；
- 字段业务用途；
- 表与表之间的关联字段及关联含义。

字段样例值最多保留前 8 个，以减少无关内容，控制 Prompt 长度。转换后的文本将作为 CoT 生成和 SQL 生成阶段的 Schema 上下文。

三级索引构建/text_builder.py

> ⭐ `text_builder.py` 主要负责根据字段级 `ColumnSchema` 信息构建三级索引文本，包括关键词索引文本、语义向量索引文本和 Rerank 精排文本，并最终封装为 `FieldDocument`。
>
> 构建过程中会使用字段名称、字段类型、业务描述、字段别名、所属表信息、样例值、数据分布以及主外键关系等 Schema 信息，以满足不同检索阶段的文本需求。
>
> 如果部分 Schema 描述需要通过大模型生成或补充，生成结果必须经过人工复核，避免因模型生成内容不准确而影响后续索引召回和排序结果。

SchemaIndexTextBuilder

> 🔥 `SchemaIndexTextBuilder` 是三级索引文本构建类，负责将结构化的 `ColumnSchema` 转换为可用于检索和排序的 `FieldDocument`。

该类没有显式定义初始化参数，主要通过各方法接收 `ColumnSchema` 对象作为输入，并根据字段中已有的 Schema 信息构建对应的三级索引文本。

构建结果主要包括：

- `keyword_text`：用于关键词检索，内容较短，重点保留字段名称、业务术语、别名和样例关键词；
- `vector_text`：用于语义向量检索，重点描述字段含义、所属表信息和业务用途；
- `rerank_text`：用于候选结果精排，包含更加完整的字段描述、数据特征和关联信息；
- `FieldDocument`：用于统一封装字段标识、字段对象以及三级索引文本，作为后续索引写入和检索的基本文档单元。

Embedding服务端/embedding_client.py

⭐ `embedding_client.py` 主要负责调用阿里云百炼 Embedding 服务，将三级索引中的 `vector_text` 转换为向量表示，为后续 Milvus 向量索引写入和语义检索提供数据。

AliyunEmbeddingClient

🔥 `AliyunEmbeddingClient` 是 Embedding 服务客户端，负责根据配置调用阿里云百炼接口，或在无 API Key 时生成本地模拟向量。

`init()` 方法用于初始化 Embedding 客户端。未显式传入配置时会使用默认配置，并在 `api_key` 为空时尝试从环境变量中读取。

`embed_texts()` 方法用于批量生成文本向量。存在 API Key 时调用真实的阿里云百炼服务；未配置 API Key 且允许 Mock 时，使用本地 Hash Embedding；两种方式均不可用时抛出异常。

`_call_aliyun_embedding()` 方法用于组织请求参数、调用阿里云百炼 Embedding API、解析返回结果，并按照输入文本顺序生成向量数组。

`_mock_embedding()` 方法用于生成本地模拟向量。该方法根据文本分词结果进行 Hash 映射，主要用于开发测试和流程验证。

`_simple_tokenize()` 方法用于对中英文文本进行简单切分，包括英文单词、数字、中文单字、二元词组和三元词组。

`_normalize()` 方法用于对向量执行 L2 归一化，使向量长度统一，便于后续进行相似度计算。

Milvus服务端/milvus_client.py

⭐ `milvus_client.py` 主要负责 Schema 索引在 Milvus 中的存储与查询，包括字段级索引 Collection 和表关系 Collection 的创建、数据写入及基础检索。

字段级索引 Collection 用于保存字段信息、三级索引文本和 Embedding 向量，支持后续字段语义召回。表关系 Collection 用于保存表间关联关系，并在字段召回完成后，根据相关表组合补充 SchemaGraph 中的 Join 关系。

当前配置支持使用 **Milvus Lite**，将索引数据保存在本地数据库文件中，也可以通过修改连接配置接入独立部署的 Milvus 服务。

MilvusSchemaIndexClient

🔥 `MilvusSchemaIndexClient` 是 Milvus Schema 索引客户端，负责管理字段索引和表关系索引的完整读写流程。

该类初始化时接收 `MilvusSchemaIndexConfig`，并根据配置中的 `uri` 和 `token` 创建 Milvus 连接。

`prepare_collections()` 方法用于统一创建字段索引 Collection 和表关系 Collection。

`prepare_field_collection()` 方法用于创建字段级索引 Collection，主要保存：

- 字段唯一标识；
- 数据库、表名和字段名；
- 字段类型和语义角色；
- 关键词索引文本、向量索引文本和 Rerank 索引文本；
- 字段对应的 Embedding 向量。

该 Collection 会为 Embedding 字段创建向量索引，用于后续语义相似度检索。

`prepare_relation_collection()` 方法用于创建表关系 Collection，主要保存：

- 表关系唯一标识；
- 起始表、目标表及对应字段；
- 关系类型和业务描述；
- SQL Join 条件；
- 完整的表关系 JSON 信息。

表关系不参与字段召回，主要用于 SchemaGraph 构建阶段补充表之间的关联边。

`insert_field_documents()` 方法用于将字段索引文档和对应的 Embedding 向量写入字段 Collection。写入前会检查文档数量与向量数量是否一致，并在写入后执行数据落盘。

`insert_relations()` 方法用于将表关系写入关系 Collection。每条关系同时保存结构化字段和 JSON 格式的完整信息，便于后续查询和对象还原。

`search_fields()` 方法用于根据 Query Embedding 执行字段向量检索，并返回相似度较高的字段及其三级索引文本。该方法主要用于验证字段向量索引和语义召回能力。

`query_relations_by_table_pairs()` 方法用于根据数据库名称和表名组合查询表间关系。查询结果会从 JSON 数据恢复为 `TableRelation` 对象，供 SchemaGraph 构建使用。

`build_relation_key()` 方法用于生成无方向的表关系键。两张表的名称会先进行排序，再按照以下格式拼接：

```
代码块
1   database:table_a__table_b
```

该方式可以保证表名顺序不同时仍然能够查询到同一条表关系。

索引构建与存储/index_builder.py

🔥 `index_builder.py` 负责组织 Schema 索引构建的完整流程，将字段 Schema 转换为三级索引文档，生成语义向量，并将字段索引和表关系索引写入 Milvus。

该模块主要连接以下类：

- `SchemaIndexTextBuilder`：构建字段的三级索引文本；
- `AliyunEmbeddingClient`：为 `vector_text` 生成 Embedding 向量；
- `MilvusSchemaIndexClient`：创建 Collection，并写入字段索引和表关系数据。

SchemaIndexBuilder

`SchemaIndexBuilder` 是 Schema 索引构建流程的调度类，负责协调索引文本构建、向量生成和 Milvus 数据写入。

初始化时接收三个依赖对象：

- `text_builder`：三级索引文本构建器；
- `embedding_client`：Embedding 向量生成客户端；
- `milvus_client`：Milvus 索引存储客户端。

`build()` 方法用于执行完整的索引构建流程，主要包括：

- 检查字段 Schema 是否为空；
- 将 `ColumnSchema` 批量转换为 `FieldDocument`；
- 提取字段的 `vector_text` 并生成 Embedding 向量；
- 创建字段索引和表关系 Collection；
- 写入字段文档、三级索引文本及向量；
- 写入表间关联关系；
- 返回索引名称和数据数量等构建结果。

`build_documents_only()` 方法只负责将字段 Schema 转换为字段索引文档，不生成向量，也不写入 Milvus。该方法主要用于检查三级索引文本的构建结果或进行单独测试。

测试Demo/build_index_demo.py

代码块

```
from __future__ import annotations

import os
from itertools import combinations
```

```python
from typing import List

from .embedding_client import AliyunEmbeddingClient, AliyunEmbeddingConfig
from .index_builder import SchemaIndexBuilder
from .milvus_client import MilvusSchemaIndexClient, MilvusSchemaIndexConfig
from .objects import ColumnSchema, IndexTextBundle, TableRelation
from .text_builder import SchemaIndexTextBuilder

def build_demo_columns() -> List[ColumnSchema]:
    """
    构造 Demo 字段元数据。

    字段只描述字段本身，表关系单独由 TableRelation 表达。
    """
    return [
        ColumnSchema(
            database="trade_db",
            table_name="trade_summary",
            column_name="user_id",
            data_type="int",
            nullable=False,
            table_description="该表记录用户交易汇总信息，包括用户累计交易笔数、交易金额以及交易活跃度等内容。",
            table_aliases=["用户交易汇总表", "交易统计表"],
            description="用户唯一标识。",
            aliases=["用户ID", "客户ID"],
            samples=["1001", "1002", "1003"],
            business_usage="用于唯一标识交易汇总表中的用户。",
            semantic_role="join_key",
            is_primary_key=True,
        ),
        ColumnSchema(
            database="trade_db",
            table_name="trade_summary",
            column_name="total_trade_count",
            data_type="int",
            nullable=False,
            table_description="该表记录用户交易汇总信息，包括用户累计交易笔数、交易金额以及交易活跃度等内容。",
            table_aliases=["用户交易汇总表", "交易统计表"],
            description="用户累计交易总笔数。",
            aliases=["总交易笔数", "累计交易笔数", "交易笔数", "交易次数"],
            samples=["120", "5800", "56000", "102430"],
            value_range="整数，>= 0",
            data_distribution="多数用户集中在0-50区间，高频交易用户可能超过50000。",
            business_usage="用于衡量用户交易活跃程度，筛选高频交易用户。",
            semantic_role="metric_filter",
```

```
            index_texts=IndexTextBundle(
                keyword_text=(
                    "total_trade_count 总交易笔数 累计交易笔数 交易笔数 "
                    "交易次数 高频交易 用户交易汇总表 交易统计表"
                ),
                vector_text=(
                    "字段名：total_trade_count。所属表：trade_summary。"
                    "该字段表示用户累计交易总笔数，用于衡量用户交易活跃程度，"
                    "可用于筛选高频交易用户。"
                ),
                rerank_text=(
                    "数据库：trade_db\n"
                    "表名：trade_summary\n"
                    "字段名：total_trade_count\n"
                    "数据类型：int\n"
                    "字段含义：用户累计交易总笔数。\n"
                    "字段别名：总交易笔数、累计交易笔数、交易笔数、交易次数。\n"
                    "取值范围：整数，>= 0。\n"
                    "数据分布：高频交易用户可能超过50000。\n"
                    "业务用途：用于筛选总交易笔数大于某个阈值的用户。"
                ),
            ),
        ),
        ColumnSchema(
            database="trade_db",
            table_name="interest_info",
            column_name="user_id",
            data_type="int",
            nullable=False,
            table_description="该表记录用户利率信息，包括用户利率类型、利率数值以及利率生效状态等内容。",
            table_aliases=["用户利率表", "利率信息表"],
            description="用户唯一标识。",
            aliases=["用户ID", "客户ID"],
            samples=["1001", "1002", "1003"],
            business_usage="用于唯一标识利率信息表中的用户。",
            semantic_role="join_key",
            is_primary_key=True,
        ),
        ColumnSchema(
            database="trade_db",
            table_name="interest_info",
            column_name="interest_rate",
            data_type="decimal",
            nullable=False,
            table_description="该表记录用户利率信息，包括用户利率类型、利率数值以及利率生效状态等内容。",
```

```
            table_aliases=["用户利率表", "利率信息表"],
            description="用户对应年化利率。",
            aliases=["利率", "年化利率", "用户利率", "利率数值"],
            samples=["2.35", "3.12", "4.58"],
            value_range="0-100",
            data_distribution="不同用户分层对应不同利率。",
            business_usage="用于展示或分析用户当前适用的年化利率。",
            semantic_role="output_metric",
            index_texts=IndexTextBundle(
                keyword_text=(
                    "interest_rate 利率 年化利率 用户利率 利率数值 "
                    "用户利率表 利率信息表"
                ),
                vector_text=(
                    "字段名：interest_rate。所属表：interest_info。"
                    "该字段表示用户对应年化利率，通常作为查询结果输出。"
                ),
                rerank_text=(
                    "数据库：trade_db\n"
                    "表名：interest_info\n"
                    "字段名：interest_rate\n"
                    "数据类型：decimal\n"
                    "字段含义：用户对应年化利率。\n"
                    "字段别名：利率、年化利率、用户利率、利率数值。\n"
                    "取值范围：0-100。\n"
                    "业务用途：当用户询问利率是多少时，通常作为输出字段。"
                ),
            ),
        ),
    ]

def build_demo_relations() -> List[TableRelation]:
    """
    构造 Demo 表关系。

    表间 join 信息不要写进字段 business_usage。
    关系应该作为独立的图边存储。
    """
    return [
        TableRelation(
            database="trade_db",
            source_table="trade_summary",
            source_column="user_id",
            target_table="interest_info",
            target_column="user_id",
            relation_type="business_relation",
            description="通过用户ID关联用户交易汇总信息与用户利率信息。",
```

```
        )
    ]

def main() -> None:
    """
    运行示例。

    安装依赖:
        pip install pymilvus milvus-lite numpy

    执行:
        python -m schema_indexing.build_index_demo

    如果要使用阿里云真实 Embedding:
        export DASHSCOPE_API_KEY="你的APIKey"
    """
    embedding_dim = 1024

    embedding_client = AliyunEmbeddingClient(
        AliyunEmbeddingConfig(
            api_key=os.getenv("DASHSCOPE_API_KEY", ""),
            model="text-embedding-v4",
            dimensions=embedding_dim,
            use_mock_when_no_api_key=True,
        )
    )

    milvus_client = MilvusSchemaIndexClient(
        MilvusSchemaIndexConfig(
            uri=os.getenv("SCHEMA_MILVUS_URI", "./schema_index_demo.db"),
            token=os.getenv("SCHEMA_MILVUS_TOKEN", ""),
            field_collection_name=os.getenv("SCHEMA_FIELD_COLLECTION",
"schema_field_index_demo"),
            relation_collection_name=os.getenv("SCHEMA_RELATION_COLLECTION",
"schema_relation_index_demo"),
            embedding_dim=embedding_dim,
            recreate_collection=True,
            metric_type="COSINE",
        )
    )

    index_builder = SchemaIndexBuilder(
        text_builder=SchemaIndexTextBuilder(),
        embedding_client=embedding_client,
        milvus_client=milvus_client,
    )

```

```
    columns = build_demo_columns()
    relations = build_demo_relations()

    print("=" * 80)
    print("Schema Indexing Demo")
    print("=" * 80)

    print("\n[1] 构建字段三级索引文本: ")
    documents = index_builder.build_documents_only(columns)

    for index, doc in enumerate(documents, start=1):
        print(f"\n{index}. {doc.doc_id}")
        print(f"keyword_text: {doc.keyword_text}")
        print(f"vector_text : {doc.vector_text}")
        print(f"rerank_text : {doc.rerank_text}")

    print("\n[2] 写入字段索引和表关系索引: ")
    result = index_builder.build(
        columns=columns,
        relations=relations,
    )

    print(f"field_collection_name   : {result.field_collection_name}")
    print(f"relation_collection_name: {result.relation_collection_name}")
    print(f"field_document_count    : {result.field_document_count}")
    print(f"relation_count          : {result.relation_count}")

    print("\n[3] 简单字段向量检索验证: ")
    query_vector = embedding_client.embed_texts(["总交易笔数"])
[0].astype(float).tolist()
    search_results = milvus_client.search_fields(
        query_embedding=query_vector,
        top_k=3,
    )

    hit_tables = []

    for rank, item in enumerate(search_results, start=1):
        entity = item.get("entity", {})
        table_name = entity.get("table_name")
        column_name = entity.get("column_name")

        if table_name and table_name not in hit_tables:
            hit_tables.append(table_name)

        print(
            f"{rank}. "
```

```
            f"{table_name}.{column_name} "
            f"score={item.get('distance')}"
        )
        print(f"   vector_text: {entity.get('vector_text')}")

    print("\n[4] 根据命中字段所属表查询表关系: ")

    # Demo 中人为加入 interest_info，模拟两阶段检索同时命中了交易字段和利率字段。
    for table_name in ["interest_info"]:
        if table_name not in hit_tables:
            hit_tables.append(table_name)

    table_pairs = list(combinations(hit_tables, 2))

    print(f"命中表集合: {hit_tables}")
    print(f"两两组合  : {table_pairs}")

    graph_relations = milvus_client.query_relations_by_table_pairs(
        database="trade_db",
        table_pairs=table_pairs,
    )

    for relation in graph_relations:
        print(f"- {relation.join_condition}; {relation.description}")

if __name__ == "__main__":
    main()
```

测试指令与结果：

代码块

```
export DASHSCOPE_API_KEY="你的阿里云百炼APIKey"
export DASHSCOPE_WORKSPACE_ID="你的WorkspaceId"
测试指令：
python -m schema_indexing.build_index_demo
本地会生成一个schema_index_demo.db，本来Mivus应该单独部署的，这里是本地部署的

测试结果：
================================================================================
Schema Indexing Demo
================================================================================

```

[1] 构建字段三级索引文本：

1. trade_db.trade_summary.user_id
keyword_text: trade_db
trade_summary
user_id
int
用户唯一标识。
用户ID 客户ID
用户交易汇总表 交易统计表
1001 1002 1003
用于唯一标识交易汇总表中的用户。
join_key
vector_text : 字段名：user_id。所属表：trade_summary。表格含义：该表记录用户交易汇总信息，包括用户累计交易笔数、交易金额以及交易活跃度等内容。。表别名：用户交易汇总表，交易统计表。字段含义：用户唯一标识。。字段别名：用户ID，客户ID。业务用途：用于唯一标识交易汇总表中的用户。。字段角色：join_key
rerank_text : 数据库：trade_db
表名：trade_summary
字段名：user_id
数据类型：int
表格摘要：该表记录用户交易汇总信息，包括用户累计交易笔数、交易金额以及交易活跃度等内容。
表别名：用户交易汇总表，交易统计表
字段含义：用户唯一标识。
字段别名：用户ID，客户ID
字段角色：join_key
业务用途：用于唯一标识交易汇总表中的用户。
样例值：1001，1002，1003
主键信息：该字段是主键

2. trade_db.trade_summary.total_trade_count
keyword_text: total_trade_count 总交易笔数 累计交易笔数 交易笔数 交易次数 高频交易 用户交易汇总表 交易统计表
vector_text : 字段名：total_trade_count。所属表：trade_summary。该字段表示用户累计交易总笔数，用于衡量用户交易活跃程度，可用于筛选高频交易用户。
rerank_text : 数据库：trade_db
表名：trade_summary
字段名：total_trade_count
数据类型：int
字段含义：用户累计交易总笔数。
字段别名：总交易笔数、累计交易笔数、交易笔数、交易次数。
取值范围：整数，>= 0。
数据分布：高频交易用户可能超过50000。
业务用途：用于筛选总交易笔数大于某个阈值的用户。

3. trade_db.interest_info.user_id
keyword_text: trade_db

```
interest_info
user_id
int
用户唯一标识。
用户ID 客户ID
用户利率表 利率信息表
1001 1002 1003
用于唯一标识利率信息表中的用户。
join_key
vector_text : 字段名：user_id。所属表：interest_info。表格含义：该表记录用户利率信息，包括用户利率类型、利率数值以及利率生效状态等内容。。表别名：用户利率表，利率信息表。字段含义：用户唯一标识。。字段别名：用户ID，客户ID。业务用途：用于唯一标识利率信息表中的用户。。字段角色：join_key
rerank_text : 数据库：trade_db
表名：interest_info
字段名：user_id
数据类型：int
表格摘要：该表记录用户利率信息，包括用户利率类型、利率数值以及利率生效状态等内容。
表别名：用户利率表，利率信息表
字段含义：用户唯一标识。
字段别名：用户ID，客户ID
字段角色：join_key
业务用途：用于唯一标识利率信息表中的用户。
样例值：1001，1002，1003
主键信息：该字段是主键

4. trade_db.interest_info.interest_rate
keyword_text: interest_rate 利率 年化利率 用户利率 利率数值 用户利率表 利率信息表
vector_text : 字段名：interest_rate。所属表：interest_info。该字段表示用户对应年化利率，通常作为查询结果输出。
rerank_text : 数据库：trade_db
表名：interest_info
字段名：interest_rate
数据类型：decimal
字段含义：用户对应年化利率。
字段别名：利率、年化利率、用户利率、利率数值。
取值范围：0-100。
业务用途：当用户询问利率是多少时，通常作为输出字段。

[2] 写入字段索引和表关系索引:
field_collection_name   : schema_field_index_demo
relation_collection_name: schema_relation_index_demo
field_document_count    : 4
relation_count          : 1

[3] 简单字段向量检索验证:
1. trade_summary.user_id score=0.2981129586696625
```

```
97      vector_text: 字段名：user_id。所属表：trade_summary。表格含义：该表记录用户交易汇总信息，包括用户累计交易笔数、交易金额以及交易活跃度等内容。。表别名：用户交易汇总表，交易统计表。字段含义：用户唯一标识。。字段别名：用户ID，客户ID。业务用途：用于唯一标识交易汇总表中的用户。。字段角色：join_key
98  2. trade_summary.total_trade_count score=0.2543160021305084
99      vector_text: 字段名：total_trade_count。所属表：trade_summary。该字段表示用户累计交易总笔数，用于衡量用户交易活跃程度，可用于筛选高频交易用户。
100 3. interest_info.interest_rate score=0.02609090320765972
101     vector_text: 字段名：interest_rate。所属表：interest_info。该字段表示用户对应年化利率，通常作为查询结果输出。
102
103 [4] 根据命中字段所属表查询表关系：
104 命中表集合: ['trade_summary', 'interest_info']
105 两两组合  : [('trade_summary', 'interest_info')]
106 - trade_summary.user_id = interest_info.user_id；通过用户ID关联用户交易汇总信息与用户利率信息。
107
```

Schema检索与召回(schema_retrieval)

🔥 `schema_retrieval` 模块主要负责根据用户自然语言 Query 检索相关数据库 Schema，并构建后续 CoT 生成和 SQL 生成所需的局部 SchemaGraph。

该模块首先解析数据库中的表、字段和表间关系，并以字段为基本检索单元构建检索文档。随后通过大模型提取 Query 中的核心关键词，分别执行 BM25 关键词召回和向量语义召回，再使用 RRF 对两路结果进行融合，并通过 Rerank 模型完成最终精排。

在获得相关字段后，系统根据字段所属表及表间关联关系构建 Schema 子图，最终输出与当前 Query 相关的表、字段和 Join 关系。

主要功能包括：

- 解析 SQLite 数据库 Schema；
- 构建字段级检索文档；
- 对中英文 Query 进行分词和关键词提取；
- 执行 BM25 关键词召回；
- 执行 Embedding 向量语义召回；
- 使用 RRF 融合关键词和向量召回结果；
- 使用 Rerank 模型进行相关性精排；
- 根据召回字段构建 SchemaGraph；
- 提供关键词召回、向量召回、混合召回和完整 Schema 检索流程的示例。

目录结构

```
schema_retrieval/
├── __init__.py
│
├── objects.py                      # 核心数据结构
├── sqlite_loader.py                # SQLite Schema 解析
├── document_builder.py             # 字段级检索文档构建
├── graph_builder.py                # Schema 子图构建
│
├── tokenizer.py                    # 中英文混合分词
├── bm25.py                         # BM25 关键词索引
├── retriever.py                    # 关键词召回主流程
│
├── keyword_extractor_client.py     # 大模型关键词抽取服务端
├── embedding_client.py             # 阿里云向量服务端
├── vector_index.py                 # 内存向量索引
│
├── rrf_fusion_client.py            # RRF 融合
├── rerank_client.py                # Rerank 精排
├── hybrid_schema_retrieval_service.py #混合排序+Rerank服务端
│
├── keyword_recall_demo.py          # 关键词召回 Demo
├── vector_recall_demo.py           # 向量召回 Demo
├── hybrid_recall_demo.py           # 关键词 + 向量 + RRF 融合 Demo
├── hybrid_rerank_schema_demo.py # 混合召回 + Rerank + SchemaGraph Demo
│
└── README.md
```

关键词检索/retriever.py

🔥 `retriever.py` 主要负责字段级 Schema 的关键词检索。该文件以字段检索文档为基础构建 BM25 索引，并结合字段名称、字段别名、表名、表别名和业务描述等信息进行规则加分，最终返回与用户 Query 相关的字段结果。

SchemaRetriever

`SchemaRetriever` 是字段级 Schema 关键词检索器，负责构建字段文档、初始化 BM25 索引并执行关键词召回。

初始化时接收以下参数：

- `tables`：数据库中的表级 Schema 集合；
- `columns`：字段级 Schema 列表，是实际参与检索的基本数据；

- `relations`：表间关联关系列表，用于保留完整 Schema 信息；
- `documents`：根据 `ColumnSchema` 构建的字段级检索文档；
- `keyword_index`：BM25 关键词索引，调用 `build()` 后完成初始化。

`build()` 方法用于提取所有字段文档中的 `keyword_text`，并基于这些文本构建 BM25 索引。执行字段检索前必须先调用该方法。

`retrieve()` 方法用于根据用户 Query 执行关键词召回，主要包括：

- 使用 BM25 召回候选字段；
- 将初始召回数量扩大到目标数量的三倍，且不少于 20 条；
- 对候选字段执行业务规则加分；
- 将 BM25 得分和业务加分合并；
- 按最终得分降序排序；
- 返回前 `top_k` 个 `SchemaHit`。

`_business_boost()` 方法用于对 BM25 召回结果进行简单的业务规则加分，主要包括：

- Query 中直接包含字段名时加分；
- Query 中直接包含表名时加分；
- Query 命中字段别名时加分；
- Query 命中表别名时加分；
- Query 分词与字段描述或表描述存在重合时加分。

其中字段别名的加分权重最高，用于增强业务术语与实际字段之间的精确匹配能力。

向量检索/keyword_extractor_client.py，vector_index.py

🔥 `keyword_extractor_client.py` 主要负责调用阿里云百炼大模型，从用户自然语言 Query 中提取核心检索词，为后续 BM25 关键词召回和向量语义召回提供输入。

关键词抽取过程尽量保留用户原始表达，并重点识别字段名称、业务指标、表级概念和关键业务限定词。模型输出会经过正则解析、分隔和去重，最终转换为标准关键词列表。

`vector_index.py` 主要负责构建字段级内存向量索引，并根据用户 Query 执行语义相似度召回。

当前实现使用 NumPy 矩阵保存字段向量，通过余弦相似度计算 Query 与各字段文档之间的相关性。该实现主要用于 Demo 和流程验证，生产环境中可以替换为 Milvus、FAISS、PGVector 或 Elasticsearch 等向量检索组件。

AliyunKeywordExtractor

🔥 `AliyunKeywordExtractor` 是基于大模型的关键词抽取客户端，负责构造 Prompt、调用阿里云百炼接口，并将模型返回内容解析为关键词列表。

`SYSTEM_PROMPT` 用于约束关键词抽取规则，主要包括：

- 提取字段、指标、表格和业务概念；
- 尽量保留用户原始表达；
- 保留完整的聚合语义和业务限定词；
- 不输出分析过程和额外说明；
- 使用统一格式返回多个关键词。

`init()` 方法用于初始化关键词抽取客户端。该方法会读取 API Key 和接口地址，并在缺少 API Key 时抛出异常，避免在未配置服务的情况下继续调用。

`extract()` 方法用于接收用户 Query，并返回核心检索词列表。该方法依次调用大模型接口和关键词解析方法，是外部使用时的主要入口。

`_call_llm()` 方法用于构造 Chat Completions 请求，调用阿里云百炼大模型，并提取模型返回的文本内容。请求中的 `temperature` 设置为 0，以降低关键词抽取结果的随机性。

`_parse_keywords()` 方法用于解析模型输出，主要包括：

- 提取“解析结果”后的关键词文本；
- 按中英文逗号、分号和换行符进行切分；
- 去除空内容和重复关键词；
- 返回按原顺序保留的关键词列表。

如果模型输出中未解析出有效关键词，该方法会抛出异常，避免空检索词进入后续召回流程。

VectorIndex

`VectorIndex` 是内存向量索引类，负责生成、保存和检索字段级文本向量。

初始化时接收以下参数：

- `embedding_client`：Embedding 服务客户端，用于将字段文本和用户 Query 转换为向量；
- `documents`：当前索引中保存的字段级向量文本列表；
- `embeddings`：字段文本对应的向量矩阵，索引构建前为空。

`build()` 方法用于构建内存向量索引，主要包括：

- 保存待索引的字段级文本；
- 调用 Embedding 服务批量生成字段向量；
- 将生成的向量保存为 NumPy 矩阵。

`search()` 方法用于根据用户 Query 执行向量召回，主要包括：

- 检查向量索引是否已经构建；
- 将用户 Query 转换为向量；
- 计算 Query 向量与所有字段向量的相似度；
- 按相似度从高到低排序；
- 返回前 `top_k` 个文档下标及对应得分。

由于字段向量和 Query 向量均已进行 L2 归一化，因此代码中直接使用向量点积计算余弦相似度。

混合检索融合/rrf_fusion_client.py

🔥 `rrf_fusion_client.py` 主要负责融合关键词召回和向量召回结果。该文件使用 RRF（Reciprocal Rank Fusion）算法，根据字段在不同召回通道中的排名计算融合得分，并输出统一排序后的候选字段。

融合过程不直接依赖各召回通道的原始分数，而是使用结果排名进行计算，可以降低关键词得分和向量相似度得分量纲不同带来的影响。

RouteRecallResult

🔥 `RouteRecallResult` 用于记录单个检索词在某个召回通道中的排序结果。

主要参数如下：

- `route_name`：召回通道名称，例如 `keyword` 或 `vector`；
- `query_term`：当前执行召回的检索词；
- `ranked_doc_indices`：按照相关性排序的字段文档下标。

RRFFusionHit

🔥 `RRFFusionHit` 用于表示经过 RRF 融合后的候选字段结果。

主要参数如下：

- `doc_index`：字段文档下标；
- `score`：字段的 RRF 融合得分；
- `matched_terms`：该字段命中的检索词；
- `sources`：该字段被哪些召回通道命中；

- `best_rank_by_source`：该字段在各召回通道中的最好排名。

RRFFusionClient

> 🔥 `RRFFusionClient` 是 RRF 融合客户端，负责对多个关键词召回和向量召回结果进行统一排序。

初始化时接收 `RRFFusionConfig` 配置。未传入配置时使用默认参数。

`fuse()` 方法用于执行 RRF 融合，主要包括：

- 遍历不同检索词和召回通道的排序结果；
- 根据通道权重、RRF 平滑参数和字段排名计算融合得分；
- 累加同一字段在不同检索词和召回通道中的得分；
- 记录字段命中的检索词、召回来源及各通道最好排名；
- 根据关键词数量动态截断融合候选；
- 按融合得分降序返回最终候选字段。

单次排名的得分计算方式为：

```
代码块
1   通道权重 /（rrf_k + 当前排名）
```

字段在多个检索词或多个召回通道中重复出现时，其得分会持续累加，因此能够同时被关键词召回和向量召回命中的字段通常具有更高排名。

`calculate_fused_top_k()` 方法用于根据关键词数量动态计算融合候选数量。计算结果会被限制在配置的最小值和最大值之间，避免关键词过少导致候选不足，或关键词过多导致候选范围过大。

Rerank重排序/rerank_client.py

> 🔥 `rerank_client.py` 主要负责对 RRF 融合后的候选字段进行二次排序。该文件调用阿里云百炼 `qwen3-rerank` 模型，根据用户原始 Query 与候选字段文本之间的相关性重新计算得分，并返回排序后的字段结果。

RerankDocument

> 🔥 `RerankDocument` 用于表示待重排序的候选文档。

主要参数如下：

- `doc_index`：候选文档在原始字段文档列表中的下标；
- `text`：参与 Rerank 的候选字段文本，通常使用信息较完整的 `rerank_text`。

RerankResult

> 🔥 `RerankResult` 用于表示模型重排序后的结果。

主要参数如下：

- `doc_index`：候选字段在原始文档列表中的下标；
- `rerank_score`：Rerank 模型输出的相关性分数；
- `text`：候选字段对应的文本内容。

AliyunRerankClient

> 🔥 `AliyunRerankClient` 是阿里云百炼 Rerank 客户端，负责接收用户 Query 和候选字段文档，并返回重新排序后的候选结果。

`init()` 方法用于初始化 Rerank 客户端，主要包括：

- 读取 API Key；
- 读取业务空间 ID；
- 根据业务空间 ID 自动生成 Rerank 接口地址；
- 保存模型、超时时间和任务说明等配置。

`rerank()` 方法用于执行候选字段重排序，主要包括：

- 检查候选文档是否为空；
- 限制 `top_n` 不超过候选文档数量；
- 配置完整时调用阿里云百炼 Rerank 服务；
- 配置不完整时使用本地 Mock 排序；
- 返回前 `top_n` 个重排序结果。

`_call_aliyun_rerank()` 方法用于调用阿里云百炼 `qwen3-rerank` 接口，主要包括：

- 提取候选文档文本；
- 构造模型请求参数；
- 调用 Rerank API；

- 解析模型返回的文档位置和相关性分数；
- 将模型返回的局部下标映射回原始 `doc_index`；
- 按相关性得分降序返回结果。

`_mock_rerank()` 方法用于在未配置真实服务时完成本地模拟排序。该方法根据 Query 和候选文本中是否包含指定业务词及字段名进行规则加分，再按照得分排序。

Mock 排序规则仅适用于当前示例场景，不能替代真实 Rerank 模型的语义相关性判断。

两阶段检索与召回/hybrid_schema_retrieval_service.py

🔥 `hybrid_schema_retrieval_service.py` 负责组织完整的 Schema 混合检索流程，将**关键词抽取、BM25 召回、向量召回、RRF 融合、Rerank 精排和 SchemaGraph 构建串联起来**。

该文件是 Schema 检索模块的主服务入口。输入用户自然语言 Query 后，系统最终返回相关字段、各阶段检索结果以及面向 CoT 和 SQL 生成的 Schema 子图。

完整流程包括：

- 获取用户 Query 的核心检索词；
- 针对Query执行 BM25 关键词召回；
- 针对每个关键词执行 Embedding 向量召回；
- 使用 RRF 融合多关键词、多通道召回结果；
- 使用 Rerank 模型对融合候选进行精排；
- 将精排结果转换为统一的 `SchemaHit`；
- 根据相关字段、表信息和表间关系构建 SchemaGraph。

HybridSchemaRetrievalService

🔥 `HybridSchemaRetrievalService` 是混合 Schema 检索的主服务类，负责初始化检索数据、构建索引并执行完整检索流程。

初始化时接收以下参数：

- `tables`：数据库中的表级 Schema 集合；
- `columns`：字段级 Schema 列表；
- `relations`：表间关联关系列表；
- `business_meta`：表和字段对应的业务元数据；
- `embedding_client`：Embedding 向量生成客户端；

- `rerank_client`：Rerank 精排客户端；
- `keyword_extractor`：可选的关键词抽取客户端；
- `config`：混合检索配置。

初始化过程中会自动补充字段业务元数据、构建三级检索文档，并创建 BM25 索引和内存向量索引。

`from_sqlite()` 方法用于从 SQLite 数据库创建混合检索服务。该方法通过 `SQLiteSchemaLoader` 解析数据库中的表、字段和关联关系，再初始化完整的检索服务。

`retrieve()` 方法用于执行完整的混合 Schema 检索，主要包括：

- 获取核心检索词；
- 执行关键词召回和向量召回；
- 对多路召回结果进行 RRF 融合；
- 使用用户原始 Query 对候选字段进行 Rerank；
- 将精排结果转换为 `SchemaHit`；
- 根据最终字段和表间关系构建 SchemaGraph；
- 返回各阶段结果和最终 Schema 子图。

`_resolve_keywords()` 方法用于确定本次检索使用的关键词。调用方传入关键词时直接使用；未传入时调用大模型关键词抽取客户端。两种方式都不可用时抛出异常。

`_build_route_results()` 方法用于针对每个关键词分别执行 BM25 和向量召回，并将各通道的字段排名封装为 `RouteRecallResult`，供 RRF 融合使用。

`_rerank()` 方法用于将 RRF 融合后的字段转换为 `RerankDocument`，并调用 Rerank 客户端进行精排。参与精排的文本使用信息更完整的 `rerank_text`。

`_calculate_rerank_top_n()` 方法用于根据关键词数量动态计算 Rerank 输出数量。结果会受到最小值、最大值和实际候选数量限制。

`_build_schema_hits()` 方法用于将 Rerank 结果转换为 `SchemaHit`，并补充关键词排名、向量排名、RRF 得分和 Rerank 得分。代码同时兼容不包含这些扩展字段的旧版 `SchemaHit`。

`_build_indices()` 方法用于构建 BM25 索引和向量索引。BM25 使用 `keyword_text`，向量索引使用 `vector_text`。

`_build_documents()` 方法用于为每个字段构建 `HybridFieldDocument`。三级索引文本按照以下优先级获取：

- 业务元数据中显式配置的索引文本；
- `ColumnSchema.index_texts` 中已有的索引文本；
- 根据字段信息临时构建的默认索引文本。

`_enrich_columns_with_business_meta()` 方法用于将业务元数据补充到 `ColumnSchema`，包括表描述、表别名、字段描述、字段别名、字段角色、数据范围、数据分布、

业务用途和样例值。

`_build_default_keyword_text()`、`_build_default_vector_text()` 和 `_build_default_rerank_text()` 用于在未提供三级索引文本时，根据字段 Schema 信息生成默认文本，保证检索流程可以正常执行。

测试Demo/keyword_recall_demo.py，vector_recall_demo.py，hybrid_recall_demo.py，hybrid_rerank_schema_demo.py

关键词检索

keyword_recall_demo.py

```python
from __future__ import annotations

import sqlite3
import tempfile
from pathlib import Path

from schema_retrieval.graph_builder import build_schema_graph
from schema_retrieval.retriever import SchemaRetriever
from schema_retrieval.sqlite_loader import SQLiteSchemaLoader

def create_demo_database() -> Path:
    """
    创建一个最小 Demo 数据库：
    - users: 用户维表
    - orders: 订单事实表
    """
    db_path = Path(tempfile.gettempdir()) / "text2sql_schema_retrieval_demo.db"

    if db_path.exists():
        db_path.unlink()

    conn = sqlite3.connect(str(db_path))
    conn.execute("PRAGMA foreign_keys = ON")

    conn.executescript(
        '''
        CREATE TABLE users (
            user_id INTEGER PRIMARY KEY,
            city TEXT,
            gender TEXT,
            register_time TEXT
        );

        CREATE TABLE orders (
            order_id INTEGER PRIMARY KEY,
```

```
            user_id INTEGER NOT NULL,
            order_amount REAL NOT NULL,
            order_time TEXT NOT NULL,
            status TEXT,
            FOREIGN KEY (user_id) REFERENCES users(user_id)
        );
        '''
    )

    conn.executemany(
        '''
        INSERT INTO users (user_id, city, gender, register_time)
        VALUES (?, ?, ?, ?)
        ''',
        [
            (1, "北京", "男", "2024-01-01"),
            (2, "上海", "女", "2024-02-01"),
            (3, "深圳", "女", "2024-03-01"),
        ],
    )

    conn.executemany(
        '''
        INSERT INTO orders (order_id, user_id, order_amount, order_time,
status)
        VALUES (?, ?, ?, ?, ?)
        ''',
        [
            (101, 1, 99.0, "2024-04-01", "已支付"),
            (102, 1, 199.0, "2024-04-02", "已支付"),
            (103, 2, 88.0, "2024-04-02", "已取消"),
            (104, 3, 299.0, "2024-04-03", "已支付"),
        ],
    )

    conn.commit()
    conn.close()

    return db_path

def get_business_meta() -> dict:
    """
    表级和字段级业务元数据。

    真实项目里建议从 YAML / JSON / 元数据平台读取。
    """
    return {
```

```
        "orders": {
            "description": "订单事实表，记录用户每一笔交易订单，包括订单金额、订单时间、订单状态和下单用户。",
            "aliases": ["订单表", "交易表", "销售订单"],
            "columns": {
                "order_id": {
                    "description": "订单唯一标识。",
                    "aliases": ["订单ID", "订单编号"],
                },
                "user_id": {
                    "description": "下单用户ID，用于关联用户表。",
                    "aliases": ["用户ID", "客户ID", "下单用户"],
                },
                "order_amount": {
                    "description": "订单实付金额，可用于计算销售额、GMV、收入、客单价等指标。",
                    "aliases": ["销售额", "GMV", "实付金额", "订单金额", "收入"],
                },
                "order_time": {
                    "description": "订单创建时间，可用于按天、月、季度统计。",
                    "aliases": ["下单时间", "订单时间", "交易时间"],
                },
                "status": {
                    "description": "订单状态，例如已支付、已取消、已退款。",
                    "aliases": ["订单状态", "支付状态", "交易状态"],
                },
            },
        },
        "users": {
            "description": "用户维度表，记录用户的基础属性，包括城市、性别和注册时间。",
            "aliases": ["用户表", "客户表"],
            "columns": {
                "user_id": {
                    "description": "用户唯一标识。",
                    "aliases": ["用户ID", "客户ID"],
                },
                "city": {
                    "description": "用户所在城市，可作为地域分析维度。",
                    "aliases": ["城市", "地区", "用户城市"],
                },
                "gender": {
                    "description": "用户性别。",
                    "aliases": ["性别"],
                },
                "register_time": {
                    "description": "用户注册时间。",
```

```
                    "aliases": ["注册时间", "开户时间"],
                },
            },
        },
    }

def main() -> None:
    db_path = create_demo_database()

    print("=" * 80)
    print("Schema Retrieval Demo")
    print("=" * 80)
    print(f"Demo database: {db_path}")

    loader = SQLiteSchemaLoader(
        db_path=db_path,
        database_name="demo_db",
        business_meta=get_business_meta(),
        sample_size=5,
    )

    tables, columns, relations = loader.load()

    print("\n[1] Loaded tables:")
    for table in tables.values():
        print(f"- {table.table_name}: {table.description}")

    print("\n[2] Loaded columns:")
    for column in columns:
        pk_text = " PK" if column.is_primary_key else ""
        fk_text = f" FK->{column.foreign_key_ref}" if column.foreign_key_ref
else ""
        print(
            f"- {column.table_name}.{column.column_name} "
            f"({column.data_type}){pk_text}{fk_text}, "
            f"aliases={column.aliases}, samples={column.samples}"
        )

    retriever = SchemaRetriever(
        tables=tables,
        columns=columns,
        relations=relations,
    )
    retriever.build()

    query = "每个城市的销售额是多少？"

```

```
    hits = retriever.retrieve(query, top_k=6)

    print("\n[3] Query:")
    print(query)

    print("\n[4] Retrieved fields:")
    for idx, hit in enumerate(hits, start=1):
        col = hit.column
        print(
            f"{idx}. {col.table_name}.{col.column_name} "
            f"score={hit.score:.4f} "
            f"description={col.description}"
        )

    schema_graph = build_schema_graph(
        hits=hits,
        tables=tables,
        all_columns=columns,
        relations=relations,
        include_join_columns=True,
    )

    print("\n[5] Schema graph prompt context:")
    print(schema_graph.to_prompt_context())

if __name__ == "__main__":
    main()

```

测试指令与结果：

代码块

```
测试指令：
python -m schema_retrieval.keyword_recall_demo

测试结果：
================================================================================
Schema Retrieval Demo
================================================================================
Demo database: /tmp/text2sql_schema_retrieval_demo.db

[1] Loaded tables:
```

```
- orders: 订单事实表，记录用户每一笔交易订单，包括订单金额、订单时间、订单状态和下单用户。
- users: 用户维度表，记录用户的基础属性，包括城市、性别和注册时间。

[2] Loaded columns:
- orders.order_id (INTEGER) PK, aliases=['订单ID', '订单编号'], samples=['101', '102', '103', '104']
- orders.user_id (INTEGER) FK->users.user_id, aliases=['用户ID', '客户ID', '下单用户'], samples=['1', '2', '3']
- orders.order_amount (REAL), aliases=['销售额', 'GMV', '实付金额', '订单金额', '收入'], samples=['99.0', '199.0', '88.0', '299.0']
- orders.order_time (TEXT), aliases=['下单时间', '订单时间', '交易时间'], samples=['2024-04-01', '2024-04-02', '2024-04-03']
- orders.status (TEXT), aliases=['订单状态', '支付状态', '交易状态'], samples=['已支付', '已取消']
- users.user_id (INTEGER) PK, aliases=['用户ID', '客户ID'], samples=['1', '2', '3']
- users.city (TEXT), aliases=['城市', '地区', '用户城市'], samples=['北京', '上海', '深圳']
- users.gender (TEXT), aliases=['性别'], samples=['男', '女']
- users.register_time (TEXT), aliases=['注册时间', '开户时间'], samples=['2024-01-01', '2024-02-01', '2024-03-01']

[3] Query:
每个城市的销售额是多少?

[4] Retrieved fields:
1. orders.order_amount score=12.0254 description=订单实付金额，可用于计算销售额、GMV、收入、客单价等指标。
2. users.city score=10.2783 description=用户所在城市，可作为地域分析维度。
3. users.gender score=4.2125 description=用户性别。
4. users.user_id score=4.0740 description=用户唯一标识。
5. users.register_time score=3.9042 description=用户注册时间。
6. orders.order_id score=3.2450 description=订单唯一标识。

[5] Schema graph prompt context:
数据库：demo_db

表名：users
表格摘要：用户维度表，记录用户的基础属性，包括城市、性别和注册时间。
表别名：用户表，客户表
主键：user_id
字段：
- 字段名：city
  数据类型：TEXT
  字段含义：用户所在城市，可作为地域分析维度。
  字段别名：城市，地区，用户城市
```

```
    样例值：北京，上海，深圳
  - 字段名：gender
    数据类型：TEXT
    字段含义：用户性别。
    字段别名：性别
    样例值：男，女
  - 字段名：user_id
    数据类型：INTEGER
    字段含义：用户唯一标识。
    字段别名：用户ID，客户ID
    样例值：1，2，3
  - 字段名：register_time
    数据类型：TEXT
    字段含义：用户注册时间。
    字段别名：注册时间，开户时间
    样例值：2024-01-01，2024-02-01，2024-03-01

  表名：orders
  表格摘要：订单事实表，记录用户每一笔交易订单，包括订单金额、订单时间、订单状态和下单用户。
  表别名：订单表，交易表，销售订单
  主键：order_id
  字段：
  - 字段名：order_amount
    数据类型：REAL
    字段含义：订单实付金额，可用于计算销售额、GMV、收入、客单价等指标。
    字段别名：销售额，GMV，实付金额，订单金额，收入
    样例值：99.0，199.0，88.0，299.0
  - 字段名：order_id
    数据类型：INTEGER
    字段含义：订单唯一标识。
    字段别名：订单ID，订单编号
    样例值：101，102，103，104
  - 字段名：user_id
    数据类型：INTEGER
    字段含义：下单用户ID，用于关联用户表。
    字段别名：用户ID，客户ID，下单用户
    样例值：1，2，3

  表关联关系：
  - orders.user_id ↔ users.user_id
```

向量检索(语义检索)：

vector_recall_demo.py

```
from __future__ import annotations

```

```python
import hashlib
import json
import os
import re
import sqlite3
import tempfile
import urllib.request
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple

import numpy as np

from schema_retrieval.sqlite_loader import SQLiteSchemaLoader

# ============================================================================
# 1. 向量召回文档对象
# ============================================================================

@dataclass
class VectorFieldDocument:
    """
    字段级向量检索文档。

    该对象只用于当前向量召回 Demo，不依赖 document_builder.py。
    这样可以避免 objects.py 和 document_builder.py 还在逐步改造时影响 Demo 运行。
    """

    doc_id: str
    """字段文档唯一标识。"""

    column: object
    """字段 Schema 对象。"""

    vector_text: str
    """向量索引文本。"""

# ============================================================================
# 2. 关键词抽取
# ============================================================================

@dataclass
class KeywordExtractorConfig:
    """
    关键词抽取模型配置。

    当 api_key 为空时，使用本地规则抽取，便于无 API Key 环境下跑通 Demo。
```

```
    当 api_key 不为空时，调用阿里云百炼 OpenAI 兼容 Chat Completions 接口。
    """

    api_key: str = ""
    """阿里云百炼 API Key。为空时读取环境变量 DASHSCOPE_API_KEY。"""

    chat_url: str = "https://dashscope.aliyuncs.com/compatible-
mode/v1/chat/completions"
    """阿里云百炼 OpenAI 兼容 Chat Completions 接口地址。"""

    model: str = "qwen-plus"
    """用于关键词抽取的大模型名称。"""

    timeout: int = 60
    """HTTP 请求超时时间，单位为秒。"""

class KeywordExtractor:
    """
    基于大模型的核心检索词抽取器。

    该抽取器用于从用户 Query 中提取字段、指标、表级概念和关键业务语义信息。
    向量召回阶段不直接对完整 Query 做 Embedding，而是对抽取后的关键词做 Embedding。
    """

    SYSTEM_PROMPT = """你是一个字段解析助手。

任务：
从用户输入的查询中提取核心字段、业务指标、表级概念以及关键业务语义信息。

要求：
1. 提取字段、指标、表格或业务概念等核心关键词
2. 尽量保留用户原始表达，不进行语义改写
3. 不输出解释、分析过程或额外内容
4. 多个关键词之间使用全角逗号分隔
5. 若存在聚合语义，如数量、总金额、平均值等，需完整保留
6. 若存在业务限定词，如年化利率、总交易笔数等，禁止拆分

输出格式：
解析结果：<关键词1>，<关键词2>

示例1：
用户Query：查询用户年龄大于30的用户数量
解析结果：用户年龄，用户数量

示例2：
用户Query：统计每个城市的订单总金额
解析结果：城市，订单总金额
```

```
96
97  示例3:
98  用户Query: 查询总交易笔数大于50000的利率是多少
99  解析结果: 总交易笔数，利率
100 """
101
102     def __init__(self, config: KeywordExtractorConfig | None = None):
103         self.config = config or KeywordExtractorConfig()
104
105         if not self.config.api_key:
106             self.config.api_key = os.getenv("DASHSCOPE_API_KEY", "")
107
108     def extract(self, query: str) -> List[str]:
109         """
110         从用户 Query 中抽取核心检索词。
111
112         Args:
113             query: 用户自然语言查询。
114
115         Returns:
116             List[str]: 核心检索词列表。
117         """
118         if not self.config.api_key:
119             return self._extract_by_rule(query)
120
121         content = self._call_llm(query)
122         return self._parse_keywords(content)
123
124     def _call_llm(self, query: str) -> str:
125         """
126         调用阿里云百炼大模型完成关键词抽取。
127
128         Args:
129             query: 用户自然语言查询。
130
131         Returns:
132             str: 大模型返回的原始文本。
133         """
134         payload = {
135             "model": self.config.model,
136             "messages": [
137                 {
138                     "role": "system",
139                     "content": self.SYSTEM_PROMPT,
140                 },
141                 {
142                     "role": "user",
```

```
                    "content": f"用户Query: {query}",
                },
            ],
            "temperature": 0,
        }

        data = json.dumps(payload, ensure_ascii=False).encode("utf-8")

        headers = {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {self.config.api_key}",
        }

        request = urllib.request.Request(
            self.config.chat_url,
            data=data,
            headers=headers,
            method="POST",
        )

        try:
            with urllib.request.urlopen(request, timeout=self.config.timeout)
as response:
                body = response.read().decode("utf-8")
                result = json.loads(body)

        except Exception as exc:
            raise RuntimeError(f"调用关键词抽取模型失败: {exc}") from exc

        try:
            return result["choices"][0]["message"]["content"].strip()

        except Exception as exc:
            raise RuntimeError(f"解析关键词抽取模型返回失败，原始返回: {result}")
from exc

    def _parse_keywords(self, text: str) -> List[str]:
        """
        从模型输出中解析关键词。

        Args:
            text: 模型输出文本。

        Returns:
            List[str]: 去重后的关键词列表。
        """
        match = re.search(r"解析结果[:: ]\s*(.+)", text)
```

```
188
189         if match:
190             keyword_text = match.group(1).strip()
191         else:
192             keyword_text = text.strip()
193
194         raw_keywords = re.split(r"[, ,\n; ;]+", keyword_text)
195
196         keywords: List[str] = []
197         seen = set()
198
199         for item in raw_keywords:
200             keyword = item.strip()
201
202             if not keyword:
203                 continue
204
205             if keyword not in seen:
206                 keywords.append(keyword)
207                 seen.add(keyword)
208
209         if not keywords:
210             raise ValueError(f"未能从模型输出中解析到关键词，模型输出为：{text}")
211
212         return keywords
213
214     def _extract_by_rule(self, query: str) -> List[str]:
215         """
216         本地规则抽取。
217
218         仅用于无 API Key 环境下跑通 Demo。生产环境建议使用大模型抽取。
219         """
220         known_phrases = [
221             "订单总金额",
222             "总交易笔数",
223             "累计交易笔数",
224             "交易笔数",
225             "交易次数",
226             "年化利率",
227             "利率",
228             "城市",
229             "销售额",
230             "订单金额",
231             "用户数量",
232             "用户年龄",
233         ]
234
```

```
        keywords: List[str] = []
        seen = set()

        for phrase in known_phrases:
            if phrase in query and phrase not in seen:
                keywords.append(phrase)
                seen.add(phrase)

        if not keywords:
            normalized = re.sub(r"[? ?。 .\s]+", "", query)
            if normalized:
                keywords.append(normalized)

        return keywords

# ================================================================================
# 3. Embedding Client
# ================================================================================

@dataclass
class AliyunEmbeddingConfig:
    """
    阿里云百炼 Embedding 配置。

    当 api_key 为空时，使用本地 Hash Embedding，便于无 API Key 环境下跑通 Demo。
    当 api_key 不为空时，调用阿里云百炼 OpenAI 兼容 Embedding 接口。
    """

    api_key: str = ""
    """阿里云百炼 API Key。为空时读取环境变量 DASHSCOPE_API_KEY。"""

    embedding_url: str = "https://dashscope.aliyuncs.com/compatible-
mode/v1/embeddings"
    """阿里云百炼 OpenAI 兼容 Embedding 接口地址。"""

    model: str = "text-embedding-v4"
    """Embedding 模型名称。"""

    dimensions: int = 1024
    """Embedding 向量维度。"""

    timeout: int = 60
    """HTTP 请求超时时间，单位为秒。"""

class AliyunEmbeddingClient:
    """
    阿里云百炼 Embedding Client。
```

```python

    该 Client 用于将字段索引文本和抽取后的核心检索词转换为向量。
    """

    def __init__(self, config: AliyunEmbeddingConfig | None = None):
        self.config = config or AliyunEmbeddingConfig()

        if not self.config.api_key:
            self.config.api_key = os.getenv("DASHSCOPE_API_KEY", "")

    def embed_texts(self, texts: List[str]) -> np.ndarray:
        """
        批量生成文本向量。

        Args:
            texts: 待向量化的文本列表。

        Returns:
            np.ndarray: 归一化后的向量矩阵。
        """
        if not texts:
            return np.empty((0, self.config.dimensions), dtype=np.float32)

        if not self.config.api_key:
            return self._local_hash_embedding(texts)

        return self._call_aliyun_embedding(texts)

    def _call_aliyun_embedding(self, texts: List[str]) -> np.ndarray:
        """
        调用阿里云百炼 Embedding API。

        Args:
            texts: 待向量化的文本列表。

        Returns:
            np.ndarray: 归一化后的向量矩阵。
        """
        payload = {
            "model": self.config.model,
            "input": texts,
            "dimensions": self.config.dimensions,
            "encoding_format": "float",
        }

        data = json.dumps(payload, ensure_ascii=False).encode("utf-8")

```

```
328         headers = {
329             "Content-Type": "application/json",
330             "Authorization": f"Bearer {self.config.api_key}",
331         }
332 
333         request = urllib.request.Request(
334             self.config.embedding_url,
335             data=data,
336             headers=headers,
337             method="POST",
338         )
339 
340         try:
341             with urllib.request.urlopen(request, timeout=self.config.timeout)
    as response:
342                 body = response.read().decode("utf-8")
343                 result = json.loads(body)
344 
345         except Exception as exc:
346             raise RuntimeError(f"调用阿里云 Embedding API 失败: {exc}") from exc
347 
348         try:
349             data_items = sorted(result["data"], key=lambda item:
    item.get("index", 0))
350             embeddings = [item["embedding"] for item in data_items]
351 
352         except Exception as exc:
353             raise RuntimeError(f"解析 Embedding API 返回结果失败，原始返回:
    {result}") from exc
354 
355         vectors = np.array(embeddings, dtype=np.float32)
356         return self._normalize(vectors)
357 
358     def _local_hash_embedding(self, texts: List[str]) -> np.ndarray:
359         """
360         本地 Hash Embedding。
361 
362         仅用于无 API Key 环境下验证流程，不代表真实语义向量效果。
363         """
364         vectors = np.zeros((len(texts), self.config.dimensions),
    dtype=np.float32)
365 
366         for row_index, text in enumerate(texts):
367             for token in self._tokenize_for_hash(text):
368                 digest = hashlib.md5(token.encode("utf-8")).hexdigest()
369                 col_index = int(digest, 16) % self.config.dimensions
370                 vectors[row_index, col_index] += 1.0
```

```python

        return self._normalize(vectors)

    def _normalize(self, vectors: np.ndarray) -> np.ndarray:
        """
        对向量进行 L2 归一化。

        Args:
            vectors: 原始向量矩阵。

        Returns:
            np.ndarray: 归一化后的向量矩阵。
        """
        norms = np.linalg.norm(vectors, axis=1, keepdims=True)
        return vectors / np.maximum(norms, 1e-8)

    def _tokenize_for_hash(self, text: str) -> List[str]:
        """
        本地 Hash Embedding 使用的简单分词方法。
        """
        text = text.lower()
        tokens: List[str] = []

        tokens.extend(re.findall(r"[a-zA-Z0-9_]+", text))

        chinese_spans = re.findall(r"[\u4e00-\u9fff]+", text)
        for span in chinese_spans:
            tokens.append(span)
            tokens.extend(list(span))

            for i in range(len(span) - 1):
                tokens.append(span[i:i + 2])

            for i in range(len(span) - 2):
                tokens.append(span[i:i + 3])

        return [token for token in tokens if token.strip()]

# ============================================================================
# 4. 内存向量索引
# ============================================================================

class VectorIndex:
    """
    内存向量索引。

    当前 Demo 使用 numpy 矩阵保存字段向量，并通过余弦相似度完成向量召回。
```

```
418         生产环境可替换为 FAISS、Milvus、PGVector 或 Elasticsearch dense_vector。
419         """
420
421         def __init__(self, embedding_client: AliyunEmbeddingClient):
422             self.embedding_client = embedding_client
423             self.documents: List[str] = []
424             self.embeddings: np.ndarray | None = None
425
426         def build(self, documents: List[str]) -> None:
427             """
428             构建向量索引。
429
430             Args:
431                 documents: 字段级向量索引文本列表。
432             """
433             self.documents = documents
434             self.embeddings = self.embedding_client.embed_texts(documents)
435
436         def search(self, query_text: str, top_k: int = 5) -> List[Tuple[int,
    float]]:
437             """
438             使用单个检索词进行向量召回。
439
440             Args:
441                 query_text: 单个核心检索词。
442                 top_k: 返回候选数量。
443
444             Returns:
445                 List[Tuple[int, float]]: 文档下标与相似度得分。
446             """
447             if self.embeddings is None:
448                 raise RuntimeError("VectorIndex 尚未构建，请先调用 build()。")
449
450             query_vector = self.embedding_client.embed_texts([query_text])[0]
451             scores = self.embeddings @ query_vector
452
453             top_indices = np.argsort(-scores)[:top_k]
454
455             return [
456                 (int(index), float(scores[index]))
457                 for index in top_indices
458             ]
459
460     # ============================================================================
461     # 5. Demo 数据库与业务元数据
462     # ============================================================================
463
```

```python
def create_demo_database() -> Path:
    """
    创建向量召回 Demo 使用的测试数据库。

    测试数据库包含两张表:
    - trade_summary: 用户交易汇总表
    - interest_info: 用户利率信息表
    """
    db_path = Path(tempfile.gettempdir()) / "text2sql_vector_recall_demo.db"

    if db_path.exists():
        db_path.unlink()

    conn = sqlite3.connect(str(db_path))
    conn.execute("PRAGMA foreign_keys = ON")

    conn.executescript(
        """
        CREATE TABLE trade_summary (
            user_id INTEGER PRIMARY KEY,
            total_trade_count INTEGER NOT NULL,
            total_trade_amount REAL NOT NULL,
            active_days INTEGER NOT NULL,
            last_trade_time TEXT
        );

        CREATE TABLE interest_info (
            user_id INTEGER PRIMARY KEY,
            interest_rate REAL NOT NULL,
            rate_type TEXT,
            effective_status TEXT,
            FOREIGN KEY (user_id) REFERENCES trade_summary(user_id)
        );
        """
    )

    conn.executemany(
        """
        INSERT INTO trade_summary (
            user_id,
            total_trade_count,
            total_trade_amount,
            active_days,
            last_trade_time
        )
        VALUES (?, ?, ?, ?, ?)
        """,
```

```
            [
                (1001, 12, 5300.50, 5, "2024-05-01"),
                (1002, 5800, 230000.00, 180, "2024-05-06"),
                (1003, 56000, 1800000.00, 320, "2024-05-10"),
                (1004, 102430, 3700000.00, 365, "2024-05-11"),
                (1005, 0, 0.00, 0, None),
            ],
        )

        conn.executemany(
            """
            INSERT INTO interest_info (
                user_id,
                interest_rate,
                rate_type,
                effective_status
            )
            VALUES (?, ?, ?, ?)
            """,
            [
                (1001, 2.35, "standard", "active"),
                (1002, 3.12, "vip", "active"),
                (1003, 4.58, "high_value", "active"),
                (1004, 4.95, "high_value", "active"),
                (1005, 1.80, "standard", "inactive"),
            ],
        )

        conn.commit()
        conn.close()

        return db_path

    def get_business_meta() -> dict:
        """
        返回 Demo 使用的表级和字段级业务元数据。

        向量召回主要依赖字段语义描述，因此该元数据重点维护字段描述、
        字段别名、业务用途和 vector_text。
        """
        return {
            "trade_summary": {
                "description": "该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金额、活跃交易天数以及最近交易时间。",
                "aliases": ["用户交易汇总表", "交易统计表"],
                "columns": {
                    "user_id": {
```

```
                    "description": "用户唯一标识，用于关联用户交易汇总信息与用户利率信息。",
                    "aliases": ["用户ID", "客户ID"],
                    "semantic_role": "join_key",
                    "business_usage": "用于表关联。",
                },
                "total_trade_count": {
                    "description": "用户累计交易总笔数，表示用户在统计周期内完成的交易次数。",
                    "aliases": ["总交易笔数", "累计交易笔数", "交易笔数", "交易次数"],
                    "semantic_role": "metric_filter",
                    "value_range": "整数，>= 0",
                    "data_distribution": "多数用户集中在0-50区间，高频交易用户可能超过50000。",
                    "business_usage": "用于衡量用户交易活跃程度，筛选高频交易用户。",
                    "samples": ["0", "12", "5800", "56000", "102430"],
                    "index_texts": {
                        "vector_text": (
                            "字段名：total_trade_count。所属表：trade_summary，"
                            "该表记录用户交易汇总信息。该字段表示用户在统计周期内的累计交易总笔数，"
                            "用于衡量用户交易活跃程度，可用于筛选高频交易用户。"
                        )
                    },
                },
                "total_trade_amount": {
                    "description": "用户累计交易金额，表示用户在统计周期内完成交易的总金额。",
                    "aliases": ["交易金额", "累计交易金额", "成交金额", "交易总额"],
                    "semantic_role": "metric",
                    "value_range": ">= 0",
                    "business_usage": "用于衡量用户交易价值、统计交易金额或识别高价值用户。",
                    "samples": ["0.00", "5300.50", "230000.00", "1800000.00"],
                    "index_texts": {
                        "vector_text": (
                            "字段名：total_trade_amount。所属表：trade_summary。"
                            "该字段表示用户在统计周期内完成交易的累计金额，"
                            "用于衡量用户交易价值或交易规模。"
                        )
                    },
                },
                "active_days": {
                    "description": "用户在统计周期内发生交易的活跃天数。",
                    "aliases": ["活跃天数", "交易活跃天数"],
                    "semantic_role": "metric",
```

```
                    "business_usage": "用于衡量用户交易频率和持续活跃程度。",
                },
                "last_trade_time": {
                    "description": "用户最近一次交易发生时间。",
                    "aliases": ["最近交易时间", "最后交易时间"],
                    "semantic_role": "time",
                    "business_usage": "用于判断用户最近活跃情况。",
                },
            },
        },
        "interest_info": {
            "description": "该表记录用户利率信息，包括用户利率类型、年化利率数值以及利率生效状态。",
            "aliases": ["用户利率表", "利率信息表"],
            "columns": {
                "user_id": {
                    "description": "用户唯一标识，用于关联交易汇总表。",
                    "aliases": ["用户ID", "客户ID"],
                    "semantic_role": "join_key",
                    "business_usage": "用于表关联。",
                },
                "interest_rate": {
                    "description": "用户对应的年化利率数值。",
                    "aliases": ["利率", "年化利率", "用户利率", "利率数值"],
                    "semantic_role": "output_metric",
                    "value_range": "0-100",
                    "business_usage": "用于展示或分析用户当前适用的年化利率。",
                    "samples": ["2.35", "3.12", "4.58", "4.95"],
                    "index_texts": {
                        "vector_text": (
                            "字段名：interest_rate。所属表：interest_info，"
                            "该表记录用户利率信息。该字段表示用户当前对应的年化利率数值，"
                            "通常作为查询结果输出。"
                        )
                    },
                },
                "rate_type": {
                    "description": "用户适用的利率类型，例如standard、vip、high_value。",
                    "aliases": ["利率类型", "费率类型"],
                    "semantic_role": "dimension",
                    "business_usage": "用于区分不同用户分层下的利率类型。",
                },
                "effective_status": {
                    "description": "利率生效状态，例如active、inactive。",
                    "aliases": ["生效状态", "利率状态"],
```

```
641                     "semantic_role": "filter",
642                     "business_usage": "用于筛选当前有效或无效的利率记录。",
643                 },
644             },
645         },
646     }
647
648 def enrich_columns_with_business_meta(columns: list, business_meta: dict) ->
    None:
649     """
650     将业务元数据补充回 ColumnSchema。
651
652     该函数用于兼容当前 Demo：即使 sqlite_loader.py 还没有支持 value_range、
653     business_usage、index_texts 等字段，也可以在 Demo 中把这些信息补充到字段对象上。
654     """
655     for column in columns:
656         table_meta = business_meta.get(column.table_name, {})
657         column_meta = table_meta.get("columns", {}).get(column.column_name, {})
658
659         if not column_meta:
660             continue
661
662         column.description = column_meta.get("description", getattr(column,
    "description", ""))
663         column.aliases = column_meta.get("aliases", getattr(column, "aliases",
    []))
664         column.semantic_role = column_meta.get("semantic_role",
    getattr(column, "semantic_role", ""))
665         column.value_range = column_meta.get("value_range", getattr(column,
    "value_range", ""))
666         column.data_distribution = column_meta.get("data_distribution",
    getattr(column, "data_distribution", ""))
667         column.business_usage = column_meta.get("business_usage",
    getattr(column, "business_usage", ""))
668
669         if "samples" in column_meta:
670             column.samples = [str(item) for item in column_meta["samples"]]
671
672         index_texts = column_meta.get("index_texts", {})
673         column.vector_text = index_texts.get("vector_text", "")
674
675 def build_vector_documents(columns: list) -> List[VectorFieldDocument]:
676     """
677     构建字段级向量检索文档。
678
679     Args:
680         columns: 字段 Schema 列表。
```

```
681
682         Returns:
683             List[VectorFieldDocument]: 字段级向量检索文档列表。
684         """
685         documents: List[VectorFieldDocument] = []
686
687         for column in columns:
688             doc_id = getattr(column, "full_name", None)
689             if not doc_id:
690                 doc_id = f"{column.database}.{column.table_name}.
    {column.column_name}"
691
692             vector_text = getattr(column, "vector_text", "")
693
694             if not vector_text:
695                 vector_text = "\n".join(
696                     part for part in [
697                         f"字段名: {column.column_name}",
698                         f"所属表: {column.table_name}",
699                         f"所属表业务含义: {getattr(column, 'table_description',
    '')}",
700                         f"字段含义: {getattr(column, 'description', '')}",
701                         f"字段别名: {', '.join(getattr(column, 'aliases', []))}" if
    getattr(column, "aliases", []) else "",
702                         f"业务用途: {getattr(column, 'business_usage', '')}" if
    getattr(column, "business_usage", "") else "",
703                         f"字段角色: {getattr(column, 'semantic_role', '')}" if
    getattr(column, "semantic_role", "") else "",
704                     ]
705                     if part
706                 )
707
708             documents.append(
709                 VectorFieldDocument(
710                     doc_id=doc_id,
711                     column=column,
712                     vector_text=vector_text,
713                 )
714             )
715
716         return documents
717
718     # ================================================================================
719     # 6. 多关键词向量召回
720     # ================================================================================
721
722     def search_by_extracted_keywords(
```

```python
    vector_index: VectorIndex,
    documents: List[VectorFieldDocument],
    keywords: List[str],
    per_keyword_top_k: int = 5,
    final_top_k: int = 5,
) -> List[Tuple[int, float, List[str]]]:
    """
    使用抽取后的核心关键词进行多路向量召回。

    每个关键词会单独进行向量检索。多个关键词的召回结果会按照字段文档去重，
    并保留每个字段的最高相似度得分。

    Args:
        vector_index: 已构建完成的向量索引。
        documents: 字段级检索文档列表。
        keywords: 核心检索词列表。
        per_keyword_top_k: 每个关键词单独召回的候选数量。
        final_top_k: 最终返回的字段数量。

    Returns:
        List[Tuple[int, float, List[str]]]: 文档下标、最终得分、命中的关键词列表。
    """
    merged: Dict[int, Dict[str, object]] = {}

    for keyword in keywords:
        keyword_hits = vector_index.search(keyword, top_k=per_keyword_top_k)

        for doc_index, score in keyword_hits:
            if doc_index not in merged:
                merged[doc_index] = {
                    "score": score,
                    "matched_keywords": [keyword],
                }
            else:
                merged[doc_index]["score"] = max(
                    float(merged[doc_index]["score"]),
                    score,
                )

                matched_keywords = merged[doc_index]["matched_keywords"]
                assert isinstance(matched_keywords, list)

                if keyword not in matched_keywords:
                    matched_keywords.append(keyword)

    results = [
        (
```

```
            doc_index,
            float(payload["score"]),
            payload["matched_keywords"],
        )
        for doc_index, payload in merged.items()
    ]

    results.sort(key=lambda item: item[1], reverse=True)

    return results[:final_top_k]

# ================================================================================
# 7. 主流程
# ================================================================================

def main() -> None:
    db_path = create_demo_database()
    business_meta = get_business_meta()

    print("=" * 80)
    print("向量召回 Demo")
    print("=" * 80)
    print(f"Demo database: {db_path}")

    loader = SQLiteSchemaLoader(
        db_path=db_path,
        database_name="demo_finance",
        business_meta=business_meta,
        sample_size=5,
    )

    tables, columns, relations = loader.load()
    enrich_columns_with_business_meta(columns, business_meta)

    documents = build_vector_documents(columns)

    print("\n[1] Vector documents:")
    for index, doc in enumerate(documents):
        print(f"{index}. {doc.doc_id}")
        print(f"   {doc.vector_text}")

    embedding_client = AliyunEmbeddingClient(
        AliyunEmbeddingConfig(
            api_key="",
            embedding_url="https://dashscope.aliyuncs.com/compatible-
mode/v1/embeddings",
            model="text-embedding-v4",
```

```
            dimensions=1024,
        )
    )

    vector_index = VectorIndex(embedding_client)

    vector_texts = [
        doc.vector_text
        for doc in documents
    ]

    print("\n[2] Building vector index...")
    vector_index.build(vector_texts)

    if embedding_client.config.api_key:
        print("Vector index built by Aliyun Embedding API.")
    else:
        print("Vector index built by local mock embedding because DASHSCOPE_API_KEY is not set.")

    query = "查询总交易笔数大于50000的年化利率是多少"

    print("\n[3] Query:")
    print(query)

    keyword_extractor = KeywordExtractor(
        KeywordExtractorConfig(
            api_key="",
            chat_url="https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions",
            model="qwen-plus",
        )
    )

    keywords = keyword_extractor.extract(query)

    print("\n[4] Extracted keywords:")
    print(", ".join(keywords))

    hits = search_by_extracted_keywords(
        vector_index=vector_index,
        documents=documents,
        keywords=keywords,
        per_keyword_top_k=5,
        final_top_k=5,
    )

```

```
    print("\n[5] Vector recall results by extracted keywords:")
    for rank, (doc_index, score, matched_keywords) in enumerate(hits, start=1):
        doc = documents[doc_index]
        col = doc.column

        print(
            f"{rank}. {col.table_name}.{col.column_name} "
            f"score={score:.6f} "
            f"matched_keywords={matched_keywords}"
        )
        print(f"   字段含义: {getattr(col, 'description', '')}")
        print(f"   向量文本: {doc.vector_text}")

if __name__ == "__main__":
    main()

```

测试指令与结果：

代码块

```
测试指令:
python -m schema_retrieval.vector_recall_demo

测试结果:
向量召回 Demo
================================================================================
Demo database: /tmp/text2sql_vector_recall_demo.db

[1] Vector documents:
0. demo_finance.interest_info.user_id
   字段名: user_id
所属表: interest_info
所属表业务含义: 该表记录用户利率信息，包括用户利率类型、年化利率数值以及利率生效状态。
字段含义: 用户唯一标识，用于关联交易汇总表。
字段别名: 用户ID, 客户ID
业务用途: 用于表关联。
字段角色: join_key
1. demo_finance.interest_info.interest_rate
   字段名: interest_rate。所属表: interest_info，该表记录用户利率信息。该字段表示用户当前对应的年化利率数值，通常作为查询结果输出。
2. demo_finance.interest_info.rate_type
   字段名: rate_type
所属表: interest_info
所属表业务含义: 该表记录用户利率信息，包括用户利率类型、年化利率数值以及利率生效状态。
```

字段含义：用户适用的利率类型，例如standard、vip、high_value。
字段别名：利率类型，费率类型
业务用途：用于区分不同用户分层下的利率类型。
字段角色：dimension
3. demo_finance.interest_info.effective_status
   字段名：effective_status
所属表：interest_info
所属表业务含义：该表记录用户利率信息，包括用户利率类型、年化利率数值以及利率生效状态。
字段含义：利率生效状态，例如active、inactive。
字段别名：生效状态，利率状态
业务用途：用于筛选当前有效或无效的利率记录。
字段角色：filter
4. demo_finance.trade_summary.user_id
   字段名：user_id
所属表：trade_summary
所属表业务含义：该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金额、活跃交易天数以及最近交易时间。
字段含义：用户唯一标识，用于关联用户交易汇总信息与用户利率信息。
字段别名：用户ID，客户ID
业务用途：用于表关联。
字段角色：join_key
5. demo_finance.trade_summary.total_trade_count
   字段名：total_trade_count。所属表：trade_summary，该表记录用户交易汇总信息。该字段表示用户在统计周期内的累计交易总笔数，用于衡量用户交易活跃程度，可用于筛选高频交易用户。
6. demo_finance.trade_summary.total_trade_amount
   字段名：total_trade_amount。所属表：trade_summary。该字段表示用户在统计周期内完成交易的累计金额，用于衡量用户交易价值或交易规模。
7. demo_finance.trade_summary.active_days
   字段名：active_days
所属表：trade_summary
所属表业务含义：该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金额、活跃交易天数以及最近交易时间。
字段含义：用户在统计周期内发生交易的活跃天数。
字段别名：活跃天数，交易活跃天数
业务用途：用于衡量用户交易频率和持续活跃程度。
字段角色：metric
8. demo_finance.trade_summary.last_trade_time
   字段名：last_trade_time
所属表：trade_summary
所属表业务含义：该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金额、活跃交易天数以及最近交易时间。
字段含义：用户最近一次交易发生时间。
字段别名：最近交易时间，最后交易时间
业务用途：用于判断用户最近活跃情况。
字段角色：time

[2] Building vector index...

```
Vector index built by local mock embedding because DASHSCOPE_API_KEY is not set.

[3] Query:
查询总交易笔数大于50000的年化利率是多少

[4] Extracted keywords:
总交易笔数，交易笔数，年化利率，利率

[5] Vector recall results by extracted keywords:
1. interest_info.effective_status score=0.448359 matched_keywords=['年化利率', '利率']
   字段含义：利率生效状态，例如active、inactive。
   向量文本：字段名：effective_status
所属表：interest_info
所属表业务含义：该表记录用户利率信息，包括用户利率类型、年化利率数值以及利率生效状态。
字段含义：利率生效状态，例如active、inactive。
字段别名：生效状态，利率状态
业务用途：用于筛选当前有效或无效的利率记录。
字段角色：filter
2. interest_info.rate_type score=0.441221 matched_keywords=['年化利率', '利率']
   字段含义：用户适用的利率类型，例如standard、vip、high_value。
   向量文本：字段名：rate_type
所属表：interest_info
所属表业务含义：该表记录用户利率信息，包括用户利率类型、年化利率数值以及利率生效状态。
字段含义：用户适用的利率类型，例如standard、vip、high_value。
字段别名：利率类型，费率类型
业务用途：用于区分不同用户分层下的利率类型。
字段角色：dimension
3. trade_summary.active_days score=0.414121 matched_keywords=['总交易笔数', '交易笔数']
   字段含义：用户在统计周期内发生交易的活跃天数。
   向量文本：字段名：active_days
所属表：trade_summary
所属表业务含义：该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金额、活跃交易天数以及最近交易时间。
字段含义：用户在统计周期内发生交易的活跃天数。
字段别名：活跃天数，交易活跃天数
业务用途：用于衡量用户交易频率和持续活跃程度。
字段角色：metric
4. trade_summary.last_trade_time score=0.372223 matched_keywords=['总交易笔数', '交易笔数']
   字段含义：用户最近一次交易发生时间。
   向量文本：字段名：last_trade_time
所属表：trade_summary
所属表业务含义：该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金额、活跃交易天数以及最近交易时间。
```

```
字段含义：用户最近一次交易发生时间。
字段别名：最近交易时间，最后交易时间
业务用途：用于判断用户最近活跃情况。
字段角色：time
5. interest_info.interest_rate score=0.331832 matched_keywords=['年化利率', '利率']
   字段含义：用户对应的年化利率数值。
   向量文本：字段名：interest_rate。所属表：interest_info，该表记录用户利率信息。该字段表示用户当前对应的年化利率数值，通常作为查询结果输出。
```

混合检索+RFF融合

hybrid_recall_demo.py

```python
from __future__ import annotations

import sqlite3
import tempfile
from dataclasses import dataclass
from pathlib import Path
from typing import List

from schema_retrieval.bm25 import BM25Index
from schema_retrieval.sqlite_loader import SQLiteSchemaLoader

from schema_retrieval.vector_recall_demo import (
    AliyunEmbeddingClient,
    AliyunEmbeddingConfig,
    KeywordExtractor,
    KeywordExtractorConfig,
    VectorIndex,
)

from schema_retrieval.rrf_fusion_client import (
    RRFFusionClient,
    RRFFusionConfig,
    RouteRecallResult,
)

# ============================================================================
# 1. 混合召回文档对象
# ============================================================================

@dataclass
class HybridFieldDocument:
    """
    混合召回使用的字段级文档。
```

```
34
35      一个字段同时维护：
36      - keyword_text：给 BM25 关键词召回使用
37      - vector_text：给 Embedding 向量召回使用
38      """
39
40      doc_id: str
41      column: object
42      keyword_text: str
43      vector_text: str
44
45  # ============================================================================
46  # 2. Demo 数据库
47  # ============================================================================
48
49  def create_demo_database() -> Path:
50      """
51      创建混合召回 Demo 使用的测试数据库。
52
53      测试数据库包含两张表：
54      - trade_summary: 用户交易汇总表
55      - interest_info: 用户利率信息表
56      """
57      db_path = Path(tempfile.gettempdir()) / "text2sql_hybrid_recall_demo.db"
58
59      if db_path.exists():
60          db_path.unlink()
61
62      conn = sqlite3.connect(str(db_path))
63      conn.execute("PRAGMA foreign_keys = ON")
64
65      conn.executescript(
66          """
67          CREATE TABLE trade_summary (
68              user_id INTEGER PRIMARY KEY,
69              total_trade_count INTEGER NOT NULL,
70              total_trade_amount REAL NOT NULL,
71              active_days INTEGER NOT NULL,
72              last_trade_time TEXT
73          );
74
75          CREATE TABLE interest_info (
76              user_id INTEGER PRIMARY KEY,
77              interest_rate REAL NOT NULL,
78              rate_type TEXT,
79              effective_status TEXT,
80              FOREIGN KEY (user_id) REFERENCES trade_summary(user_id)
```

```
        );
        """
    )

    conn.executemany(
        """
        INSERT INTO trade_summary (
            user_id,
            total_trade_count,
            total_trade_amount,
            active_days,
            last_trade_time
        )
        VALUES (?, ?, ?, ?, ?)
        """,
        [
            (1001, 12, 5300.50, 5, "2024-05-01"),
            (1002, 5800, 230000.00, 180, "2024-05-06"),
            (1003, 56000, 1800000.00, 320, "2024-05-10"),
            (1004, 102430, 3700000.00, 365, "2024-05-11"),
            (1005, 0, 0.00, 0, None),
        ],
    )

    conn.executemany(
        """
        INSERT INTO interest_info (
            user_id,
            interest_rate,
            rate_type,
            effective_status
        )
        VALUES (?, ?, ?, ?)
        """,
        [
            (1001, 2.35, "standard", "active"),
            (1002, 3.12, "vip", "active"),
            (1003, 4.58, "high_value", "active"),
            (1004, 4.95, "high_value", "active"),
            (1005, 1.80, "standard", "inactive"),
        ],
    )

    conn.commit()
    conn.close()

    return db_path
```

```python

# ============================================================================
# 3. 业务元数据
# ============================================================================

def get_business_meta() -> dict:
    """
    返回 Demo 使用的表级和字段级业务元数据。

    keyword_text 更偏向词面命中。
    vector_text 更偏向业务语义表达。
    """
    return {
        "trade_summary": {
            "description": "该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金额、活跃交易天数以及最近交易时间。",
            "aliases": ["用户交易汇总表", "交易统计表"],
            "columns": {
                "user_id": {
                    "description": "用户唯一标识，用于关联用户交易汇总信息与用户利率信息。",
                    "aliases": ["用户ID", "客户ID"],
                    "semantic_role": "join_key",
                    "business_usage": "用于表关联。",
                    "keyword_text": "user_id 用户ID 客户ID 用户唯一标识 关联字段",
                    "vector_text": (
                        "字段名：user_id。所属表：trade_summary。"
                        "该字段表示用户唯一标识，用于关联用户交易汇总信息与用户利率信息。"
                    ),
                },
                "total_trade_count": {
                    "description": "用户累计交易总笔数，表示用户在统计周期内完成的交易次数。",
                    "aliases": ["总交易笔数", "累计交易笔数", "交易笔数", "交易次数"],
                    "semantic_role": "metric_filter",
                    "value_range": "整数，>= 0",
                    "data_distribution": "多数用户集中在0-50区间，高频交易用户可能超过50000。",
                    "business_usage": "用于衡量用户交易活跃程度，筛选高频交易用户。",
                    "samples": ["0", "12", "5800", "56000", "102430"],
                    "keyword_text": (
                        "total_trade_count 总交易笔数 累计交易笔数 交易笔数 "
                        "交易次数 高频交易 大于50000 用户交易汇总表 交易统计表"
                    ),
                    "vector_text": (
```

```
                        "字段名：total_trade_count。所属表：trade_summary，"
                        "该表记录用户交易汇总信息。该字段表示用户在统计周期内的累计交易总笔数，"
                        "用于衡量用户交易活跃程度，可用于筛选高频交易用户。"
                    ),
                },
                "total_trade_amount": {
                    "description": "用户累计交易金额，表示用户在统计周期内完成交易的总金额。",
                    "aliases": ["交易金额", "累计交易金额", "成交金额", "交易总额"],
                    "semantic_role": "metric",
                    "value_range": ">= 0",
                    "business_usage": "用于衡量用户交易价值、统计交易金额或识别高价值用户。",
                    "samples": ["0.00", "5300.50", "230000.00", "1800000.00"],
                    "keyword_text": (
                        "total_trade_amount 交易金额 累计交易金额 成交金额 交易总额 "
                        "交易规模 高价值用户"
                    ),
                    "vector_text": (
                        "字段名：total_trade_amount。所属表：trade_summary。"
                        "该字段表示用户在统计周期内完成交易的累计金额，"
                        "用于衡量用户交易价值或交易规模。"
                    ),
                },
                "active_days": {
                    "description": "用户在统计周期内发生交易的活跃天数。",
                    "aliases": ["活跃天数", "交易活跃天数"],
                    "semantic_role": "metric",
                    "business_usage": "用于衡量用户交易频率和持续活跃程度。",
                    "keyword_text": "active_days 活跃天数 交易活跃天数 交易频率 持续活跃",
                    "vector_text": (
                        "字段名：active_days。所属表：trade_summary。"
                        "该字段表示用户在统计周期内发生交易的活跃天数，用于衡量交易频率。"
                    ),
                },
                "last_trade_time": {
                    "description": "用户最近一次交易发生时间。",
                    "aliases": ["最近交易时间", "最后交易时间"],
                    "semantic_role": "time",
                    "business_usage": "用于判断用户最近活跃情况。",
                    "keyword_text": "last_trade_time 最近交易时间 最后交易时间 交易时间 最近活跃",
                    "vector_text": (
```

```
                        "字段名：last_trade_time。所属表：trade_summary。"
                        "该字段表示用户最近一次交易发生时间，用于判断用户最近活跃情况。"
                    ),
                },
            },
        },
        "interest_info": {
            "description": "该表记录用户利率信息，包括用户利率类型、年化利率数值以及利率生效状态。",
            "aliases": ["用户利率表", "利率信息表"],
            "columns": {
                "user_id": {
                    "description": "用户唯一标识，用于关联交易汇总表。",
                    "aliases": ["用户ID", "客户ID"],
                    "semantic_role": "join_key",
                    "business_usage": "用于表关联。",
                    "keyword_text": "user_id 用户ID 客户ID 用户唯一标识 关联字段",
                    "vector_text": (
                        "字段名：user_id。所属表：interest_info。"
                        "该字段表示用户唯一标识，用于关联交易汇总表。"
                    ),
                },
                "interest_rate": {
                    "description": "用户对应的年化利率数值。",
                    "aliases": ["利率", "年化利率", "用户利率", "利率数值"],
                    "semantic_role": "output_metric",
                    "value_range": "0-100",
                    "business_usage": "用于展示或分析用户当前适用的年化利率。",
                    "samples": ["2.35", "3.12", "4.58", "4.95"],
                    "keyword_text": (
                        "interest_rate 利率 年化利率 用户利率 利率数值 "
                        "利率是多少 用户利率表 利率信息表"
                    ),
                    "vector_text": (
                        "字段名：interest_rate。所属表：interest_info，"
                        "该表记录用户利率信息。该字段表示用户当前对应的年化利率数值，"
                        "通常作为查询结果输出。"
                    ),
                },
                "rate_type": {
                    "description": "用户适用的利率类型，例如standard、vip、high_value。",
                    "aliases": ["利率类型", "费率类型"],
                    "semantic_role": "dimension",
                    "business_usage": "用于区分不同用户分层下的利率类型。",
```

```
                    "keyword_text": "rate_type 利率类型 费率类型 standard vip high_value",
                    "vector_text": (
                        "字段名：rate_type。所属表：interest_info。"
                        "该字段表示用户适用的利率类型，用于区分不同用户分层下的利率。"
                    ),
                },
                "effective_status": {
                    "description": "利率生效状态，例如active、inactive。",
                    "aliases": ["生效状态", "利率状态"],
                    "semantic_role": "filter",
                    "business_usage": "用于筛选当前有效或无效的利率记录。",
                    "keyword_text": "effective_status 生效状态 利率状态 active inactive 有效 无效",
                    "vector_text": (
                        "字段名：effective_status。所属表：interest_info。"
                        "该字段表示利率生效状态，用于筛选当前有效或无效的利率记录。"
                    ),
                },
            },
        },
    }

def enrich_columns_with_business_meta(columns: list, business_meta: dict) -> None:
    """
    将业务元数据补充回 ColumnSchema。

    这样即使 sqlite_loader.py 暂时没有完整解析所有业务字段，
    Demo 也可以使用 description、aliases、semantic_role 等信息。
    """
    for column in columns:
        table_meta = business_meta.get(column.table_name, {})
        column_meta = table_meta.get("columns", {}).get(column.column_name, {})

        if not column_meta:
            continue

        column.description = column_meta.get("description", getattr(column, "description", ""))
        column.aliases = column_meta.get("aliases", getattr(column, "aliases", []))
        column.semantic_role = column_meta.get("semantic_role", getattr(column, "semantic_role", ""))
        column.value_range = column_meta.get("value_range", getattr(column, "value_range", ""))
        column.data_distribution = column_meta.get(
```

```
292             "data_distribution",
293             getattr(column, "data_distribution", ""),
294         )
295         column.business_usage = column_meta.get(
296             "business_usage",
297             getattr(column, "business_usage", ""),
298         )
299 
300         if "samples" in column_meta:
301             column.samples = [str(item) for item in column_meta["samples"]]
302 
303 # ================================================================================
304 # 4. 构建混合召回文档
305 # ================================================================================
306 
307 def build_hybrid_documents(columns: list, business_meta: dict) ->
    List[HybridFieldDocument]:
308     """
309     构建混合召回字段文档。
310 
311     Args:
312         columns: ColumnSchema 列表。
313         business_meta: 业务元数据。
314 
315     Returns:
316         List[HybridFieldDocument]: 混合召回字段文档列表。
317     """
318     documents: List[HybridFieldDocument] = []
319 
320     for column in columns:
321         table_meta = business_meta.get(column.table_name, {})
322         column_meta = table_meta.get("columns", {}).get(column.column_name, {})
323 
324         keyword_text = column_meta.get("keyword_text") or
    build_default_keyword_text(column)
325         vector_text = column_meta.get("vector_text") or
    build_default_vector_text(column)
326 
327         documents.append(
328             HybridFieldDocument(
329                 doc_id=column.full_name,
330                 column=column,
331                 keyword_text=keyword_text,
332                 vector_text=vector_text,
333             )
334         )
335 
```

```
336         return documents
337
338     def build_default_keyword_text(column: object) -> str:
339         """
340         构建默认关键词索引文本。
341
342         关键词索引文本尽量短，避免把整张表的长描述注入到每个字段里。
343         """
344         parts = [
345             getattr(column, "database", ""),
346             getattr(column, "table_name", ""),
347             getattr(column, "column_name", ""),
348             getattr(column, "data_type", ""),
349             getattr(column, "description", ""),
350             " ".join(getattr(column, "aliases", []) or []),
351             " ".join(getattr(column, "samples", [])[:5] or []),
352         ]
353
354         return "\n".join(part for part in parts if part)
355
356     def build_default_vector_text(column: object) -> str:
357         """
358         构建默认向量索引文本。
359         """
360         parts = [
361             f"字段名：{getattr(column, 'column_name', '')}",
362             f"所属表：{getattr(column, 'table_name', '')}",
363             f"字段含义：{getattr(column, 'description', '')}",
364         ]
365
366         aliases = getattr(column, "aliases", []) or []
367         if aliases:
368             parts.append(f"字段别名：{', '.join(aliases)}")
369
370         business_usage = getattr(column, "business_usage", "")
371         if business_usage:
372             parts.append(f"业务用途：{business_usage}")
373
374         semantic_role = getattr(column, "semantic_role", "")
375         if semantic_role:
376             parts.append(f"字段角色：{semantic_role}")
377
378         return "\n".join(part for part in parts if part)
379
380     # ================================================================================
381     # 5. 混合召回
382     # ================================================================================
```

```python

def build_route_results(
    keywords: List[str],
    keyword_index: BM25Index,
    vector_index: VectorIndex,
    per_keyword_top_k: int,
) -> List[RouteRecallResult]:
    """
    分别执行关键词召回和向量召回，并组装成 RRF Client 所需的输入格式。

    Args:
        keywords: 大模型抽取出的关键词列表。
        keyword_index: BM25 关键词索引。
        vector_index: 向量索引。
        per_keyword_top_k: 每个关键词在每一路召回中的 Top-K。

    Returns:
        List[RouteRecallResult]: 多路召回结果。
    """
    route_results: List[RouteRecallResult] = []

    for keyword in keywords:
        keyword_hits = keyword_index.search(
            keyword,
            top_k=per_keyword_top_k,
        )

        route_results.append(
            RouteRecallResult(
                route_name="keyword",
                query_term=keyword,
                ranked_doc_indices=[
                    doc_index
                    for doc_index, _ in keyword_hits
                ],
            )
        )

        vector_hits = vector_index.search(
            keyword,
            top_k=per_keyword_top_k,
        )

        route_results.append(
            RouteRecallResult(
                route_name="vector",
                query_term=keyword,
```

```
                ranked_doc_indices=[
                    doc_index
                    for doc_index, _ in vector_hits
                ],
            )
        )

    return route_results

# ============================================================================
# 6. 主流程
# ============================================================================

def main() -> None:
    db_path = create_demo_database()
    business_meta = get_business_meta()

    print("=" * 80)
    print("关键词 + 向量 + RRF 融合召回 Demo")
    print("=" * 80)
    print(f"Demo database: {db_path}")

    loader = SQLiteSchemaLoader(
        db_path=db_path,
        database_name="demo_finance",
        business_meta=business_meta,
        sample_size=5,
    )

    tables, columns, relations = loader.load()
    enrich_columns_with_business_meta(columns, business_meta)

    documents = build_hybrid_documents(
        columns=columns,
        business_meta=business_meta,
    )

    print("\n[1] Hybrid documents:")
    for index, doc in enumerate(documents):
        print(f"{index}. {doc.doc_id}")
        print(f"   keyword_text: {doc.keyword_text}")
        print(f"   vector_text : {doc.vector_text}")

    # ------------------------------------------------------------------------
    # 构建关键词 BM25 索引
    # ------------------------------------------------------------------------
    keyword_texts = [
```

```
            doc.keyword_text
            for doc in documents
        ]

        keyword_index = BM25Index(keyword_texts)

        # ------------------------------------------------------------------------
        # 构建向量索引
        # ------------------------------------------------------------------------
        embedding_client = AliyunEmbeddingClient(
            AliyunEmbeddingConfig(
                api_key="",
                embedding_url="https://dashscope.aliyuncs.com/compatible-mode/v1/embeddings",
                model="text-embedding-v4",
                dimensions=1024,
            )
        )

        vector_index = VectorIndex(embedding_client)

        vector_texts = [
            doc.vector_text
            for doc in documents
        ]

        print("\n[2] Building vector index...")
        vector_index.build(vector_texts)

        if embedding_client.config.api_key:
            print("Vector index built by Aliyun Embedding API.")
        else:
            print("Vector index built by local mock embedding because DASHSCOPE_API_KEY is not set.")

        # ------------------------------------------------------------------------
        # 大模型抽取关键词
        # ------------------------------------------------------------------------
        query = "查询总交易笔数大于50000的利率是多少"

        print("\n[3] Query:")
        print(query)

        keyword_extractor = KeywordExtractor(
            KeywordExtractorConfig(
                api_key="",
```

```
            chat_url="https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions",
            model="qwen-plus",
        )
    )

    keywords = keyword_extractor.extract(query)

    print("\n[4] Extracted keywords:")
    print(", ".join(keywords))

    # ------------------------------------------------------------------------
    # 可调参数位置
    # ------------------------------------------------------------------------
    per_keyword_top_k = 20
    """
    每个关键词在每一路召回中的候选数量。

    例如关键词是：
        总交易笔数，利率

    那么：
        总交易笔数 -> BM25 Top 20 + Vector Top 20
        利率       -> BM25 Top 20 + Vector Top 20
    """

    rrf_client = RRFFusionClient(
        RRFFusionConfig(
            rrf_k=60,

            # 这里就是你说的“关键词提取数量的 5~6 倍”的位置。
            # fused_top_k = len(keywords) * truncate_multiplier
            truncate_multiplier=6,

            min_fused_top_k=10,
            max_fused_top_k=50,

            # 最终展示多少个字段。
            final_top_k=8,

            # 两路召回权重。
            route_weights={
                "keyword": 1.0,
                "vector": 1.0,
            },
        )
    )
```

```
567
568         fused_top_k = rrf_client.calculate_fused_top_k(
569             keyword_count=len(keywords),
570         )
571
572         print("\n[5] Hybrid recall config:")
573         print(f"per_keyword_top_k = {per_keyword_top_k}")
574         print(f"keyword_count = {len(keywords)}")
575         print(f"truncate_multiplier = {rrf_client.config.truncate_multiplier}")
576         print(f"fused_top_k = {fused_top_k}")
577         print(f"final_top_k = {rrf_client.config.final_top_k}")
578         print(f"rrf_k = {rrf_client.config.rrf_k}")
579         print(f"route_weights = {rrf_client.config.route_weights}")
580
581         # ------------------------------------------------------------------------
582         # 执行两路召回
583         # ------------------------------------------------------------------------
584         route_results = build_route_results(
585             keywords=keywords,
586             keyword_index=keyword_index,
587             vector_index=vector_index,
588             per_keyword_top_k=per_keyword_top_k,
589         )
590
591         print("\n[6] Route recall results:")
592         for route_result in route_results:
593             print(
594                 f"route={route_result.route_name}, "
595                 f"term={route_result.query_term}, "
596                 f"ranked_doc_indices={route_result.ranked_doc_indices}"
597             )
598
599         # ------------------------------------------------------------------------
600         # RRF 融合
601         # ------------------------------------------------------------------------
602         fusion_hits = rrf_client.fuse(
603             route_results=route_results,
604             keyword_count=len(keywords),
605         )
606
607         print("\n[7] RRF fused recall results:")
608         for rank, hit in enumerate(fusion_hits, start=1):
609             doc = documents[hit.doc_index]
610             col = doc.column
611
612             print(
613                 f"{rank}. {col.table_name}.{col.column_name} "
```

```
            f"score={hit.score:.6f} "
            f"sources={hit.sources} "
            f"matched_terms={hit.matched_terms} "
            f"best_rank_by_source={hit.best_rank_by_source}"
        )
        print(f"   字段含义: {col.description}")
        print(f"   keyword_text: {doc.keyword_text}")
        print(f"   vector_text : {doc.vector_text}")

if __name__ == "__main__":
    main()
```

测试指令与结果：

代码块

```
测试指令：
python -m schema_retrieval.hybrid_recall_demo

测试结果：
================================================================================
关键词 + 向量 + RRF 融合召回 Demo
================================================================================
Demo database: /tmp/text2sql_hybrid_recall_demo.db

[1] Hybrid documents:
0. demo_finance.interest_info.user_id
   keyword_text: user_id 用户ID 客户ID 用户唯一标识 关联字段
   vector_text : 字段名：user_id。所属表：interest_info。该字段表示用户唯一标识，用于关联交易汇总表。
1. demo_finance.interest_info.interest_rate
   keyword_text: interest_rate 利率 年化利率 用户利率 利率数值 利率是多少 用户利率表 利率信息表
   vector_text : 字段名：interest_rate。所属表：interest_info，该表记录用户利率信息。该字段表示用户当前对应的年化利率数值，通常作为查询结果输出。
2. demo_finance.interest_info.rate_type
   keyword_text: rate_type 利率类型 费率类型 standard vip high_value
   vector_text : 字段名：rate_type。所属表：interest_info。该字段表示用户适用的利率类型，用于区分不同用户分层下的利率。
3. demo_finance.interest_info.effective_status
   keyword_text: effective_status 生效状态 利率状态 active inactive 有效 无效
   vector_text : 字段名：effective_status。所属表：interest_info。该字段表示利率生效状态，用于筛选当前有效或无效的利率记录。
4. demo_finance.trade_summary.user_id
```

```
   keyword_text: user_id 用户ID 客户ID 用户唯一标识 关联字段
   vector_text : 字段名：user_id。所属表：trade_summary。该字段表示用户唯一标识，用于关联用户交易汇总信息与用户利率信息。
5. demo_finance.trade_summary.total_trade_count
   keyword_text: total_trade_count 总交易笔数 累计交易笔数 交易笔数 交易次数 高频交易 大于50000 用户交易汇总表 交易统计表
   vector_text : 字段名：total_trade_count。所属表：trade_summary，该表记录用户交易汇总信息。该字段表示用户在统计周期内的累计交易总笔数，用于衡量用户交易活跃程度，可用于筛选高频交易用户。
6. demo_finance.trade_summary.total_trade_amount
   keyword_text: total_trade_amount 交易金额 累计交易金额 成交金额 交易总额 交易规模 高价值用户
   vector_text : 字段名：total_trade_amount。所属表：trade_summary。该字段表示用户在统计周期内完成交易的累计金额，用于衡量用户交易价值或交易规模。
7. demo_finance.trade_summary.active_days
   keyword_text: active_days 活跃天数 交易活跃天数 交易频率 持续活跃
   vector_text : 字段名：active_days。所属表：trade_summary。该字段表示用户在统计周期内发生交易的活跃天数，用于衡量交易频率。
8. demo_finance.trade_summary.last_trade_time
   keyword_text: last_trade_time 最近交易时间 最后交易时间 交易时间 最近活跃
   vector_text : 字段名：last_trade_time。所属表：trade_summary。该字段表示用户最近一次交易发生时间，用于判断用户最近活跃情况。

[2] Building vector index...
Vector index built by local mock embedding because DASHSCOPE_API_KEY is not set.

[3] Query:
查询总交易笔数大于50000的利率是多少

[4] Extracted keywords:
总交易笔数，交易笔数，利率

[5] Hybrid recall config:
per_keyword_top_k = 20
keyword_count = 3
truncate_multiplier = 6
fused_top_k = 18
final_top_k = 8
rrf_k = 60
route_weights = {'keyword': 1.0, 'vector': 1.0}

[6] Route recall results:
route=keyword, term=总交易笔数, ranked_doc_indices=[5, 6, 7, 8, 1]
route=vector, term=总交易笔数, ranked_doc_indices=[5, 6, 7, 0, 4, 8, 1, 3, 2]
route=keyword, term=交易笔数, ranked_doc_indices=[5, 7, 6, 8, 1]
route=vector, term=交易笔数, ranked_doc_indices=[5, 7, 6, 8, 0, 4, 1, 3, 2]
```

```
route=keyword, term=利率, ranked_doc_indices=[1, 2, 3, 7]
route=vector, term=利率, ranked_doc_indices=[1, 3, 2, 4, 7, 0, 5, 6, 8]

[7] RRF fused recall results:
1. trade_summary.active_days score=0.095014 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 2, 'vector': 2}
   字段含义：用户在统计周期内发生交易的活跃天数。
   keyword_text: active_days 活跃天数 交易活跃天数 交易频率 持续活跃
   vector_text : 字段名：active_days。所属表：trade_summary。该字段表示用户在统计周期内发生交易的活跃天数，用于衡量交易频率。
2. interest_info.interest_rate score=0.093407 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 1, 'vector': 1}
   字段含义：用户对应的年化利率数值。
   keyword_text: interest_rate 利率 年化利率 用户利率 利率数值 利率是多少 用户利率表 利率信息表
   vector_text : 字段名：interest_rate。所属表：interest_info，该表记录用户利率信息。该字段表示用户当前对应的年化利率数值，通常作为查询结果输出。
3. trade_summary.total_trade_count score=0.080499 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 1, 'vector': 1}
   字段含义：用户累计交易总笔数，表示用户在统计周期内完成的交易次数。
   keyword_text: total_trade_count 总交易笔数 累计交易笔数 交易笔数 交易次数 高频交易 大于50000 用户交易汇总表 交易统计表
   vector_text : 字段名：total_trade_count。所属表：trade_summary，该表记录用户交易汇总信息。该字段表示用户在统计周期内的累计交易总笔数，用于衡量用户交易活跃程度，可用于筛选高频交易用户。
4. trade_summary.total_trade_amount score=0.078710 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 2, 'vector': 2}
   字段含义：用户累计交易金额，表示用户在统计周期内完成交易的总金额。
   keyword_text: total_trade_amount 交易金额 累计交易金额 成交金额 交易总额 交易规模 高价值用户
   vector_text : 字段名：total_trade_amount。所属表：trade_summary。该字段表示用户在统计周期内完成交易的累计金额，用于衡量用户交易价值或交易规模。
5. trade_summary.last_trade_time score=0.076519 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 4, 'vector': 4}
   字段含义：用户最近一次交易发生时间。
   keyword_text: last_trade_time 最近交易时间 最后交易时间 交易时间 最近活跃
   vector_text : 字段名：last_trade_time。所属表：trade_summary。该字段表示用户最近一次交易发生时间，用于判断用户最近活跃情况。
6. interest_info.effective_status score=0.061414 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'vector': 2, 'keyword': 3}
   字段含义：利率生效状态，例如active、inactive。
```

```
   keyword_text: effective_status 生效状态 利率状态 active inactive 有效 无效
   vector_text : 字段名：effective_status。所属表：interest_info。该字段表示利率生效状态，用于筛选当前有效或无效的利率记录。
7. interest_info.rate_type score=0.060988 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'vector': 3, 'keyword': 2}
   字段含义：用户适用的利率类型，例如standard、vip、high_value。
   keyword_text: rate_type 利率类型 费率类型 standard vip high_value
   vector_text : 字段名：rate_type。所属表：interest_info。该字段表示用户适用的利率类型，用于区分不同用户分层下的利率。
8. interest_info.user_id score=0.046161 sources=['vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'vector': 4}
   字段含义：用户唯一标识，用于关联交易汇总表。
   keyword_text: user_id 用户ID 客户ID 用户唯一标识 关联字段
   vector_text : 字段名：user_id。所属表：interest_info。该字段表示用户唯一标识，用于关联交易汇总表。
```

混合检索+重排序

hybrid_rerank_schema_demo.py

```python
from __future__ import annotations

import sqlite3
import tempfile
from dataclasses import dataclass
from pathlib import Path
from typing import List

from schema_retrieval.bm25 import BM25Index
from schema_retrieval.graph_builder import build_schema_graph
from schema_retrieval.objects import SchemaHit
from schema_retrieval.rrf_fusion_client import (
    RRFFusionClient,
    RRFFusionConfig,
    RouteRecallResult,
)
from schema_retrieval.rerank_client import (
    AliyunRerankClient,
    AliyunRerankConfig,
    RerankDocument,
)
from schema_retrieval.sqlite_loader import SQLiteSchemaLoader
from schema_retrieval.vector_recall_demo import (
    AliyunEmbeddingClient,
    AliyunEmbeddingConfig,
    KeywordExtractor,
```

```
    KeywordExtractorConfig,
    VectorIndex,
)

@dataclass
class HybridFieldDocument:
    """
    混合检索字段文档。

    keyword_text 用于 BM25。
    vector_text 用于 Embedding。
    rerank_text 用于 Rerank。
    """

    doc_id: str
    column: object
    keyword_text: str
    vector_text: str
    rerank_text: str

def create_demo_database() -> Path:
    """
    创建 Demo 数据库。
    """
    db_path = Path(tempfile.gettempdir()) /
"text2sql_hybrid_rerank_schema_demo.db"

    if db_path.exists():
        db_path.unlink()

    conn = sqlite3.connect(str(db_path))
    conn.execute("PRAGMA foreign_keys = ON")

    conn.executescript(
        """
        CREATE TABLE trade_summary (
            user_id INTEGER PRIMARY KEY,
            total_trade_count INTEGER NOT NULL,
            total_trade_amount REAL NOT NULL,
            active_days INTEGER NOT NULL,
            last_trade_time TEXT
        );

        CREATE TABLE interest_info (
            user_id INTEGER PRIMARY KEY,
            interest_rate REAL NOT NULL,
            rate_type TEXT,
```

```
            effective_status TEXT,
            FOREIGN KEY (user_id) REFERENCES trade_summary(user_id)
        );
        """
    )

    conn.executemany(
        """
        INSERT INTO trade_summary (
            user_id,
            total_trade_count,
            total_trade_amount,
            active_days,
            last_trade_time
        )
        VALUES (?, ?, ?, ?, ?)
        """,
        [
            (1001, 12, 5300.50, 5, "2024-05-01"),
            (1002, 5800, 230000.00, 180, "2024-05-06"),
            (1003, 56000, 1800000.00, 320, "2024-05-10"),
            (1004, 102430, 3700000.00, 365, "2024-05-11"),
            (1005, 0, 0.00, 0, None),
        ],
    )

    conn.executemany(
        """
        INSERT INTO interest_info (
            user_id,
            interest_rate,
            rate_type,
            effective_status
        )
        VALUES (?, ?, ?, ?)
        """,
        [
            (1001, 2.35, "standard", "active"),
            (1002, 3.12, "vip", "active"),
            (1003, 4.58, "high_value", "active"),
            (1004, 4.95, "high_value", "active"),
            (1005, 1.80, "standard", "inactive"),
        ],
    )

    conn.commit()
    conn.close()
```

```
120
121     return db_path
122
123 def get_business_meta() -> dict:
124     """
125     Demo 业务元数据。
126
127     三类文本：
128     - keyword_text: 关键词召回
129     - vector_text: 向量召回
130     - rerank_text: Rerank 精排
131     """
132     return {
133         "trade_summary": {
134             "description": "该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金
    额、活跃交易天数以及最近交易时间。",
135             "aliases": ["用户交易汇总表", "交易统计表"],
136             "columns": {
137                 "user_id": {
138                     "description": "用户唯一标识，用于关联用户交易汇总信息与用户利率
    信息。",
139                     "aliases": ["用户ID", "客户ID"],
140                     "semantic_role": "join_key",
141                     "business_usage": "用于表关联。",
142                     "keyword_text": "user_id 用户ID 客户ID 用户唯一标识 关联字段",
143                     "vector_text": (
144                         "字段名：user_id。所属表：trade_summary。"
145                         "该字段表示用户唯一标识，用于关联用户交易汇总信息与用户利率信
    息。"
146                     ),
147                     "rerank_text": (
148                         "字段：trade_summary.user_id。"
149                         "含义：用户唯一标识。"
150                         "角色：join_key。"
151                         "用途：用于关联 trade_summary 和 interest_info。"
152                     ),
153                 },
154                 "total_trade_count": {
155                     "description": "用户累计交易总笔数，表示用户在统计周期内完成的交
    易次数。",
156                     "aliases": ["总交易笔数", "累计交易笔数", "交易笔数", "交易次
    数"],
157                     "semantic_role": "metric_filter",
158                     "value_range": "整数，>= 0",
159                     "data_distribution": "多数用户集中在0-50区间，高频交易用户可能超
    过50000。",
160                     "business_usage": "用于衡量用户交易活跃程度，筛选高频交易用户。",
```

```
                    "samples": ["0", "12", "5800", "56000", "102430"],
                    "keyword_text": (
                        "total_trade_count 总交易笔数 累计交易笔数 交易笔数 "
                        "交易次数 高频交易 大于50000 用户交易汇总表 交易统计表"
                    ),
                    "vector_text": (
                        "字段名：total_trade_count。所属表：trade_summary，"
                        "该表记录用户交易汇总信息。该字段表示用户在统计周期内的累计交易总笔数，"
                        "用于衡量用户交易活跃程度，可用于筛选高频交易用户。"
                    ),
                    "rerank_text": (
                        "字段：trade_summary.total_trade_count。"
                        "含义：用户累计交易总笔数，表示用户在统计周期内完成的交易次数。"
                        "别名：总交易笔数、累计交易笔数、交易笔数、交易次数。"
                        "角色：metric_filter。"
                        "取值范围：整数，>= 0。"
                        "数据分布：高频交易用户可能超过50000。"
                        "用途：用于筛选总交易笔数大于某个阈值的用户。"
                    ),
                },
                "total_trade_amount": {
                    "description": "用户累计交易金额，表示用户在统计周期内完成交易的总金额。",
                    "aliases": ["交易金额", "累计交易金额", "成交金额", "交易总额"],
                    "semantic_role": "metric",
                    "value_range": ">= 0",
                    "business_usage": "用于衡量用户交易价值、统计交易金额或识别高价值用户。",
                    "samples": ["0.00", "5300.50", "230000.00", "1800000.00"],
                    "keyword_text": (
                        "total_trade_amount 交易金额 累计交易金额 成交金额 交易总额 "
                        "交易规模 高价值用户"
                    ),
                    "vector_text": (
                        "字段名：total_trade_amount。所属表：trade_summary。"
                        "该字段表示用户在统计周期内完成交易的累计金额，"
                        "用于衡量用户交易价值或交易规模。"
                    ),
                    "rerank_text": (
                        "字段：trade_summary.total_trade_amount。"
                        "含义：用户累计交易金额，表示交易总额或成交金额。"
                        "角色：metric。"
                        "用途：统计金额，不表示交易笔数。"
                    ),
```

```
                },
                "active_days": {
                    "description": "用户在统计周期内发生交易的活跃天数。",
                    "aliases": ["活跃天数", "交易活跃天数"],
                    "semantic_role": "metric",
                    "business_usage": "用于衡量用户交易频率和持续活跃程度。",
                    "keyword_text": "active_days 活跃天数 交易活跃天数 交易频率 持续活跃",
                    "vector_text": (
                        "字段名：active_days。所属表：trade_summary。"
                        "该字段表示用户在统计周期内发生交易的活跃天数，用于衡量交易频率。"
                    ),
                    "rerank_text": (
                        "字段：trade_summary.active_days。"
                        "含义：用户发生交易的活跃天数。"
                        "角色：metric。"
                        "用途：衡量交易频率，不表示交易笔数或利率。"
                    ),
                },
                "last_trade_time": {
                    "description": "用户最近一次交易发生时间。",
                    "aliases": ["最近交易时间", "最后交易时间"],
                    "semantic_role": "time",
                    "business_usage": "用于判断用户最近活跃情况。",
                    "keyword_text": "last_trade_time 最近交易时间 最后交易时间 交易时间 最近活跃",
                    "vector_text": (
                        "字段名：last_trade_time。所属表：trade_summary。"
                        "该字段表示用户最近一次交易发生时间，用于判断用户最近活跃情况。"
                    ),
                    "rerank_text": (
                        "字段：trade_summary.last_trade_time。"
                        "含义：最近一次交易时间。"
                        "角色：time。"
                        "用途：时间筛选，不表示交易笔数或利率。"
                    ),
                },
            },
        },
        "interest_info": {
            "description": "该表记录用户利率信息，包括用户利率类型、年化利率数值以及利率生效状态。",
            "aliases": ["用户利率表", "利率信息表"],
            "columns": {
                "user_id": {
```

```
                    "description": "用户唯一标识，用于关联交易汇总表。",
                    "aliases": ["用户ID", "客户ID"],
                    "semantic_role": "join_key",
                    "business_usage": "用于表关联。",
                    "keyword_text": "user_id 用户ID 客户ID 用户唯一标识 关联字段",
                    "vector_text": (
                        "字段名：user_id。所属表：interest_info。"
                        "该字段表示用户唯一标识，用于关联交易汇总表。"
                    ),
                    "rerank_text": (
                        "字段：interest_info.user_id。"
                        "含义：用户唯一标识。"
                        "角色：join_key。"
                        "用途：用于关联 trade_summary 和 interest_info。"
                    ),
                },
                "interest_rate": {
                    "description": "用户对应的年化利率数值。",
                    "aliases": ["利率", "年化利率", "用户利率", "利率数值"],
                    "semantic_role": "output_metric",
                    "value_range": "0-100",
                    "business_usage": "用于展示或分析用户当前适用的年化利率。",
                    "samples": ["2.35", "3.12", "4.58", "4.95"],
                    "keyword_text": (
                        "interest_rate 利率 年化利率 用户利率 利率数值 "
                        "利率是多少 用户利率表 利率信息表"
                    ),
                    "vector_text": (
                        "字段名：interest_rate。所属表：interest_info，"
                        "该表记录用户利率信息。该字段表示用户当前对应的年化利率数值，"
                        "通常作为查询结果输出。"
                    ),
                    "rerank_text": (
                        "字段：interest_info.interest_rate。"
                        "含义：用户对应的年化利率数值。"
                        "别名：利率、年化利率、用户利率、利率数值。"
                        "角色：output_metric。"
                        "用途：当用户询问利率是多少时，通常作为查询输出字段。"
                    ),
                },
                "rate_type": {
                    "description": "用户适用的利率类型，例如standard、vip、
    high_value。",
                    "aliases": ["利率类型", "费率类型"],
                    "semantic_role": "dimension",
                    "business_usage": "用于区分不同用户分层下的利率类型。",
```

```
                    "keyword_text": "rate_type 利率类型 费率类型 standard vip high_value",
                    "vector_text": (
                        "字段名：rate_type。所属表：interest_info。"
                        "该字段表示用户适用的利率类型，用于区分不同用户分层下的利率。"
                    ),
                    "rerank_text": (
                        "字段：interest_info.rate_type。"
                        "含义：利率类型或费率类型。"
                        "角色：dimension。"
                        "用途：分类字段，不是利率数值本身。"
                    ),
                },
                "effective_status": {
                    "description": "利率生效状态，例如active、inactive。",
                    "aliases": ["生效状态", "利率状态"],
                    "semantic_role": "filter",
                    "business_usage": "用于筛选当前有效或无效的利率记录。",
                    "keyword_text": "effective_status 生效状态 利率状态 active inactive 有效 无效",
                    "vector_text": (
                        "字段名：effective_status。所属表：interest_info。"
                        "该字段表示利率生效状态，用于筛选当前有效或无效的利率记录。"
                    ),
                    "rerank_text": (
                        "字段：interest_info.effective_status。"
                        "含义：利率生效状态。"
                        "角色：filter。"
                        "用途：筛选有效或无效利率，不是利率数值。"
                    ),
                },
            },
        },
    }

def enrich_columns_with_business_meta(columns: list, business_meta: dict) -> None:
    """
    将业务元数据补充回 ColumnSchema。
    """
    for column in columns:
        table_meta = business_meta.get(column.table_name, {})
        column_meta = table_meta.get("columns", {}).get(column.column_name, {})

        if not column_meta:
            continue

```

```
        column.description = column_meta.get("description", getattr(column, "description", ""))
        column.aliases = column_meta.get("aliases", getattr(column, "aliases", []))
        column.semantic_role = column_meta.get("semantic_role", getattr(column, "semantic_role", ""))
        column.value_range = column_meta.get("value_range", getattr(column, "value_range", ""))
        column.data_distribution = column_meta.get(
            "data_distribution",
            getattr(column, "data_distribution", ""),
        )
        column.business_usage = column_meta.get(
            "business_usage",
            getattr(column, "business_usage", ""),
        )

        if "samples" in column_meta:
            column.samples = [str(item) for item in column_meta["samples"]]

def build_hybrid_documents(columns: list, business_meta: dict) -> List[HybridFieldDocument]:
    """
    构建混合检索字段文档。
    """
    documents: List[HybridFieldDocument] = []

    for column in columns:
        table_meta = business_meta.get(column.table_name, {})
        column_meta = table_meta.get("columns", {}).get(column.column_name, {})

        keyword_text = column_meta.get("keyword_text") or build_default_keyword_text(column)
        vector_text = column_meta.get("vector_text") or build_default_vector_text(column)
        rerank_text = column_meta.get("rerank_text") or build_default_rerank_text(column)

        documents.append(
            HybridFieldDocument(
                doc_id=column.full_name,
                column=column,
                keyword_text=keyword_text,
                vector_text=vector_text,
                rerank_text=rerank_text,
            )
        )
```

```
373
374         return documents
375
376     def build_default_keyword_text(column: object) -> str:
377         """
378         构建默认关键词索引文本。
379         """
380         parts = [
381             getattr(column, "database", ""),
382             getattr(column, "table_name", ""),
383             getattr(column, "column_name", ""),
384             getattr(column, "data_type", ""),
385             getattr(column, "description", ""),
386             " ".join(getattr(column, "aliases", []) or []),
387             " ".join(getattr(column, "samples", [])[:5] or []),
388         ]
389
390         return "\n".join(part for part in parts if part)
391
392     def build_default_vector_text(column: object) -> str:
393         """
394         构建默认向量索引文本。
395         """
396         parts = [
397             f"字段名: {getattr(column, 'column_name', '')}",
398             f"所属表: {getattr(column, 'table_name', '')}",
399             f"字段含义: {getattr(column, 'description', '')}",
400         ]
401
402         aliases = getattr(column, "aliases", []) or []
403         if aliases:
404             parts.append(f"字段别名: {', '.join(aliases)}")
405
406         business_usage = getattr(column, "business_usage", "")
407         if business_usage:
408             parts.append(f"业务用途: {business_usage}")
409
410         semantic_role = getattr(column, "semantic_role", "")
411         if semantic_role:
412             parts.append(f"字段角色: {semantic_role}")
413
414         return "\n".join(part for part in parts if part)
415
416     def build_default_rerank_text(column: object) -> str:
417         """
418         构建默认 Rerank 文本。
419
```

```
420         Rerank 文本可以比 keyword_text 和 vector_text 更详细。
421         """
422         parts = [
423             f"字段: {getattr(column, 'table_name', '')}.{getattr(column,
    'column_name', '')}",
424             f"数据类型: {getattr(column, 'data_type', '')}",
425             f"字段含义: {getattr(column, 'description', '')}",
426         ]
427 
428         aliases = getattr(column, "aliases", []) or []
429         if aliases:
430             parts.append(f"字段别名: {', '.join(aliases)}")
431 
432         semantic_role = getattr(column, "semantic_role", "")
433         if semantic_role:
434             parts.append(f"字段角色: {semantic_role}")
435 
436         value_range = getattr(column, "value_range", "")
437         if value_range:
438             parts.append(f"取值范围: {value_range}")
439 
440         data_distribution = getattr(column, "data_distribution", "")
441         if data_distribution:
442             parts.append(f"数据分布: {data_distribution}")
443 
444         business_usage = getattr(column, "business_usage", "")
445         if business_usage:
446             parts.append(f"业务用途: {business_usage}")
447 
448         samples = getattr(column, "samples", []) or []
449         if samples:
450             parts.append(f"样例值: {', '.join(samples[:8])}")
451 
452         return "\n".join(part for part in parts if part)
453 
454     def build_route_results(
455         keywords: List[str],
456         keyword_index: BM25Index,
457         vector_index: VectorIndex,
458         per_keyword_top_k: int,
459     ) -> List[RouteRecallResult]:
460         """
461         执行关键词召回和向量召回。
462         """
463         route_results: List[RouteRecallResult] = []
464 
465         for keyword in keywords:
```

```
            keyword_hits = keyword_index.search(
                keyword,
                top_k=per_keyword_top_k,
            )

            route_results.append(
                RouteRecallResult(
                    route_name="keyword",
                    query_term=keyword,
                    ranked_doc_indices=[
                        doc_index
                        for doc_index, _ in keyword_hits
                    ],
                )
            )

            vector_hits = vector_index.search(
                keyword,
                top_k=per_keyword_top_k,
            )

            route_results.append(
                RouteRecallResult(
                    route_name="vector",
                    query_term=keyword,
                    ranked_doc_indices=[
                        doc_index
                        for doc_index, _ in vector_hits
                    ],
                )
            )

    return route_results

def calculate_rerank_top_n(
    keyword_count: int,
    multiplier: int,
    min_top_n: int,
    max_top_n: int,
    candidate_count: int,
) -> int:
    """
    根据关键词数量动态计算 Rerank 输出数量。

    rerank_top_n = keyword_count * multiplier
    """
    dynamic_top_n = keyword_count * multiplier
```

```
513
514         top_n = max(
515             min_top_n,
516             min(dynamic_top_n, max_top_n),
517         )
518
519         return min(top_n, candidate_count)
520
521     def main() -> None:
522         db_path = create_demo_database()
523         business_meta = get_business_meta()
524
525         print("=" * 80)
526         print("混合检索 + RRF 融合 + Rerank + Schema 图 Demo")
527         print("=" * 80)
528         print(f"Demo database: {db_path}")
529
530         loader = SQLiteSchemaLoader(
531             db_path=db_path,
532             database_name="demo_finance",
533             business_meta=business_meta,
534             sample_size=5,
535         )
536
537         tables, columns, relations = loader.load()
538         enrich_columns_with_business_meta(columns, business_meta)
539
540         documents = build_hybrid_documents(
541             columns=columns,
542             business_meta=business_meta,
543         )
544
545         print("\n[1] Hybrid documents:")
546         for index, doc in enumerate(documents):
547             print(f"{index}. {doc.doc_id}")
548             print(f"   keyword_text: {doc.keyword_text}")
549             print(f"   vector_text : {doc.vector_text}")
550             print(f"   rerank_text : {doc.rerank_text}")
551
552         # ------------------------------------------------------------------------
553         # 构建关键词索引
554         # ------------------------------------------------------------------------
555         keyword_texts = [
556             doc.keyword_text
557             for doc in documents
558         ]
559
```

```
560     keyword_index = BM25Index(keyword_texts)
561 
562     # ------------------------------------------------------------------------
563     # 构建向量索引
564     # ------------------------------------------------------------------------
565     embedding_client = AliyunEmbeddingClient(
566         AliyunEmbeddingConfig(
567             api_key="",
568             embedding_url="https://dashscope.aliyuncs.com/compatible-
    mode/v1/embeddings",
569             model="text-embedding-v4",
570             dimensions=1024,
571         )
572     )
573 
574     vector_index = VectorIndex(embedding_client)
575 
576     vector_texts = [
577         doc.vector_text
578         for doc in documents
579     ]
580 
581     print("\n[2] Building vector index...")
582     vector_index.build(vector_texts)
583 
584     if embedding_client.config.api_key:
585         print("Vector index built by Aliyun Embedding API.")
586     else:
587         print("Vector index built by local mock embedding because
    DASHSCOPE_API_KEY is not set.")
588 
589     # ------------------------------------------------------------------------
590     # LLM 抽取关键词
591     # ------------------------------------------------------------------------
592     query = "查询总交易笔数大于50000的利率是多少"
593 
594     print("\n[3] Query:")
595     print(query)
596 
597     keyword_extractor = KeywordExtractor(
598         KeywordExtractorConfig(
599             api_key="",
600             chat_url="https://dashscope.aliyuncs.com/compatible-
    mode/v1/chat/completions",
601             model="qwen-plus",
602         )
603     )
```

```
604
605         keywords = keyword_extractor.extract(query)
606
607         print("\n[4] Extracted keywords:")
608         print(", ".join(keywords))
609
610         # ------------------------------------------------------------------------
611         # 可调参数位置
612         # ------------------------------------------------------------------------
613         per_keyword_top_k = 20
614
615         rrf_client = RRFFusionClient(
616             RRFFusionConfig(
617                 rrf_k=60,
618
619                 # RRF 候选截断:
620                 # fused_top_k = 关键词数量 * truncate_multiplier
621                 truncate_multiplier=6,
622
623                 min_fused_top_k=10,
624                 max_fused_top_k=50,
625                 final_top_k=20,
626
627                 route_weights={
628                     "keyword": 1.0,
629                     "vector": 1.0,
630                 },
631             )
632         )
633
634         rerank_top_multiplier = 2
635         """
636         Rerank 输出数量倍数。
637
638         这里就是你说的:
639             rerank_top_k = 关键词数量 * 2
640
641         可改为 3 或 4。
642         """
643
644         rerank_min_top_n = 2
645         rerank_max_top_n = 20
646
647         print("\n[5] Config:")
648         print(f"per_keyword_top_k = {per_keyword_top_k}")
649         print(f"rrf_truncate_multiplier = {rrf_client.config.truncate_multiplier}")
```

```
650         print(f"rrf_fused_top_k =
    {rrf_client.calculate_fused_top_k(len(keywords))}")
651         print(f"rrf_final_top_k = {rrf_client.config.final_top_k}")
652         print(f"rerank_top_multiplier = {rerank_top_multiplier}")
653         print(f"rerank_min_top_n = {rerank_min_top_n}")
654         print(f"rerank_max_top_n = {rerank_max_top_n}")
655 
656         # ------------------------------------------------------------------------
657         # 关键词召回 + 向量召回
658         # ------------------------------------------------------------------------
659         route_results = build_route_results(
660             keywords=keywords,
661             keyword_index=keyword_index,
662             vector_index=vector_index,
663             per_keyword_top_k=per_keyword_top_k,
664         )
665 
666         print("\n[6] Route recall results:")
667         for route_result in route_results:
668             print(
669                 f"route={route_result.route_name}, "
670                 f"term={route_result.query_term}, "
671                 f"ranked_doc_indices={route_result.ranked_doc_indices}"
672             )
673 
674         # ------------------------------------------------------------------------
675         # RRF 融合
676         # ------------------------------------------------------------------------
677         fusion_hits = rrf_client.fuse(
678             route_results=route_results,
679             keyword_count=len(keywords),
680         )
681 
682         print("\n[7] RRF fused results:")
683         for rank, hit in enumerate(fusion_hits, start=1):
684             doc = documents[hit.doc_index]
685             col = doc.column
686 
687             print(
688                 f"{rank}. {col.table_name}.{col.column_name} "
689                 f"rrf_score={hit.score:.6f} "
690                 f"sources={hit.sources} "
691                 f"matched_terms={hit.matched_terms} "
692                 f"best_rank_by_source={hit.best_rank_by_source}"
693             )
694 
695         # ------------------------------------------------------------------------
```

```
    # Rerank
    # ------------------------------------------------------------------------
    rerank_candidates = [
        RerankDocument(
            doc_index=hit.doc_index,
            text=documents[hit.doc_index].rerank_text,
        )
        for hit in fusion_hits
    ]

    rerank_top_n = calculate_rerank_top_n(
        keyword_count=len(keywords),
        multiplier=rerank_top_multiplier,
        min_top_n=rerank_min_top_n,
        max_top_n=rerank_max_top_n,
        candidate_count=len(rerank_candidates),
    )

    rerank_client = AliyunRerankClient(
        AliyunRerankConfig(
            api_key="",

            # qwen3-rerank 需要业务空间 ID。
            # 建议通过环境变量设置:
            # export DASHSCOPE_WORKSPACE_ID="你的WorkspaceId"
            workspace_id="",

            # 如果你想直接写完整 URL，也可以设置 rerank_url。
            rerank_url="",

            # 官方当前模型名为 qwen3-rerank。
            # 如果你账号里有 qwen-rerank 别名，可以改成 "qwen-rerank"。
            model="qwen3-rerank",
        )
    )

    rerank_results = rerank_client.rerank(
        query=query,
        documents=rerank_candidates,
        top_n=rerank_top_n,
    )

    print("\n[8] Rerank results:")
    for rank, result in enumerate(rerank_results, start=1):
        doc = documents[result.doc_index]
        col = doc.column

```

```
            print(
                f"{rank}. {col.table_name}.{col.column_name} "
                f"rerank_score={result.rerank_score:.6f}"
            )
            print(f"   rerank_text: {doc.rerank_text}")

        # ------------------------------------------------------------------------
        # Rerank 结果转 SchemaHit
        # ------------------------------------------------------------------------
        schema_hits: List[SchemaHit] = []

        for result in rerank_results:
            doc = documents[result.doc_index]

            schema_hits.append(
                SchemaHit(
                    doc_id=doc.doc_id,
                    score=result.rerank_score,
                    column=doc.column,
                    rerank_score=result.rerank_score,
                )
            )

        # ------------------------------------------------------------------------
        # 构建 Schema 图
        # ------------------------------------------------------------------------
        schema_graph = build_schema_graph(
            tables=tables,
            all_columns=columns,
            relations=relations,
            hits=schema_hits,
            include_join_columns=True,
        )

        print("\n[9] Schema graph prompt context:")
        print(schema_graph.to_prompt_context())

    if __name__ == "__main__":
        main()
```

测试指令与结果：

代码块

```
1  测试指令：
2  python -m schema_retrieval.hybrid_rerank_schema_demo
```

测试结果：

```
================================================================================
混合检索 + RRF 融合 + Rerank + Schema 图 Demo
================================================================================
Demo database: /tmp/text2sql_hybrid_rerank_schema_demo.db

[1] Hybrid documents:
0. demo_finance.interest_info.user_id
   keyword_text: user_id 用户ID 客户ID 用户唯一标识 关联字段
   vector_text : 字段名：user_id。所属表：interest_info。该字段表示用户唯一标识，用于关联交易汇总表。
   rerank_text : 字段：interest_info.user_id。含义：用户唯一标识。角色：join_key。用途：用于关联 trade_summary 和 interest_info。
1. demo_finance.interest_info.interest_rate
   keyword_text: interest_rate 利率 年化利率 用户利率 利率数值 利率是多少 用户利率表 利率信息表
   vector_text : 字段名：interest_rate。所属表：interest_info，该表记录用户利率信息。该字段表示用户当前对应的年化利率数值，通常作为查询结果输出。
   rerank_text : 字段：interest_info.interest_rate。含义：用户对应的年化利率数值。别名：利率、年化利率、用户利率、利率数值。角色：output_metric。用途：当用户询问利率是多少时，通常作为查询输出字段。
2. demo_finance.interest_info.rate_type
   keyword_text: rate_type 利率类型 费率类型 standard vip high_value
   vector_text : 字段名：rate_type。所属表：interest_info。该字段表示用户适用的利率类型，用于区分不同用户分层下的利率。
   rerank_text : 字段：interest_info.rate_type。含义：利率类型或费率类型。角色：dimension。用途：分类字段，不是利率数值本身。
3. demo_finance.interest_info.effective_status
   keyword_text: effective_status 生效状态 利率状态 active inactive 有效 无效
   vector_text : 字段名：effective_status。所属表：interest_info。该字段表示利率生效状态，用于筛选当前有效或无效的利率记录。
   rerank_text : 字段：interest_info.effective_status。含义：利率生效状态。角色：filter。用途：筛选有效或无效利率，不是利率数值。
4. demo_finance.trade_summary.user_id
   keyword_text: user_id 用户ID 客户ID 用户唯一标识 关联字段
   vector_text : 字段名：user_id。所属表：trade_summary。该字段表示用户唯一标识，用于关联用户交易汇总信息与用户利率信息。
   rerank_text : 字段：trade_summary.user_id。含义：用户唯一标识。角色：join_key。用途：用于关联 trade_summary 和 interest_info。
5. demo_finance.trade_summary.total_trade_count
   keyword_text: total_trade_count 总交易笔数 累计交易笔数 交易笔数 交易次数 高频交易 大于50000 用户交易汇总表 交易统计表
   vector_text : 字段名：total_trade_count。所属表：trade_summary，该表记录用户交易汇总信息。该字段表示用户在统计周期内的累计交易总笔数，用于衡量用户交易活跃程度，可用于筛选
```

```
高频交易用户。
   rerank_text : 字段：trade_summary.total_trade_count。含义：用户累计交易总笔数，表示用户在统计周期内完成的交易次数。别名：总交易笔数、累计交易笔数、交易笔数、交易次数。角色：metric_filter。取值范围：整数，>= 0。数据分布：高频交易用户可能超过50000。用途：用于筛选总交易笔数大于某个阈值的用户。
6. demo_finance.trade_summary.total_trade_amount
   keyword_text: total_trade_amount 交易金额 累计交易金额 成交金额 交易总额 交易规模 高价值用户
   vector_text : 字段名：total_trade_amount。所属表：trade_summary。该字段表示用户在统计周期内完成交易的累计金额，用于衡量用户交易价值或交易规模。
   rerank_text : 字段：trade_summary.total_trade_amount。含义：用户累计交易金额，表示交易总额或成交金额。角色：metric。用途：统计金额，不表示交易笔数。
7. demo_finance.trade_summary.active_days
   keyword_text: active_days 活跃天数 交易活跃天数 交易频率 持续活跃
   vector_text : 字段名：active_days。所属表：trade_summary。该字段表示用户在统计周期内发生交易的活跃天数，用于衡量交易频率。
   rerank_text : 字段：trade_summary.active_days。含义：用户发生交易的活跃天数。角色：metric。用途：衡量交易频率，不表示交易笔数或利率。
8. demo_finance.trade_summary.last_trade_time
   keyword_text: last_trade_time 最近交易时间 最后交易时间 交易时间 最近活跃
   vector_text : 字段名：last_trade_time。所属表：trade_summary。该字段表示用户最近一次交易发生时间，用于判断用户最近活跃情况。
   rerank_text : 字段：trade_summary.last_trade_time。含义：最近一次交易时间。角色：time。用途：时间筛选，不表示交易笔数或利率。

[2] Building vector index...
Vector index built by local mock embedding because DASHSCOPE_API_KEY is not set.

[3] Query:
查询总交易笔数大于50000的利率是多少

[4] Extracted keywords:
总交易笔数，交易笔数，利率

[5] Config:
per_keyword_top_k = 20
rrf_truncate_multiplier = 6
rrf_fused_top_k = 18
rrf_final_top_k = 20
rerank_top_multiplier = 2
rerank_min_top_n = 2
rerank_max_top_n = 20

[6] Route recall results:
route=keyword, term=总交易笔数, ranked_doc_indices=[5, 6, 7, 8, 1]
route=vector, term=总交易笔数, ranked_doc_indices=[5, 6, 7, 0, 4, 8, 1, 3, 2]
```

```
route=keyword, term=交易笔数, ranked_doc_indices=[5, 7, 6, 8, 1]
route=vector, term=交易笔数, ranked_doc_indices=[5, 7, 6, 8, 0, 4, 1, 3, 2]
route=keyword, term=利率, ranked_doc_indices=[1, 2, 3, 7]
route=vector, term=利率, ranked_doc_indices=[1, 3, 2, 4, 7, 0, 5, 6, 8]

[7] RRF fused results:
1. trade_summary.active_days rrf_score=0.095014 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 2, 'vector': 2}
2. interest_info.interest_rate rrf_score=0.093407 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 1, 'vector':1}
3. trade_summary.total_trade_count rrf_score=0.080499 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 1, 'vector': 1}
4. trade_summary.total_trade_amount rrf_score=0.078710 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 2, 'vector': 2}
5. trade_summary.last_trade_time rrf_score=0.076519 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'keyword': 4, 'vector': 4}
6. interest_info.effective_status rrf_score=0.061414 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'vector': 2, 'keyword': 3}
7. interest_info.rate_type rrf_score=0.060988 sources=['keyword', 'vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'vector': 3, 'keyword': 2}
8. interest_info.user_id rrf_score=0.046161 sources=['vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'vector': 4}
9. trade_summary.user_id rrf_score=0.046161 sources=['vector'] matched_terms=['交易笔数', '利率', '总交易笔数'] best_rank_by_source={'vector': 4}

[8] Rerank results:
1. trade_summary.total_trade_count rerank_score=2.800000
   rerank_text: 字段：trade_summary.total_trade_count。含义：用户累计交易总笔数，表示用户在统计周期内完成的交易次数。别名：总交易笔数、累计交易笔数、交易笔数、交易次数。角色：metric_filter。取值范围：整数，>= 0。数据分布：高频交易用户可能超过50000。用途：用于筛选总交易笔数大于某个阈值的用户。
2. trade_summary.active_days rerank_score=2.000000
   rerank_text: 字段：trade_summary.active_days。含义：用户发生交易的活跃天数。角色：metric。用途：衡量交易频率，不表示交易笔数或利率。
3. trade_summary.last_trade_time rerank_score=2.000000
   rerank_text: 字段：trade_summary.last_trade_time。含义：最近一次交易时间。角色：time。用途：时间筛选，不表示交易笔数或利率。
4. interest_info.interest_rate rerank_score=1.800000
   rerank_text: 字段：interest_info.interest_rate。含义：用户对应的年化利率数值。别名：利率、年化利率、用户利率、利率数值。角色：output_metric。用途：当用户询问利率是多少
```

时，通常作为查询输出字段。
5. trade_summary.total_trade_amount rerank_score=1.000000
   rerank_text: 字段：trade_summary.total_trade_amount。含义：用户累计交易金额，表示交易总额或成交金额。角色：metric。用途：统计金额，不表示交易笔数。
6. interest_info.effective_status rerank_score=1.000000
   rerank_text: 字段：interest_info.effective_status。含义：利率生效状态。角色：filter。用途：筛选有效或无效利率，不是利率数值。

[9] Schema graph prompt context:
数据库：demo_finance

表名：trade_summary
表格摘要：该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金额、活跃交易天数以及最近交易时间。
表别名：用户交易汇总表，交易统计表
主键：user_id
字段：
- 字段名：total_trade_count
  数据类型：INTEGER
  字段含义：用户累计交易总笔数，表示用户在统计周期内完成的交易次数。
  字段别名：总交易笔数，累计交易笔数，交易笔数，交易次数
  字段角色：metric_filter
  样例值：0，12，5800，56000，102430
  取值范围：整数，>= 0
  数据分布：多数用户集中在0-50区间，高频交易用户可能超过50000。
  业务用途：用于衡量用户交易活跃程度，筛选高频交易用户。
- 字段名：active_days
  数据类型：INTEGER
  字段含义：用户在统计周期内发生交易的活跃天数。
  字段别名：活跃天数，交易活跃天数
  字段角色：metric
  样例值：5，180，320，365，0
  业务用途：用于衡量用户交易频率和持续活跃程度。
- 字段名：last_trade_time
  数据类型：TEXT
  字段含义：用户最近一次交易发生时间。
  字段别名：最近交易时间，最后交易时间
  字段角色：time
  样例值：2024-05-01，2024-05-06，2024-05-10，2024-05-11
  业务用途：用于判断用户最近活跃情况。
- 字段名：total_trade_amount
  数据类型：REAL
  字段含义：用户累计交易金额，表示用户在统计周期内完成交易的总金额。
  字段别名：交易金额，累计交易金额，成交金额，交易总额
  字段角色：metric
  样例值：0.00，5300.50，230000.00，1800000.00
  取值范围：>= 0

```
  业务用途：用于衡量用户交易价值、统计交易金额或识别高价值用户。
- 字段名：user_id
  数据类型：INTEGER
  字段含义：用户唯一标识，用于关联用户交易汇总信息与用户利率信息。
  字段别名：用户ID，客户ID
  字段角色：join_key
  样例值：1001，1002，1003，1004，1005
  业务用途：用于表关联。

表名：interest_info
表格摘要：该表记录用户利率信息，包括用户利率类型、年化利率数值以及利率生效状态。
表别名：用户利率表，利率信息表
主键：user_id
字段：
- 字段名：interest_rate
  数据类型：REAL
  字段含义：用户对应的年化利率数值。
  字段别名：利率，年化利率，用户利率，利率数值
  字段角色：output_metric
  样例值：2.35，3.12，4.58，4.95
  取值范围：0-100
  业务用途：用于展示或分析用户当前适用的年化利率。
- 字段名：effective_status
  数据类型：TEXT
  字段含义：利率生效状态，例如active、inactive。
  字段别名：生效状态，利率状态
  字段角色：filter
  样例值：active，inactive
  业务用途：用于筛选当前有效或无效的利率记录。
- 字段名：user_id
  数据类型：INTEGER
  字段含义：用户唯一标识，用于关联交易汇总表。
  字段别名：用户ID，客户ID
  字段角色：join_key
  样例值：1001，1002，1003，1004，1005
  业务用途：用于表关联。

表关联关系：
- interest_info.user_id ↔ trade_summary.user_id
```

CoT思维链规划(cot_planning)

🔥 `cot_planning` 模块主要负责将用户需求与检索得到的 Schema 信息结合，转换为结构化、链式的 CoT 操作步骤，位于 Schema 检索与 SQL 生成之间。

该模块通过构建规划 Prompt，引导思考模型按照固定四元组格式输出执行计划。每个步骤包含数据库、处理对象、操作指令和输出目标，用于明确后续 SQL 生成所需的查询范围、处理顺序和结果要求。

主要功能包括：

- 定义 CoT 规划相关数据结构；
- 构建用户 Query、Schema 上下文和输出格式组成的 Prompt；
- 调用思考模型生成规划结果；
- 支持非流式和流式输出；
- 解析模型返回的四元组步骤；
- 为后续 SQL 生成提供结构化执行计划；
- 提供完整 CoT 规划流程的示例。

模块最终输出格式化的 CoT 规划结果，使复杂的自然语言查询能够被拆解为清晰、可执行的数据处理步骤。

目录结构

```
cot_planning/
├── __init__.py
├── objects.py              # CoT规划数据结构
├── prompt_builder.py       # Prompt构建
├── thinking_client.py      # 思考模型调用，支持流式输出
├── cot_planner.py          # CoT规划主流程，支持 plan 和 plan_stream
└── cot_planning_demo.py    # 流式输出 Demo
```

思考大模型服务/thinking_client.py

🔥 `thinking_model_client.py` 主要负责调用思考模型，将用户 Query 和检索得到的 SchemaGraph 转换为结构化 CoT 规划结果。

该文件同时支持非流式输出和流式输出。当未配置 API Key 时，可以使用本地 Mock 结果跑通流程，主要用于开发测试。

ThinkingModelConfig

🔥 `ThinkingModelConfig` 用于配置思考模型的调用方式。

主要参数如下：

- `api_key`：模型服务 API Key。未显式配置时，从环境变量 `DASHSCOPE_API_KEY` 中读取；
- `base_url`：模型接口地址，默认使用阿里云百炼 OpenAI 兼容接口；
- `model`：指定使用的思考模型，当前默认配置为 `qwen-plus`；
- `temperature`：控制模型输出随机性，默认值为 0；
- `timeout`：接口请求超时时间；
- `use_mock_when_no_api_key`：未配置 API Key 时是否启用本地 Mock；
- `mock_stream_delay`：Mock 流式输出时每段文本之间的延迟。

当前代码默认使用 `qwen-plus`，但该客户端采用 OpenAI Compatible 接口格式，因此可以根据实际部署环境替换为其他兼容模型，例如 DeepSeek、其他通义千问模型或自建模型服务。替换时需要同步修改 `model` 和 `base_url` 配置。

ThinkingModelClient

`ThinkingModelClient` 是思考模型客户端，负责接收 Prompt，并生成结构化 CoT 规划结果。

`generate()` 方法用于非流式生成。当配置了 API Key 时调用真实模型；未配置 API Key 且允许 Mock 时返回本地模拟结果。

`generate_stream()` 方法用于流式生成规划结果。该方法返回文本迭代器，可以边接收边输出模型生成内容。

`_call_model()` 方法用于调用真实模型的非流式接口，并从返回结果中提取完整文本。

`_call_model_stream()` 方法用于调用真实模型的流式接口，按照 SSE 格式逐行解析模型返回内容，并持续输出增量文本。

`_post_json()` 方法用于统一发送非流式 JSON 请求，并解析接口返回结果。

`_mock_generate()` 方法用于生成固定的模拟 CoT 结果。当 Prompt 中包含指定字段和表关系时，返回对应的查询规划；否则返回 Schema 信息不足的说明。

`_mock_generate_stream()` 方法用于模拟流式输出，将 Mock 结果按字符逐步返回，便于测试前端流式展示效果。

CoT规划链生成/prompt_builder.py，cot_planner.py

🔥 `prompt_builder.py` 主要负责构建 CoT 规划阶段使用的 Prompt，将用户 Query、SchemaGraph 上下文、输出格式要求和可选的 Few-shot 示例组织为完整输入，供思考模型生成结构化四元组规划结果。

`cot_planner.py` 主要负责执行 CoT 规划流程，位于 Schema 检索与 SQL 生成之间。

该模块接收用户 Query 和 SchemaGraph，先构建 CoT Prompt，再调用思考模型生成规划结果，并将模型输出解析为结构化的四元组步骤。

DEFAULT_OUTPUT_FORMAT_SPEC

`DEFAULT_OUTPUT_FORMAT_SPEC` 定义 CoT 规划结果的默认输出格式，要求模型：

- 仅输出步骤和四元组；
- 使用“步骤N：”标识每个步骤；
- 每个步骤只包含一个四元组；
- 四元组包含数据库、处理对象、操作指令和输出目标；
- 操作指令按照实际数据处理顺序进行描述。

当请求中未提供自定义输出格式时，系统会使用该默认规范。

CotPromptBuilder

`CotPromptBuilder` 是 CoT 规划 Prompt 构建器，负责将不同来源的输入内容按照固定顺序组合为完整 Prompt。

该类没有显式初始化参数，主要通过 `CotPlanningRequest` 接收以下内容：

- 用户自然语言 Query；
- SchemaGraph 转换后的 Schema 上下文；
- CoT 输出格式要求；
- 可选的 Few-shot 示例。

`build()` 方法用于构建完整的 CoT 规划 Prompt，主要包括：

- 添加 CoT 规划任务说明；
- 在存在 Few-shot 示例时添加示例内容；
- 使用请求中的输出格式，未提供时使用默认格式；
- 添加当前用户 Query 和 Schema 上下文；
- 将各部分按照顺序拼接为最终 Prompt。

`_build_task_instruction()` 方法用于构建 CoT 规划任务说明，主要规定：

- 四元组中各字段的含义；
- 处理对象需要包含相关表、字段和关联关系；
- 操作指令需要按照筛选、关联、聚合、分组、排序和计算顺序描述；
- 单个步骤只能访问一个数据库；
- 多数据库查询需要拆分为多个步骤；
- 所有表、字段和关联关系必须来自 Schema；
- Schema 信息不足时需要明确说明缺失内容；
- 规划阶段不直接生成 SQL。

`_build_few_shot_section()` 方法用于构建 Few-shot 示例部分。每个示例包含用户 Query、Schema 图上下文和对应的 CoT 输出，用于向模型展示预期的分析方式和输出格式。

`_build_input_section()` 方法用于构建当前任务的输入部分，将用户 Query 和 Schema 上下文分别放置在对应标题下，作为思考模型进行规划的主要依据。

CotPlanner

`CotPlanner` 是 CoT 规划服务类，负责组织 Prompt 构建、思考模型调用和结果解析。

初始化时接收以下参数：

- `prompt_builder`：CoT Prompt 构建器，用于组织用户 Query、Schema 上下文、输出格式和 Few-shot 示例；
- `thinking_client`：思考模型客户端，用于生成 CoT 规划结果。

未显式传入时，系统会使用默认的 `CotPromptBuilder` 和 `ThinkingModelClient`。

`plan()` 方法用于非流式生成 CoT 规划结果，主要包括：

- 将 SchemaGraph 转换为文本上下文；
- 构建 `CotPlanningRequest`；
- 生成完整 Prompt；
- 调用思考模型获取完整输出；
- 将模型输出解析为 `CotPlanningStep` 列表；
- 返回包含 Prompt、原始输出和结构化步骤的 `CotPlanningResult`。

`plan_stream()` 方法用于流式生成 CoT 规划结果。该方法会完成 Schema 转换和 Prompt 构建，并持续返回思考模型生成的文本片段。该方法本身不解析步骤，如需获得结构化结果，需要在外部收集完整文本后调用 `parse_steps()`。

`parse_steps()` 方法用于解析模型输出中的四元组步骤，提取以下内容：

- 步骤编号；
- 数据库；
- 处理对象；
- 操作指令；
- 输出目标。

该方法通过正则表达式匹配固定输出格式，并将每个步骤转换为 `CotPlanningStep`。如果模型输出格式不符合要求，则返回空列表，不中断主流程。

`_schema_to_text()` 方法用于将 SchemaGraph 转换为 Prompt 文本。输入可以是字符串，也可以是实现了 `to_prompt_context()` 方法的对象，否则会抛出类型异常。

`_clean()` 方法用于清理解析出的字段内容，去除首尾空格和多余逗号。

测试Demo/cot_planning_demo.py

cot_planning_demo.py

```python
from __future__ import annotations

import os
import sys

from .cot_planner import CotPlanner
from .thinking_client import ThinkingModelClient, ThinkingModelConfig

def build_demo_schema_context() -> str:
    """
    构造 Demo Schema 图文本。

    实际项目中，这里直接传入：
        schema_graph.to_prompt_context()
    """
    return """数据库：trade_db
表名：trade_summary
表格摘要：该表记录用户交易汇总信息，包括用户累计交易笔数、交易金额以及交易活跃度等内容。
字段：
- 字段名：total_trade_count
  数据类型：int
  字段含义：用户累计交易总笔数
  样例值：120，5800，56000，102430
  取值范围：>=0
- 字段名：user_id
  数据类型：int
  字段含义：用户唯一标识

表名：interest_info
表格摘要：该表记录用户利率信息，包括用户利率类型、利率数值以及利率生效状态等内容。
字段：
- 字段名：interest_rate
  数据类型：decimal
  字段含义：用户对应年化利率
  样例值：2.35，3.12，4.58
  取值范围：0-100
- 字段名：user_id
  数据类型：int
  字段含义：用户唯一标识

表关联关系：
- trade_summary.user_id ↔ interest_info.user_id；关联含义：通过用户ID关联用户交易汇总信息与用户利率信息"""
```

```python

def main() -> None:
    """
    运行流式 Demo。

    不配置 DASHSCOPE_API_KEY 时走 Mock 流式输出。
    配置后调用真实思考模型流式输出。

    示例:
        export DASHSCOPE_API_KEY="你的APIKey"
        export DASHSCOPE_COT_MODEL="qwen-plus"
        python -m cot_planning.cot_planning_demo
    """
    planner = CotPlanner(
        thinking_client=ThinkingModelClient(
            ThinkingModelConfig(
                api_key=os.getenv("DASHSCOPE_API_KEY", ""),
                model=os.getenv("DASHSCOPE_COT_MODEL", "qwen-plus"),
                temperature=0.0,
                use_mock_when_no_api_key=True,
                mock_stream_delay=0.01,
            )
        )
    )

    user_query = "查询总交易笔数大于50000的利率是多少"
    schema_context = build_demo_schema_context()

    print("=" * 80)
    print("CoT Planning Stream Output")
    print("=" * 80)

    chunks = []

    for chunk in planner.plan_stream(
        user_query=user_query,
        schema_graph=schema_context,
    ):
        print(chunk, end="", flush=True)
        chunks.append(chunk)

    print()

    raw_output = "".join(chunks)

    print("\n" + "=" * 80)
    print("Parsed Steps")
```

```
    print("=" * 80)

    steps = planner.parse_steps(raw_output)

    for step in steps:
        print(f"步骤{step.step_no}:")
        print(f"数据库: {step.database}")
        print(f"处理对象: {step.processing_objects}")
        print(f"操作指令: {step.operation_instruction}")
        print(f"输出目标: {step.output_target}")

    if not steps:
        print("未解析到结构化步骤，请检查模型输出格式。", file=sys.stderr)

if __name__ == "__main__":
    main()
```

测试指令与结果：

代码块

```
测试指令:
python -m cot_planning.cot_planning_demo

测试结果:
================================================================================
CoT Planning Stream Output
================================================================================
步骤1:
(
  数据库: trade_db,
  处理对象: trade_summary.total_trade_count, interest_info.interest_rate, trade_summary.user_id, interest_info.user_id, trade_summary.user_id ↔ interest_info.user_id,
  操作指令: 先在trade_summary表中筛选total_trade_count大于50000的记录，并获取对应user_id；再基于user_id关联interest_info表；最后获取对应的interest_rate,
  输出目标: interest_info.interest_rate
)

================================================================================
Parsed Steps
```

```
================================================================================
=
步骤1:
数据库: trade_db
处理对象: trade_summary.total_trade_count, interest_info.interest_rate,
trade_summary.user_id, interest_info.user_id, trade_summary.user_id ↔
interest_info.user_id
操作指令: 先在trade_summary表中筛选total_trade_count大于50000的记录，并获取对应
user_id；再基于user_id关联interest_info表；最后获取对应的interest_rate
输出目标: interest_info.interest_rate
```

SQL语句生成(sql_generation)

🔥 `sql_generation` 模块主要负责根据 CoT 规划结果生成可执行 SQL，位于 CoT 规划之后、数据库执行之前。

该模块首先解析 CoT 中的四元组步骤，提取数据库、处理对象、操作指令和输出目标，再从局部 Schema 中获取相关表、字段及关联关系，并将这些信息组织为 Coder 模型所需的 Prompt。随后调用代码生成模型生成 SQL，并按照统一的数据结构返回结果。

主要功能包括：

- 解析 CoT 四元组步骤；
- 提取当前步骤所需的局部 Schema；
- 构建 SQL 生成 Prompt；
- 调用 Coder 模型生成 SQL；
- 封装数据库名称、SQL 文本及生成过程信息；
- 支持 Mock 模式和真实模型调用；
- 提供完整 SQL 生成流程的示例。

目录结构

```
sql_generation/
├── __init__.py
├── objects.py              # SQL生成数据结构
├── cot_parser.py           # 正则解析CoT四元组
├── schema_store.py         # 局部Schema存储与提取
├── prompt_builder.py       # Coder Prompt构建
├── coder_client.py         # Coder模型调用
├── sql_generator.py        # SQL生成主流程
└── sql_generation_demo.py  # Demo
```

数据结构/objects.py

> 🔥 `objects.py` 主要用于定义 SQL 生成模块中的核心数据结构，包括 CoT 步骤、局部 Schema、SQL 生成请求以及 SQL 生成结果。
>
> 这些对象用于统一不同模块之间的数据传递格式，使 CoT 解析、Schema 裁剪、Prompt 构建、SQL 生成和后续数据库执行能够按照固定结构衔接。

CotStep

`CotStep` 用于表示 CoT 四元组中的单个规划步骤。

主要参数如下：

- `database`：当前步骤需要访问的数据库，主要用于后续数据库路由；
- `processing_objects`：当前步骤涉及的表、字段和关联对象；
- `operation_instruction`：当前步骤需要执行的数据处理逻辑；
- `output_target`：当前步骤最终需要返回的字段或计算结果。

SQL 生成 Prompt 主要使用处理对象、操作指令和输出目标，数据库名称用于确定后续 SQL 的执行目标。

LocalSchema

`LocalSchema` 表示当前 CoT 步骤对应的局部 Schema，由完整 SchemaGraph 裁剪得到，只保留当前步骤涉及的表、字段和表间关系。

主要参数如下：

- `tables`：局部表集合，以表名为键、`TableSchema` 为值；
- `relations`：局部表之间的关联关系列表。

`to_prompt_context()` 方法用于将结构化的局部 Schema 转换为 SQL 生成 Prompt 中使用的文本，主要包括：

- 当前步骤涉及的表名；
- 每张表中的相关字段；
- 字段业务描述和数据类型；
- 表与表之间的关联关系。

当字段缺少描述或数据类型时，会分别使用“无描述”和 `unknown` 作为默认值。

SqlGenerationRequest

`SqlGenerationRequest` 用于封装单次 SQL 生成所需的输入信息。

主要参数如下：

- `cot_step`：当前需要执行的 CoT 步骤；
- `local_schema`：当前步骤对应的局部 Schema；
- `sql_dialect`：目标 SQL 方言，默认使用标准 SQL。

SqlGenerationResult

`SqlGenerationResult` 用于保存 SQL 生成阶段的完整结果。

主要参数如下：

- `database`：SQL 对应的目标数据库；
- `prompt`：发送给 Coder 模型的完整 Prompt；
- `raw_output`：模型返回的原始内容；
- `sql`：从模型输出中提取出的最终 SQL。

`to_execution_request()` 方法用于将生成结果转换为统一的数据库执行请求，格式为：

代码块

```
{
  "database": "目标数据库",
  "sql": "生成的SQL"
}
```

该结果可以交给后续 MCP 路由或数据库执行模块处理。

局部Schema裁剪/schema_store.py

🔥 `schema_store.py` 主要负责保存 SQL 生成阶段使用的 Schema 信息，并根据当前 CoT 步骤裁剪出对应的局部 Schema。

SQL 生成阶段不会直接使用完整 Schema，而是从 CoT 中提取涉及的表、字段和关联关系，仅保留当前步骤需要的内容，以减少 Prompt 中的无关信息。

当前实现使用内存对象保存 Schema，后续可以替换为 Milvus、SchemaGraph、元数据服务或其他 Schema 存储方式。

LocalSchemaStore

`LocalSchemaStore` 是局部 Schema 存储与提取类，负责保存完整的表结构和表关联关系，并根据 CoT 步骤生成 `LocalSchema`。

初始化时接收以下参数：

- `tables`：完整的表级 Schema 集合，以表名为键；

- `relations`：完整的表间关联关系列表。

`build_demo_store()` 方法用于构造测试环境中的示例 Schema，包括交易汇总表、利率信息表及两张表之间的用户 ID 关联关系。该方法主要用于 Demo，不参与正式 Schema 检索流程。

`from_schema_graph()` 方法用于将 Schema 检索阶段生成的 `SchemaGraph` 转换为 `LocalSchemaStore`，主要包括：

- 读取 SchemaGraph 中的相关表；
- 将字段转换为 `TableColumn`；
- 将表信息转换为 `TableSchema`；
- 将表间关系转换为 SQL 生成模块使用的 `TableRelation`；
- 保存为本地 Schema 存储对象。

`extract_local_schema()` 方法用于根据当前 CoT 四元组裁剪局部 Schema，主要包括：

- 合并处理对象、操作指令和输出目标；
- 从文本中提取 `table.column` 格式的字段；
- 识别文本中出现的表名；
- 根据命中表查找相关表关系；
- 自动补充 Join 两端的关联字段；
- 仅保留当前步骤涉及的表和字段；
- 返回包含局部表结构和关联关系的 `LocalSchema`。

如果 CoT 中只提到表名，没有明确指定字段，则会保留该表的全部字段，避免因字段信息不足影响 SQL 生成。

`_extract_table_columns()` 方法用于通过正则表达式从 CoT 文本中提取 `table.column` 格式的字段，并过滤掉本地 Schema 中不存在的表。

`_extract_table_names()` 方法用于检查 CoT 文本中出现的表名，并返回有效表名集合。

`_find_relations_by_tables()` 方法用于根据命中的表集合查找相关关联关系。该方法会对表名进行两两组合，并保留关系两端都出现在当前表集合中的 `TableRelation`。

Coder大模型服务端/coder_client.py

🔥 `coder_client.py` 主要负责调用 Coder 模型，根据 SQL 生成 Prompt 生成对应的 SQL 语句，并对模型输出进行清理和格式化。

CoderModelConfig

`CoderModelConfig` 用于配置 Coder 模型的调用参数。

主要参数如下：

- `api_key`：模型服务 API Key。未显式配置时，从环境变量 `DASHSCOPE_API_KEY` 中读取；
- `base_url`：模型接口地址，默认使用阿里云百炼 OpenAI Compatible 接口；
- `model`：指定使用的 Coder 模型，当前默认配置为 `qwen-plus`；
- `temperature`：模型输出随机性，默认值为 0，以提高 SQL 生成结果的稳定性；
- `timeout`：接口请求超时时间；
- `use_mock_when_no_api_key`：未配置 API Key 时是否启用本地 Mock。

当前默认使用 `qwen-plus`，也可以根据实际部署环境替换为其他支持代码生成的模型，例如 DeepSeek、其他通义千问模型或自建模型服务。替换时需要同步调整 `model` 和 `base_url` 配置。

CoderModelClient

`CoderModelClient` 是 SQL 生成模型客户端，负责接收 Prompt、调用模型并返回 SQL 生成结果。

`generate_sql()` 方法用于生成 SQL，主要包括：

- 配置 API Key 时调用真实 Coder 模型；
- 未配置 API Key且允许 Mock 时返回本地模拟 SQL；
- 未配置真实服务且未启用 Mock 时抛出异常。

`_call_model()` 方法用于调用真实 Coder 模型，主要包括：

- 构造 Chat Completions 请求；
- 将 SQL 生成 Prompt 作为用户消息传入；
- 调用模型接口；
- 解析并返回模型生成的原始文本。

`_mock_generate()` 方法用于生成固定的模拟 SQL。当 Prompt 中包含示例表和字段时，返回对应的多表关联查询；否则返回简单的 `SELECT 1`。该方法仅用于流程验证，不能代表真实 SQL 生成效果。

`clean_sql()` 方法用于清理模型输出，只保留可执行 SQL，主要包括：

- 去除 Markdown SQL 代码块；
- 去除“输出SQL语句”“SQL语句”等文本前缀；
- 提取以 `SELECT` 或 `WITH` 开头的 SQL 内容；
- 在 SQL 末尾缺少分号时自动补充分号。

清理后的 SQL 将作为后续数据库执行模块的输入。

SQL代码生成/prompt_builder.py，sql_generator.py

🔥 `prompt_builder.py` 主要负责构建 SQL 生成阶段使用的 Prompt，将当前 CoT 步骤、局部 Schema 和 SQL 要求组织为完整输入，供 Coder 模型生成 SQL。

`sql_generator.py` 主要负责组织 SQL 生成的完整流程，根据 CoT 四元组提取当前步骤所需的局部 Schema，构建 Coder Prompt，调用模型生成 SQL，并返回标准化结果。

SqlPromptBuilder

`SqlPromptBuilder` 是 SQL 生成 Prompt 构建器，负责将 `SqlGenerationRequest` 转换为 Coder 模型可以直接使用的文本输入。

该类没有显式初始化参数，主要通过请求对象获取以下内容：

- 当前 CoT 步骤中的处理对象；
- 当前 CoT 步骤中的操作指令；
- 当前 CoT 步骤中的输出目标；
- 当前步骤对应的局部 Schema；
- SQL 方言要求。

`build()` 方法用于构建完整的 SQL 生成 Prompt，主要包括：

- 指定模型角色为 SQL 生成助手；
- 要求 SQL 严格基于当前 CoT 规划生成；
- 限制只能使用局部 Schema 中存在的表、字段和关联关系；
- 禁止编造不存在的 Schema 信息；
- 指定目标 SQL 方言；
- 要求只输出 SQL，不输出解释或 Markdown 代码块；
- 将局部 Schema 转换为文本后加入 Prompt。

数据库名称不会写入 Prompt，而是保留在 `CotStep.database` 中，供后续 MCP 或数据库执行模块进行路由。

SqlGenerator

`SqlGenerator` 是 SQL 生成服务类，负责根据单个 CoT 步骤生成对应的 SQL。

初始化时接收以下参数：

- `schema_store`：局部 Schema 存储对象，用于根据 CoT 步骤提取相关表、字段和关联关系；
- `prompt_builder`：SQL Prompt 构建器，未显式传入时使用默认的 `SqlPromptBuilder`；
- `coder_client`：Coder 模型客户端，未显式传入时使用默认的 `CoderModelClient`。

`generate()` 方法用于执行完整的 SQL 生成流程，主要包括：

- 根据 CoT 步骤从 `schema_store` 中提取局部 Schema；
- 构建 `SqlGenerationRequest`；
- 将 CoT、局部 Schema 和 SQL 方言组织为 Coder Prompt；
- 调用 Coder 模型生成 SQL；
- 清理模型输出，去除代码块和解释性内容；
- 返回包含数据库名称、Prompt、模型原始输出和最终 SQL 的 `SqlGenerationResult`。

其中，数据库名称不参与 SQL Prompt 构建，仅用于后续 MCP 路由和数据库执行。

测试Demo/sql_generation_demo.py

sql_generation_demo.py

```
from __future__ import annotations

import json
import os

from .coder_client import CoderModelClient, CoderModelConfig
from .cot_parser import CotStepParser
from .schema_store import LocalSchemaStore
from .sql_generator import SqlGenerator

def build_demo_cot_text() -> str:
    """
    构造 Demo CoT 四元组。

    这里模拟上一阶段 CoT Planning 的输出。
    """
    return """步骤1:
(
数据库: trade_db,
处理对象: trade_summary.total_trade_count，interest_info.interest_rate，
trade_summary.user_id，interest_info.user_id，trade_summary.user_id ↔
interest_info.user_id,
操作指令: 先在trade_summary表中筛选total_trade_count大于50000的记录，并获取对应
user_id；再基于user_id关联interest_info表；最后获取对应的interest_rate,
输出目标: interest_info.interest_rate
)"""

def main() -> None:
    """
    运行 Demo。

```

```
    不配置 DASHSCOPE_API_KEY 时走 Mock。
    配置后调用真实 Coder 模型。

    示例:
        export DASHSCOPE_API_KEY="你的APIKey"
        export DASHSCOPE_CODER_MODEL="qwen-plus"
        python -m sql_generation.sql_generation_demo
    """
    cot_text = build_demo_cot_text()

    parser = CotStepParser()
    cot_step = parser.parse_one(cot_text)

    schema_store = LocalSchemaStore.build_demo_store()

    generator = SqlGenerator(
        schema_store=schema_store,
        coder_client=CoderModelClient(
            CoderModelConfig(
                api_key=os.getenv("DASHSCOPE_API_KEY", ""),
                model=os.getenv("DASHSCOPE_CODER_MODEL", "qwen-plus"),
                temperature=0.0,
                use_mock_when_no_api_key=True,
            )
        )
    )

    result = generator.generate(cot_step)

    print("=" * 80)
    print("Coder Prompt")
    print("=" * 80)
    print(result.prompt)

    print("\n" + "=" * 80)
    print("SQL")
    print("=" * 80)
    print(result.sql)

    print("\n" + "=" * 80)
    print("Execution Request")
    print("=" * 80)
    print(
        json.dumps(
            result.to_execution_request(),
            ensure_ascii=False,
            indent=2,
```

```
        )
    )

if __name__ == "__main__":
    main()

```

测试指令与结果：

代码块

```
测试指令：
python -m sql_generation.sql_generation_demo

测试结果：
SQL
================================================================================
SELECT interest_info.interest_rate
FROM trade_summary
JOIN interest_info
  ON trade_summary.user_id = interest_info.user_id
WHERE trade_summary.total_trade_count > 50000;

================================================================================
Execution Request
================================================================================
{
  "database": "trade_db",
  "sql": "SELECT interest_info.interest_rate\nFROM trade_summary\nJOIN
interest_info\n  ON trade_summary.user_id = interest_info.user_id\nWHERE
trade_summary.total_trade_count > 50000;"
}
```

MCP工具调用执行SQL(mcp_router)

🔥 `mcp_router` 模块主要负责接收 SQL 生成模块输出的执行请求，并根据其中的数据库名称，将 SQL 路由到对应的数据库执行器。

该模块位于 SQL 生成之后、结果校验之前，主要作用是屏蔽不同数据库之间的连接和执行差异，为上层流程提供统一的 SQL 执行入口。

主要功能包括：

- 定义统一的 MCP 执行请求和执行结果数据结构；
- 根据 `database` 字段选择对应的数据库执行器；
- 调用数据库执行器执行 SQL；
- 返回执行状态、查询结果和异常信息；
- 支持 SQLite 数据库的本地执行；
- 为后续扩展 MySQL、PostgreSQL、ClickHouse 等数据库执行器提供统一接口。

模块接收的执行请求主要包含目标数据库和 SQL 语句：

路由完成后，SQL 会交由对应执行器运行，并将执行结果返回给后续校验模块。

目录结构

```
mcp_router/
├── __init__.py
├── objects.py           # MCP执行请求和执行结果
├── sqlite_executor.py   # SQLite MCP执行器
└── router.py            # MCP路由器
```

数据结构(objects.py)

🔥 `objects.py` 主要用于定义 MCP 路由执行模块中的请求和结果数据结构，统一 SQL 生成模块与数据库执行模块之间的数据传递格式。

MCPExecutionRequest

`MCPExecutionRequest` 用于表示一次 SQL 执行请求。

主要参数如下：

- `database`：目标数据库名称，用于路由到对应的数据库连接或执行器；
- `sql`：当前步骤需要执行的 SQL 语句。

MCPExecutionResult

`MCPExecutionResult` 用于保存 SQL 执行后的完整结果。

主要参数如下：

- `database`：实际执行 SQL 的数据库名称；
- `sql`：本次执行的 SQL 语句；
- `success`：SQL 是否执行成功；
- `columns`：查询结果中的字段名称；

- `rows`：查询返回的数据记录，每行使用字典表示；
- `row_count`：查询结果的总行数；
- `error`：执行失败时的错误信息，执行成功时为空。

`to_dict()` 方法用于将执行结果转换为可序列化字典，便于后续日志记录、接口返回和结果校验。

MCP路由(router.py)

> 🔥 `router.py` 主要负责根据执行请求中的数据库名称，将 SQL 路由到对应的数据库执行器，并返回统一的执行结果。
>
> 该文件为不同数据库提供统一的执行入口，使上层模块无需关注具体数据库的连接方式和执行实现。

MCPRouter

`MCPRouter` 是数据库执行路由器，负责维护数据库名称与执行器之间的映射关系，并根据请求选择对应执行器。

初始化时会创建 `executors` 字典，用于保存数据库名称和执行器对象之间的对应关系。

`register_executor()` 方法用于注册数据库执行器，主要参数包括：

- `database`：数据库名称，作为路由标识；
- `executor`：该数据库对应的执行器对象。

注册后，后续执行请求可以根据数据库名称找到对应执行器。

`execute()` 方法用于路由并执行 SQL 请求，主要包括：

- 接收 `MCPExecutionRequest` 对象或字典格式请求；
- 将字典请求转换为统一的 `MCPExecutionRequest`；
- 根据 `database` 查找对应执行器；
- 未找到执行器时返回失败结果和错误信息；
- 找到执行器后调用其 `execute()` 方法执行 SQL；
- 返回统一格式的 `MCPExecutionResult`。

SQL执行

> 🔥 `sqlite_executor.py` 主要负责执行路由到 SQLite 数据库的 SQL 请求，并将查询结果封装为统一的 `MCPExecutionResult`。
>
> 当前实现用于模拟 MCP 路由到数据库 API 后的执行过程。为保证 Demo 安全，在只读模式下仅允许执行 `SELECT` 和 `WITH` 查询。

SQLiteMCPExecutor

`SQLiteMCPExecutor` 是 SQLite 数据库执行器，负责连接指定 SQLite 数据库、执行 SQL 并返回查询结果。

初始化时接收以下参数：

- `database`：当前执行器对应的数据库名称，用于校验路由是否正确；
- `db_path`：SQLite 数据库文件路径；
- `timeout`：数据库连接和执行超时时间，默认值为 30 秒；
- `readonly`：是否启用只读模式，默认值为 `True`。

`execute()` 方法用于执行 SQL 请求，主要包括：

- 校验请求中的数据库名称是否与当前执行器一致；
- 在只读模式下检查 SQL 是否为 `SELECT` 或 `WITH` 查询；
- 连接 SQLite 数据库并执行 SQL；
- 提取查询结果中的字段名称和数据行；
- 将每行数据转换为字典；
- 返回执行状态、字段列表、结果数据和行数；
- 执行异常时返回失败状态和错误信息。

`_is_readonly_sql()` 方法用于判断 SQL 是否属于只读查询。该方法会先统一空白字符并转换为小写，再检查 SQL 是否以 `select` 或 `with` 开头。

AskData单次问数Pipeline(askdata_pipeline)

🔥 `askdata_pipeline` 模块用于编排单次问数场景下的端到端 Text2SQL 流程。

该模块接收一次用户自然语言 Query，依次完成 Schema 检索、CoT 规划、SQL 生成、MCP 路由执行和结果返回，用于验证各子模块能否按照统一数据结构完成完整调用。

主要功能包括：

- 创建和初始化测试数据库；
- 提供测试所需的业务 Schema 元数据；
- 提供本地关键词抽取和 Embedding 能力；
- 定义单次问数流程中的中间数据结构；
- 编排 Schema 检索、CoT 规划、SQL 生成和 SQL 执行；
- 提供独立的 MCP SQL 执行测试；
- 提供完整的端到端 Text2SQL Demo。

该模块当前面向单次 Query 的顺序执行，不包含多轮会话管理、结果校验回调和循环修正机制。

目录结构

```
askdata_pipeline/
├── __init__.py
├── demo_data.py              # 测试数据库创建与业务Schema元数据
├── create_demo_db.py         # 单独创建SQLite测试库
├── sql_execution_demo.py     # 最小MCP SQL执行Demo
├── local_clients.py          # 本地Mock关键词抽取和Embedding
├── objects.py                # Pipeline数据结构
├── text2sql_pipeline.py      # 端到端流程编排
└── end_to_end_demo.py        # 完整Text2SQL Demo
```

创建测试数据库/create_demo_db.py

🔥 `create_demo_db.py` 主要负责创建单次问数端到端测试所需的 SQLite 数据库，并验证测试数据和示例 SQL 是否可以正常执行。

该文件会读取预先编写的 SQL 脚本，重新创建本地数据库文件，并初始化以下两张测试表：

- `trade_summary`：用户交易汇总表；
- `interest_info`：用户利率信息表。

create_demo_db()

`create_demo_db()` 方法用于创建测试数据库，主要包括：

- 接收数据库文件路径和初始化 SQL 文件路径；
- 创建数据库文件所在目录；
- 删除已存在的旧测试数据库；
- 读取 SQL 脚本内容；
- 通过 SQLite 执行建表和测试数据插入脚本；
- 返回创建完成的数据库文件路径。

默认情况下，数据库生成在：

代码块

```
runtime_data/trade_demo.db
```

初始化 SQL 位于：

```
代码块
1  sql/create_trade_demo.sql
```

main()

`main()` 方法用于运行数据库创建和验证流程，主要包括：

- 调用 `create_demo_db()` 创建测试数据库；
- 输出 `trade_summary` 表中的测试数据；
- 输出 `interest_info` 表中的测试数据；
- 执行示例多表关联 SQL；
- 输出查询结果，验证数据库结构、关联关系和测试数据是否正确。

该文件主要用于准备和检查端到端 Demo 的本地数据环境。

执行结果

```
代码块
1  执行指令：
2  python -m askdata_pipeline.create_demo_db
3
4  会生成：
5
6  ```text
7  runtime_data/trade_demo.db
8  ```
9
10 测试数据 SQL 在：
11
12 ```text
13 sql/create_trade_demo.sql
14 ```
15
16 执行结果：
17 已创建测试数据库：/root/askdata_agent/runtime_data/trade_demo.db
18
19 trade_summary 数据：
20 {'user_id': 1001, 'total_trade_count': 12, 'total_trade_amount': 5300.5,
   'active_days': 5, 'last_trade_time': '2024-05-01'}
21 {'user_id': 1002, 'total_trade_count': 5800, 'total_trade_amount': 230000.0,
   'active_days': 180, 'last_trade_time': '2024-05-06'}
```

```
{'user_id': 1003, 'total_trade_count': 56000, 'total_trade_amount': 1800000.0, 'active_days': 320, 'last_trade_time': '2024-05-10'}
{'user_id': 1004, 'total_trade_count': 102430, 'total_trade_amount': 3700000.0, 'active_days': 365, 'last_trade_time': '2024-05-11'}
{'user_id': 1005, 'total_trade_count': 0, 'total_trade_amount': 0.0, 'active_days': 0, 'last_trade_time': None}
{'user_id': 1006, 'total_trade_count': 50001, 'total_trade_amount': 910000.0, 'active_days': 250, 'last_trade_time': '2024-05-13'}
{'user_id': 1007, 'total_trade_count': 49999, 'total_trade_amount': 870000.0, 'active_days': 245, 'last_trade_time': '2024-05-13'}
{'user_id': 1008, 'total_trade_count': 76000, 'total_trade_amount': 2500000.0, 'active_days': 330, 'last_trade_time': '2024-05-14'}
{'user_id': 1009, 'total_trade_count': 340, 'total_trade_amount': 35000.0, 'active_days': 40, 'last_trade_time': '2024-05-15'}
{'user_id': 1010, 'total_trade_count': 88888, 'total_trade_amount': 3200000.0, 'active_days': 350, 'last_trade_time': '2024-05-16'}

interest_info 数据:
{'user_id': 1001, 'interest_rate': 2.35, 'rate_type': 'standard', 'effective_status': 'active'}
{'user_id': 1002, 'interest_rate': 3.12, 'rate_type': 'vip', 'effective_status': 'active'}
{'user_id': 1003, 'interest_rate': 4.58, 'rate_type': 'high_value', 'effective_status': 'active'}
{'user_id': 1004, 'interest_rate': 4.95, 'rate_type': 'high_value', 'effective_status': 'active'}
{'user_id': 1005, 'interest_rate': 1.8, 'rate_type': 'standard', 'effective_status': 'inactive'}
{'user_id': 1006, 'interest_rate': 4.2, 'rate_type': 'high_value', 'effective_status': 'active'}
{'user_id': 1007, 'interest_rate': 3.88, 'rate_type': 'vip', 'effective_status': 'active'}
{'user_id': 1008, 'interest_rate': 5.15, 'rate_type': 'high_value', 'effective_status': 'inactive'}
{'user_id': 1009, 'interest_rate': 2.8, 'rate_type': 'standard', 'effective_status': 'active'}
{'user_id': 1010, 'interest_rate': 5.05, 'rate_type': 'high_value', 'effective_status': 'active'}

验证SQL结果:
{'interest_rate': 4.58}
{'interest_rate': 4.95}
{'interest_rate': 4.2}
{'interest_rate': 5.15}
{'interest_rate': 5.05
```

只测试SQL执行环境/sql_execution_demo.py

🔥 `sql_execution_demo.py` 用于验证单次问数流程中的最小 SQL 执行环境。

该文件不依赖大模型、Schema 检索或 CoT 规划，只测试 SQLite 数据库、MCP 路由和 SQL 执行器之间是否能够正常连接和执行。

main()

`main()` 方法用于运行完整的 SQL 执行测试，主要包括：

- 创建或重新初始化 SQLite 测试数据库；
- 创建 `MCPRouter` 路由器；
- 为 `trade_db` 注册 `SQLiteMCPExecutor`；
- 构造固定的多表关联 SQL；
- 将数据库名称和 SQL 封装为执行请求；
- 通过 MCP 路由器将请求发送到对应数据库执行器；
- 输出 MCP 执行请求和执行结果。

该 Demo 主要用于验证以下内容：

- SQLite 测试数据库是否可以正常创建；
- MCP 路由是否能够根据 `database` 定位执行器；
- SQLite 执行器是否可以正常执行只读 SQL；
- 执行结果是否能够转换为统一的数据结构。

执行结果

代码块

```
1  执行指令：
2  python -m askdata_pipeline.create_demo_db
3
4
5  执行结果：
6  ================================================================================
   =
7  MCP 执行请求
8  ================================================================================
   =
9  {
10   "database": "trade_db",
11   "sql": "\n    SELECT interest_info.interest_rate\n    FROM trade_summary\n
    JOIN interest_info\n      ON trade_summary.user_id = interest_info.user_id\n
```

```
    WHERE trade_summary.total_trade_count > 50000;\n    "
}

================================================================================
MCP 执行结果
================================================================================
{
  "database": "trade_db",
  "sql": "SELECT interest_info.interest_rate\n    FROM trade_summary\n    JOIN interest_info\n      ON trade_summary.user_id = interest_info.user_id\n
WHERE trade_summary.total_trade_count > 50000;",
  "success": true,
  "columns": [
    "interest_rate"
  ],
  "rows": [
    {
      "interest_rate": 4.58
    },
    {
      "interest_rate": 4.95
    },
    {
      "interest_rate": 4.2
    },
    {
      "interest_rate": 5.15
    },
    {
      "interest_rate": 5.05
    }
  ],
  "row_count": 5,
  "error": null
}
```

端到端单次问数Pipeline

🔥 `end_to_end_demo.py` 用于运行单次问数场景下的完整 Text2SQL 端到端流程。

该文件通过固定用户 Query 初始化 `AskDataText2SQLPipeline`，依次执行 Schema 检索、CoT 规划、SQL 生成、MCP 路由和数据库查询，并输出各阶段结果。

main()

`main()` 方法用于执行完整的单次问数 Demo，主要包括：

- 定义用户自然语言 Query；
- 配置目标数据库名称、SQLite 数据库路径和样例值数量；
- 初始化 `AskDataText2SQLPipeline`；
- 调用 `pipeline.run()` 执行完整流程；
- 输出 Schema 检索关键词；
- 输出检索得到的 Schema 图上下文；
- 输出 CoT 四元组规划结果；
- 逐步骤输出局部 Schema；
- 输出生成的 SQL；
- 输出 MCP 执行请求；
- 输出数据库执行结果。

该文件主要用于展示单次 Query 在整个 Text2SQL Pipeline 中的完整处理过程，并验证各模块之间的数据传递和调用是否正常。

执行结果

代码块

```
执行指令：
python -m askdata_pipeline.end_to_end_demo

执行结果：
================================================================================
1. 用户 Query
================================================================================
查询总交易笔数大于50000的利率是多少

================================================================================
2. Schema 检索关键词
================================================================================
总交易笔数，交易笔数，利率

================================================================================
```

```
3. Schema 图
================================================================================
数据库：trade_db

表名：trade_summary
表格摘要：该表记录用户交易汇总信息，包括用户累计交易笔数、累计交易金额、活跃交易天数以及最近交易时间。
表别名：用户交易汇总表，交易统计表
主键：user_id
字段：
- 字段名：total_trade_count
  数据类型：INTEGER
  字段含义：用户累计交易总笔数，表示用户在统计周期内完成的交易次数。
  字段别名：总交易笔数，累计交易笔数，交易笔数，交易次数
  字段角色：metric_filter
  样例值：0，12，5800，56000，102430
  取值范围：整数，>= 0
  数据分布：多数用户集中在0-50区间，高频交易用户可能超过50000。
  业务用途：用于衡量用户交易活跃程度，筛选高频交易用户。
- 字段名：active_days
  数据类型：INTEGER
  字段含义：用户活跃交易天数。
  字段别名：活跃天数，交易活跃天数
  字段角色：metric
  样例值：5，180，320，365，0
  业务用途：用于衡量用户交易活跃程度。
- 字段名：last_trade_time
  数据类型：TEXT
  字段含义：用户最近一次交易时间。
  字段别名：最近交易时间，最后交易时间
  字段角色：time
  样例值：2024-05-01，2024-05-06，2024-05-10，2024-05-11，2024-05-13
  业务用途：用于按最近交易时间筛选或排序。
- 字段名：user_id
  数据类型：INTEGER
  字段含义：用户唯一标识。
  字段别名：用户ID，客户ID
  字段角色：join_key
  样例值：1001，1002，1003，1004，1005
  业务用途：用于唯一标识交易汇总表中的用户。

表名：interest_info
表格摘要：该表记录用户利率信息，包括用户利率类型、利率数值以及利率生效状态等内容。
表别名：用户利率表，利率信息表
主键：user_id
字段：
```

```
- 字段名：interest_rate
  数据类型：REAL
  字段含义：用户对应年化利率。
  字段别名：利率，年化利率，用户利率，利率数值
  字段角色：output_metric
  样例值：2.35，3.12，4.58，4.95
  取值范围：0-100
  数据分布：不同用户分层对应不同利率。
  业务用途：用于展示或分析用户当前适用的年化利率。
- 字段名：rate_type
  数据类型：TEXT
  字段含义：利率类型。
  字段别名：利率类型，利率分层
  字段角色：dimension
  样例值：standard，vip，high_value
  业务用途：用于区分标准、VIP、高价值等利率类型。
- 字段名：effective_status
  数据类型：TEXT
  字段含义：利率生效状态。
  字段别名：生效状态，是否生效
  字段角色：filter
  样例值：active，inactive
  业务用途：用于筛选当前有效或无效的利率记录。
- 字段名：user_id
  数据类型：INTEGER
  字段含义：用户唯一标识。
  字段别名：用户ID，客户ID
  字段角色：join_key
  样例值：1001，1002，1003，1004，1005
  业务用途：用于唯一标识利率信息表中的用户。

表关联关系：
- interest_info.user_id ↔ trade_summary.user_id

==========
4. CoT 四元组
==========
步骤1：
(
  数据库：trade_db，
  处理对象：trade_summary.total_trade_count，interest_info.interest_rate，trade_summary.user_id，interest_info.user_id，trade_summary.user_id ↔ interest_info.user_id，
  操作指令：先在trade_summary表中筛选total_trade_count大于50000的记录，并获取对应user_id；再基于user_id关联interest_info表；最后获取对应的interest_rate，
```

```
  输出目标: interest_info.interest_rate
)

================================================================================
5.1 当前步骤局部 Schema
================================================================================
表名: interest_info
字段:
- interest_rate: 用户对应年化利率。, REAL
- user_id: 用户唯一标识。, INTEGER

表名: trade_summary
字段:
- total_trade_count: 用户累计交易总笔数，表示用户在统计周期内完成的交易次数。, INTEGER
- user_id: 用户唯一标识。, INTEGER

表关联关系:
interest_info.user_id ↔ trade_summary.user_id

================================================================================
6.1 SQL
================================================================================
SELECT interest_info.interest_rate
FROM trade_summary
JOIN interest_info
  ON trade_summary.user_id = interest_info.user_id
WHERE trade_summary.total_trade_count > 50000;

================================================================================
7.1 MCP 执行请求
================================================================================
{
  "database": "trade_db",
  "sql": "SELECT interest_info.interest_rate\nFROM trade_summary\nJOIN
interest_info\n  ON trade_summary.user_id = interest_info.user_id\nWHERE
trade_summary.total_trade_count > 50000;"
}

================================================================================
8.1 MCP 执行结果
```

```
================================================================================
=
{
  "database": "trade_db",
  "sql": "SELECT interest_info.interest_rate\nFROM trade_summary\nJOIN
interest_info\n  ON trade_summary.user_id = interest_info.user_id\nWHERE
trade_summary.total_trade_count > 50000;",
  "success": true,
  "columns": [
    "interest_rate"
  ],
  "rows": [
    {
      "interest_rate": 4.58
    },
    {
      "interest_rate": 4.95
    },
    {
      "interest_rate": 4.2
    },
    {
      "interest_rate": 5.15
    },
    {
      "interest_rate": 5.05
    }
  ],
  "row_count": 5,
  "error": null
}
```